

AN AUTOMATED POP SONG MASHUP SYSTEM

by

XINYANG WU

A Thesis Submitted to
The Hong Kong University of Science and Technology
in Partial Fulfillment of the Requirements for
the Degree of Doctor of Philosophy
in Computer Science and Engineering

August 2026, Hong Kong

Copyright © by Xinyang WU 2026

Authorization

I hereby declare that I am the sole author of the thesis.

I authorize the Hong Kong University of Science and Technology to lend this thesis to other institutions or individuals for the purpose of scholarly research.

I further authorize the Hong Kong University of Science and Technology to reproduce the thesis by photocopying or by other means, in total or in part, at the request of other institutions or individuals for the purpose of scholarly research.

A handwritten signature in black ink, appearing to read 'Xinyang Wu', written over a horizontal line.

XINYANG WU

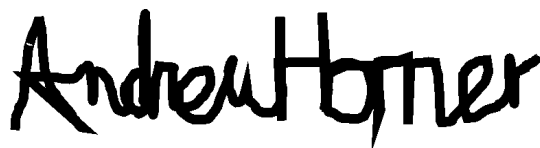
27 August 2026

AN AUTOMATED POP SONG MASHUP SYSTEM

by

XINYANG WU

This is to certify that I have examined the above Ph.D. thesis
and have found that it is complete and satisfactory in all respects,
and that any and all revisions required by
the thesis examination committee have been made.



Prof. Andrew Brian Horner, Thesis Supervisor



Prof. Xiaofang Zhou, Head of Department

Department of Computer Science and Engineering

27 August 2026

ACKNOWLEDGMENTS

I would like to express my deepest gratitude to my supervisor, Prof. Andrew Brian Horner, for his guidance, patience, and support throughout my doctoral studies. His insight into music and audio research shaped every stage of this work, from the earliest listening experiments to the final system design.

I thank the members of my Thesis Supervision Committee and the thesis examination committee for their time and their constructive feedback on this research.

I am grateful to the participants of the listening tests conducted in this work, and to my colleagues and fellow students in the Department of Computer Science and Engineering at HKUST for many helpful discussions.

Finally, I thank my family and friends for their unconditional support throughout the way.

TABLE OF CONTENTS

Title Page	i
Authorization Page	ii
Signature Page	iii
Acknowledgments	iv
Table of Contents	v
List of Figures	ix
List of Tables	xiii
Abstract	xv
Chapter 1 Introduction	1
1.1 Background	1
1.2 Related Work	2
1.2.1 Rule-Based Mashability Systems	2
1.2.2 Music Source Separation and Neural Network-Assisted Approaches	3
1.2.3 Gaps in Existing Work	4
1.3 Objective	5
1.4 System Overview	5
Chapter 2 Temporal Alignment for Mashup Generation	10
2.1 Framework Overview	10
2.2 Main Beat Pattern Extraction	11
2.2.1 Pre-processing	11
2.2.2 Spectral Flux Onset Detection	12
2.2.3 Kick/Snare Clustering	12
2.2.4 Main Pattern Selection	13
2.2.5 Intact Musical Phrase Extraction	13

2.2.6	Harmonic Mixing	14
2.2.7	Limitations	14
2.3	Global Tempo Alignment and Its Limitations	14
2.3.1	Tempo Estimation	14
2.3.2	The Cumulative Drift Problem	15
2.4	Local Beat Warping	16
2.4.1	Downbeat Detection Schemes	16
2.4.2	Downbeat Filtering and Normalization	17
2.4.3	Timemap Construction and Continuous Warping	17
2.5	Evaluation: Stem Swapping Experiments	18
2.5.1	Experiment 1: Drum Swapping	18
2.5.2	Experiment 2: Extended Multi-Strategy Stem Swapping	20
2.5.3	Learning to Rank Mashup Quality	21
2.6	Chapter Summary	25
Chapter 3	Adaptive Instrumental Rearrangement	28
3.1	Problem Formulation	28
3.2	Feature Extraction	30
3.2.1	Coarse Mel Design	30
3.2.2	Zero-Overlap Design Rationale	31
3.3	Model Architecture	32
3.3.1	Asymmetric Dual-Encoder Design	32
3.3.2	Matching and Assignment	34
3.4	Training	35
3.4.1	Self-Supervised Pair Generation	35
3.4.2	Dataset and Training Configuration	37
3.4.3	Multi-Scale Training as Compatibility-Encoder Learning	37
3.5	Results	38
3.5.1	Segment Placement Accuracy	38
3.5.2	Accuracy Across Scales	39
3.5.3	Ablation Studies	39
3.6	Application to Mashup Generation	41
3.6.1	Mashup Generation Process	41
3.6.2	Cross-Song Accompaniment Alignment	44
3.6.3	Multi-Window Selection	45

3.6.4	Compatibility Scoring	45
3.6.5	Analysis of the Section Scan	45
3.7	Subjective Listening Test	47
3.7.1	Experiment Setup	47
3.7.2	Evaluation Criteria	48
3.7.3	Results	48
3.8	Discussion	50
3.8.1	Same-Song Training and Cross-Song Application	50
3.8.2	Limitations	51
3.9	Chapter Summary	51
Chapter 4	Mix Engineering with Rectified Flow Models	53
4.1	Related Work	53
4.1.1	Rule-Based Automatic Mixing	53
4.1.2	Neural Network–Based Automatic Mixing	54
4.1.3	Diffusion and Flow Models for Audio	54
4.2	Problem Definition	55
4.3	Per-Stem Diffusion Restoration	56
4.3.1	Setup	56
4.3.2	Architecture	56
4.3.3	Formulation	57
4.3.4	Per-Stem System and Findings	59
4.3.5	Toward Real-World Mixing Issues	60
4.4	Data Preparation	61
4.4.1	Dataset	61
4.4.2	STFT Representation	61
4.4.3	Imbalanced Input Simulation	62
4.5	Model Formulation and Architecture	62
4.5.1	Phase Reuse: Predict Magnitude Only	62
4.5.2	Conditioning	64
4.5.3	UNet Backbone	65
4.6	Bridge-Flow Formulation	67
4.7	Automatic Mixing System	69
4.8	Evaluation	69
4.8.1	Baselines	69

4.8.2	Results	70
4.8.3	Conditioning Ablation	72
4.9	Discussion	73
4.9.1	Limitations	73
4.10	Chapter Summary	74
Chapter 5	Conclusions and Future Work	75
5.1	Enriching the Editing Toolkit	75
5.1.1	Content-Level Accompaniment Adaptation	75
5.1.2	Spatial Audio and Stereo Upmixing	76
5.2	From Segments to Full Compositions	77
5.2.1	Extending Beyond Fixed-Length Segments	77
5.2.2	Agent-Based Pipeline Orchestration	77
5.2.3	Generative vs. Editing-Based Mashup Paradigms	79
5.2.4	Toward Interactive Vibe Mixing	81
	List of Publications	82
	Bibliography	83

LIST OF FIGURES

1.1	Overview of the automated mashup pipeline. Two source songs are separated into stems, aligned on a downbeat-accurate grid, matched and rearranged by the multi-scale reordering family, and integrated by the flow-based mixing system. Shaded components are proposed in this thesis; unshaded components are adopted from prior work. The perceptual ranking model evaluates the combined outputs of the musical compatibility stage.	7
2.1	Main beat pattern extraction. Detected onsets are classified as kick (K) or snare (S). The most common kick-to-snare delay identifies structurally paired beats, and the segment pair with the highest cross-correlation of onset sequences is selected as the main beat pattern (red box).	11
2.2	The Camelot wheel, a circular representation of key compatibility based on the circle of fifths. Adjacent keys on the wheel are harmonically compatible. The system selects the target key (same key or perfect fifth) that minimizes the number of semitone shifts required.	14
2.3	Drum swapping mashup creation process. The base song’s vocals, bass, and other instrumentals are retained, while the drum stem is replaced with the secondary song’s drum pattern, time-stretched to match the base song’s tempo.	19
2.4	Web survey layout for the listening test. Each of the 10 pages corresponds to one base song. Participants first listen to the original version, then hear 9 mashup variations and select any they prefer over the original by clicking the selection box (highlighted in blue).	19
2.5	Learning to rank mashup quality. <i>Training</i> (top): the context stem B_i and two candidate stems A_j, A_k are embedded separately by one shared frozen OpenL3 extractor, placed in session-aware blocks, and formed into utility-difference pairs labeled by the listening-test ratings; a pairwise logistic (Bradley–Terry) ranker is trained on the resulting comparisons. <i>Inference</i> (bottom): each candidate receives a utility conditioned on the vocal, and sorting utilities yields the predicted ranking, evaluated against listeners by Spearman’s ρ .	22

- 2.6 Behavior of the utility ranker on out-of-sample predictions (all 40 leave-one-context-out folds). (a) The pairwise preference classifier separates preferred from non-preferred candidates well (AUC 0.89 overall, 0.93 on the trained pairs, and 0.64 on the excluded adjacent-rank pairs). (b) Predicted probabilities are well calibrated against empirical outcomes. (c) The model’s confidence in the listener-preferred item grows monotonically with the listeners’ rating gap, from 0.59 at gap 1 to 0.95 at gap 8; gap-1 pairs are excluded from training, so their above-chance confidence reflects genuine generalization to distinctions finer than the supervision. (d) Per-context Spearman correlations across the 40 folds: all contexts are positive (minimum 0.27), with mean $\rho = 0.74$. 26
- 3.1 Visual comparison of (a) standard mel ($n_{\text{fft}} = 2048$, hop = 512, 501 frames) vs. (b) our coarse mel ($n_{\text{fft}} = 4000$, hop = 4000, 65 frames) for the same 8-second accompaniment chunk. The standard mel captures fine temporal transients and spectral detail; the coarse mel retains only broad spectral character, with each column corresponding to one analysis window. 31
- 3.2 Architecture of the vocal-conditioned segment reordering model. The vocal encoder (left, green) receives learned positional embeddings; the instrumental encoder (right, blue) does not. This asymmetry forces the model to use the vocal as a positional template while matching instrumentals by content alone. 33
- 3.3 Data preparation pipeline. Original songs are separated into four stems via MSS, then reorganized into three stems (vocals, drums, harmonics). Segments are selected starting from the downbeat nearest the chorus and used for both training and mashup generation. 41
- 3.4 Vocal-anchored section scan. The frozen encoders of the 8s-scale model act as a compatibility embedding space: the chosen vocal section and every 8-second window of the complementary song’s accompaniment (1-second hop) are embedded, and windows are ranked by cosine similarity. 43
- 3.5 Compatibility sweep on a representative song pair (vocal: 8-second section of a pop song; accompaniment: full track of a second song). Each point scores the 8-second window starting at t (1-second hop, 256 windows) against the chosen vocal; sparse breakdown regions fall to $c \approx 0$ and the selected window is $t^* = 145$ s ($c = 0.302$). 43
- 3.6 Mashup generation process. The base song’s vocal and a portion of its harmonics are combined with the complementary song’s shuffled harmonics, which are rearranged by the trained model. The reconstructed harmonics are layered with the complementary song’s drums and the base song’s vocals to produce the final mashup. 44

3.7	Assignment matrices of the 1s-scale model at two windows of the representative song pair. At a structurally compatible window (left) the Sinkhorn assignment approaches a hard permutation—each vocal position claims a distinct candidate segment with high confidence—while at an incompatible window (right) the assignment remains diffuse. The gap between the two regimes is the confidence signal used for compatibility scoring.	47
3.8	Subjective ratings comparing the mashability baseline, our rearrangement model, and original songs across four evaluation criteria (1–5 scale). The rearrangement model significantly outperforms the mashability baseline across all criteria.	49
4.1	Architecture of the per-stem Attention-UNet backbone. (a) Overall encoder–decoder structure with skip connections. (b) Downsampling encoder and upsampling decoder units. (c) The fundamental ConvBlock unit with optional residual connections.	57
4.2	The per-stem diffusion process in the STFT domain: the forward process couples the imbalance with scheduled Gaussian noise; the reverse process denoises from the imbalanced input under re-injection conditioning.	59
4.3	The per-stem mixing system: four stem-specific diffusion models followed by a mastering model.	60
4.4	A clean instrumental and its imbalanced version under the corruption model of this chapter (EQ bells with a -5 dB volume error). The waveform overlay shows the broadband level change; the spectral centroid trace shows the tonal shift, constant over time—consistent with the time-invariant nature of production mismatch.	63
4.5	Distribution of SDR between the corrupted and clean mixes over 10,000 training windows drawn with the data-preparation pipeline above. Since corruption parameters are sampled afresh at every training step, this is a reference view of the difficulty distribution the model sees, not a fixed dataset: it spans near-transparent cases (mild bells, small volume error) through severely damaged mixes.	63
4.6	The tonal-balance representation (31 log-spaced bands, 40 Hz–16 kHz, band energy in dB) for two evaluation songs: clean vocal, clean instrumental, and the instrumental after a strong corruption (EQ ± 12 dB bells with a 5 dB volume offset; shaded region = the deviation to be corrected). The corruption is clearly legible in the descriptor while note-level musical content is not, making the representation an effective, content-blind conditioning target. In the conditioning plane the instrumental curve is expressed relative to the vocal’s level, so the descriptor carries the instrumental-versus-vocal volume relation as well as the tonal shape.	65

- 4.7 Architecture of the convolutional U-Net backbone. The four input planes pass through an input convolution and a five-level encoder with frequency-heavy pooling; the bottleneck operates at $512 \times 8 \times 11$; each decoder level concatenates its encoder skip connection at the current resolution, applies two ConvBlocks, and upsamples (nearest-neighbor) to the next skip’s resolution. The timestep embedding is added in every ConvBlock, and the zero-initialized output head produces the 1-channel velocity. 66
- 4.8 SI-SDR by corruption condition. Under the clean condition the broken input equals the reference, so its SI-SDR is unbounded (marked ∞). The steered 8-step model improves the broken input by 8–10 dB in every condition; the unconditioned model ($w=0$) improves it consistently but by less—the reference balance is worth ≈ 5 dB. 71
- 4.9 Qualitative restoration example (out-of-domain song, EQ ± 12 dB + -5 dB volume). Left to right: mel spectrogram of the broken input, the restored output ($w=2$, 8-step), and the reference mix; rightmost: the corresponding 31-band tonal-balance curves. 72

LIST OF TABLES

1.1	Components of the automated mashup pipeline. “Proposed (DSP)” denotes signal-processing methods designed in this thesis; “proposed (learned)” denotes neural models trained in this thesis.	9
2.1	Example timemap for an 8-second segment at 44.1 kHz. Source frames reflect detected downbeat positions; target frames enforce an exact 2.0 s grid.	17
2.2	Mean preference frequency for different base song and drum track style combinations (out of 20 participants). Pop base with hip-hop drums received the highest preference.	20
2.3	Leave-one-context-out ranking performance (40 folds). Pairwise accuracy over differing pairs; ρ is the mean per-context Spearman correlation. A random ranking attains accuracy 0.5 and $\rho \approx 0$.	24
2.4	Spearman’s ρ between true and predicted rankings, per swapping strategy. The learned ranker outperforms the mashability heuristic on every strategy.	25
3.1	Architectural comparison of the two encoders. The critical asymmetry is that only the vocal encoder receives positional embeddings.	32
3.2	Training configuration for the reordering model.	37
3.3	Multi-scale training summary. 95% confidence intervals are Wilson intervals over per-position placement decisions ($8 \times n_{\text{val}}$); the random baseline is 12.5% at every scale.	38
3.4	Reordering accuracy on same-song pairs (standard evaluation). The model correctly places segments at nearly 9 out of 10 positions.	38
3.5	Ablation study comparing coarse mel and standard mel features across vocal conditioning conditions. Both models show similar patterns, with the vocal signal providing the dominant structural cue.	39
3.6	Song pair categories for subjective evaluation. Each pair was tested with both songs serving as base and complementary, yielding 10 mashup evaluations.	48
3.7	Mean subjective ratings (1–5 scale) across 30 participants and 10 mashup evaluations.	48
4.1	Per-stem study: SDR (dB) on the 10-song MUSDB test split (per-stem protocol).	60

- 4.2 Restoration quality on 159 clips (10 internal-validation + 149 MUSDB18-HQ songs, all out-of-domain); mean $\pm$ std. Conditions: *clean* (transparency check); *EQ* $\pm 4/8/12$ dB (2–4 random parametric bells of the stated gain range, plus a broadband volume error $\mathcal{U}(-6, 6)$ dB); *volume-only* (a pure broadband level error of ± 3 –6 dB with no spectral shaping). w denotes guidance on the ground-truth balance condition; $w=0$ drops the condition. Bold marks the best method per metric within each condition (broken input excluded). FxNorm-Automix is the pretrained `ours_S_Lb` model applied to the same stems; the two Wave-U-Nets are retrained on the identical task and corruption model, without and with our conditioning. 70
- 4.3 Conditioning ablation, pooled over the four corrupted conditions (mean $\pm$ std over 636 clip-conditions); rightmost column: SI-SDR on clean input (transparency). Guidance $w=2$ for all provided-condition variants. 72

AN AUTOMATED POP SONG MASHUP SYSTEM

by

XINYANG WU

Department of Computer Science and Engineering

The Hong Kong University of Science and Technology

ABSTRACT

Music mashups transform familiar recordings into new works by recombining elements of existing songs. The practice demands precise rhythmic alignment, musically compatible material, and a polished final mix. Automated systems have sought to replicate this craft, but they typically treat mashup creation as a single monolithic problem. Timing is aligned only globally, so small tempo errors accumulate over the course of a song. Compatibility between songs is judged by fixed similarity rules or binary classifiers that do not reflect the graded nature of listener preference. The final mixing stage receives the least attention of all, although stems from different productions must be made to sound as one.

This thesis argues that automated mashup generation becomes tractable when the problem is decomposed into three stages and when quality is measured as graded listener preference rather than a binary label. For temporal alignment, the system locates the rhythmic backbone of each song and corrects timing at the level of individual downbeats, so that the combined material stays locked to a shared beat. For musical compatibility, large-scale listening tests establish which combinations of separated stems listeners prefer, and a family of vocal-conditioned models learns the correspondence between vocals and accompaniment. For mix

engineering, a restoration model corrects the tone and level of the instrumental so that it sits naturally beneath the vocal.

Together, these contributions form a modular, interpretable pipeline that edits and recombines real recordings rather than synthesizing new audio, preserving the identity that defines a mashup and keeping the result enjoyable to listen to.

CHAPTER 1

INTRODUCTION

1.1 Background

Music mashups creatively blend elements from different pre-existing songs to produce new auditory experiences. Originating in the early 2000s alongside the rise of digital audio workstations [1], mashups leveraged emerging music editing tools and online distribution platforms to reach broader audiences [2, 3]. By reimagining familiar songs and crossing traditional genre boundaries, mashups have become both a culturally significant form of music production and a compelling area of computational research.

Creating a high-quality mashup, however, is a complex multi-faceted task. A professional mashup producer typically works through several stages: first ensuring that the rhythmic structures of the source tracks are precisely aligned, then verifying that the musical elements—keys, melodic contours, harmonic progressions—are compatible, and finally mixing the combined stems so that the result sounds polished and balanced [4, 5]. Each of these stages presents distinct technical challenges. Even a slight misalignment at the beat level can make an otherwise compatible pair of songs sound jarring, while harmonically clashing elements can undermine the musical coherence of a well-synchronized mashup. Beyond musical compatibility, the mixing and mastering stage—balancing loudness, equalizing frequency content across stems from different productions—is critical for a professional-sounding result, yet has received little attention in mashup research.

Advances in Music Source Separation (MSS) have opened new possibilities for automated mashup generation. Early tools such as Spleeter [6] achieved a Signal-to-Distortion Ratio (SDR) of around 5, while more recent models like Hybrid Transformer Demucs [7] have raised this to approximately 9. These improvements enable high-quality isolation of individual stems—vocals, drums, bass, and other instrumentals—from mixed recordings, providing the building blocks for

stem-level mashup creation that was previously only possible with access to original multi-track sessions.

While recent generative music models can produce novel compositions from text descriptions [8, 9], mashup creation fundamentally requires preserving the identity and recognizability of existing recordings—the very qualities that distinguish a mashup from an original composition. This necessitates analytical and signal-processing approaches rather than purely generative ones.

Despite advances in both source separation and generative modeling, existing automated mashup systems remain limited. Rule-based approaches rely on “mashability” algorithms that quantify harmonic similarity between segments [10, 11], yet listening tests have shown that these scores do not reliably predict listener preference [12]. Neural network–assisted methods [13] have improved compatibility prediction but frame the problem as binary classification (compatible or not), overlooking the nuanced, graded nature of mashup quality. Moreover, most systems treat mashup creation as a monolithic problem, attempting to address alignment, musical compatibility, and audio quality in a single step, rather than systematically decomposing the problem into its constituent stages.

This thesis addresses these gaps by decomposing the automated mashup creation problem into three distinct and complementary stages—temporal alignment, musical compatibility, and mix engineering—and developing targeted methods for each. Additionally, we introduce a perceptual ranking model trained on systematic listening tests, providing a data-driven evaluation framework that measures mashup quality as a continuous ranking rather than a binary label.

1.2 Related Work

1.2.1 Rule-Based Mashability Systems

The concept of automated mashup generation was pioneered by Davies et al. with the AutoMashUpper system [10, 11], which introduced a “mashability” algorithm to evaluate harmonic similarity between song segments. The system generates beat-synchronous chromagrams, computes cosine similarity across rotational key shifts, and incorporates a tempo bonus to score segment compatibility. Lee et al. [14] re-

defined this concept by distinguishing between vertical mashability (how well two segments layer simultaneously) and horizontal mashability (how smoothly segments transition in sequence), introducing harmonic change balance weighting and volume weighting to the scoring function. However, both approaches fundamentally assume that harmonically similar segments produce better mashups—an assumption that lacks strong empirical support [15]. These systems also typically combine entire vocal and instrumental tracks, often producing dense and cluttered results. Critically, no rule-based system addresses the precise sub-beat temporal alignment needed for professional-quality results.

1.2.2 Music Source Separation and Neural Network–Assisted Approaches

The emergence of deep learning–based MSS models has significantly advanced mashup research. Spleeter [6], based on a U-Net architecture [16], enabled practical four-stem separation (vocals, drums, bass, other). Subsequent models such as Hybrid Transformer Demucs [7, 17] achieved substantially higher separation quality, making stem-level manipulation viable for mashup creation.

Miyamura et al. [12, 18] proposed a multi-source separated mashup system that uses spectral feature similarity to select intermediate materials for smooth transitions. However, their evaluation showed that the defined “smoothness” metric had minimal impact on perceived mashup quality, suggesting that spectral similarity alone is insufficient for predicting listener satisfaction.

Huang et al. [13] presented a neural compatibility–driven approach, training convolutional neural networks (PreMixNet and PostMixNet) on a database of mashups to classify segment compatibility. While their system outperformed rule-based baselines in subjective evaluation, it frames mashability as a binary classification problem and does not address the multi-dimensional nature of mashup quality. Chen et al. [19] further explored neural methods for assessing loop compatibility, though their focus was limited to loop-based electronic music. In the adjacent craft of DJ mixing, Chen et al. [20] learn transition generation from real-world DJ mixes with a generative adversarial network whose generator drives differentiable equalizer and fader effects—automating the transition between tracks, though not the layered combina-

tion of stems that defines a mashup. Most recently, self-supervised representations have been trained specifically for stem compatibility: Stem-JEPA [21] learns a joint-embedding predictive architecture over multi-track data to estimate which single-instrument recordings blend with a given musical context, and AutoMashup [22] assembles a mashup pipeline around such learned compatibility estimates, observing that vocal–instrumental compatibility is inherently *asymmetric*—the direction of the pairing matters—an observation that aligns with the vocal-anchored formulation adopted throughout this thesis.

1.2.3 Gaps in Existing Work

Across these prior works, several significant gaps remain:

1. **Temporal alignment:** Existing systems rely on global tempo matching with uniform time-stretching, ignoring the cumulative drift that occurs when source tracks have slight tempo variations. No prior work addresses local, non-uniform beat warping for precise downbeat alignment—a technique routinely used by professional producers in digital audio workstations.
2. **Musical compatibility beyond harmonic similarity:** Prior approaches evaluate compatibility using spectral or harmonic features, but do not consider adaptive rearrangement of instrumental elements to better complement the vocal track. The possibility of modifying the musical content itself to improve compatibility remains unexplored.
3. **Mix engineering:** No existing mashup system addresses the sonic quality of the final mix—the frequency balance, loudness matching, and spectral equalization needed when combining stems from different productions with different mixing and mastering characteristics.
4. **Perceptual quality as a continuous ranking:** Most evaluation methods treat mashup quality as binary (good/bad) or use coarse rating scales. A model that predicts fine-grained rankings among mashup candidates—reflecting the graded nature of listener preference—has not been developed.

1.3 Objective

This thesis aims to advance automated music mashup generation by systematically addressing the stages of the mashup creation pipeline and developing a perceptual evaluation framework. Specifically, this research seeks to:

- **Develop precise temporal alignment and evaluate stem swapping strategies.**

We propose a mashup framework that leverages MSS and main beat pattern extraction for downbeat-accurate phrase alignment. Beyond global tempo matching, we introduce a beat warping approach applying local non-uniform time-stretching anchored at downbeats to compensate for tempo estimation drift. With reliable alignment established, we systematically evaluate mashup quality across multiple stem-swapping strategies (drum, drum+bass, instrumentals, and vocal swapping) through large-scale listening tests, and develop a perceptual ranking model that predicts mashup quality as a continuous preference ordering.

- **Investigate adaptive instrumental rearrangement for musical compatibility.**

Simple stem swapping preserves the original arrangement of each source, which limits compatibility when harmonic progressions or melodic contours clash. We explore adaptive instrumental rearrangement methods that modify the musical content itself to achieve deeper compatibility between mashup components.

- **Address mix engineering for mashup quality.** We develop a mix engineering model to handle the sonic integration challenges inherent in combining stems from different productions. Using a rectified-flow bridge over the instrumental’s STFT magnitude with exact phase reuse, and a steerable tonal-balance condition, the model performs frequency equalization and volume balancing to produce polished mashup outputs.

1.4 System Overview

The methods developed in this thesis form a single modular pipeline that transforms two commercially produced songs into a polished mashup. Figure 1.1 summarizes

the pipeline and its components; Table 1.1 lists each component, whether it is proposed in this thesis or adopted from prior work, and the chapter in which it is described. The pipeline deliberately combines three kinds of components: pretrained models from the MIR community where mature solutions exist (source separation, beat tracking, audio embeddings), signal-processing methods designed in this thesis where precision requirements exceed what learned models currently provide (beat pattern extraction, local beat warping, harmonic mixing), and neural models trained in this thesis for the two problems at the heart of mashup quality—structural compatibility and mix restoration—for which no suitable prior solutions exist.

Modularity is the central design commitment of this pipeline, and it defines the thesis’s position relative to end-to-end generative alternatives. Every stage exposes an interpretable intermediate: a downbeat grid, an assignment matrix, a tonal-balance curve. The user can therefore inspect why a decision was made, override it, and re-run only what follows from it. A mashup becomes something that can be *edited*. An end-to-end model that maps two songs directly to a mashup offers the opposite profile. It can in principle optimise the final output against a single global objective, but it exposes no intermediate to inspect, no point at which to intervene, and no recourse beyond resampling when the result is unsatisfactory. Because it synthesises rather than recombines, it also cannot preserve what makes a mashup recognisable as one: the original vocal performance and the identity of the source recordings. This pipeline prioritises transparency, controllability, and fidelity to the source material. Section 5.2.3 develops the comparison against contemporary generative systems.

Processing proceeds in three stages. In the **temporal alignment** stage (Chapter 2), each source song is decomposed into four stems by HT Demucs [7]; the proposed main beat pattern extraction method analyzes the drum stem to locate the song’s rhythmic loop backbone and downbeat anchors; and the complementary song is key-matched via Camelot-wheel harmonic mixing and time-stretched to the base song’s tempo. Because global tempo estimates drift by hundreds of milliseconds over the course of a song, a local beat warping step—driven by downbeats from either the kick/snare analysis or the Beat This! neural tracker [23]—snaps every downbeat to an exact grid using Rubberband’s timemap mode [24].

In the **musical compatibility** stage (Chapters 2–3), the aligned stems are com-

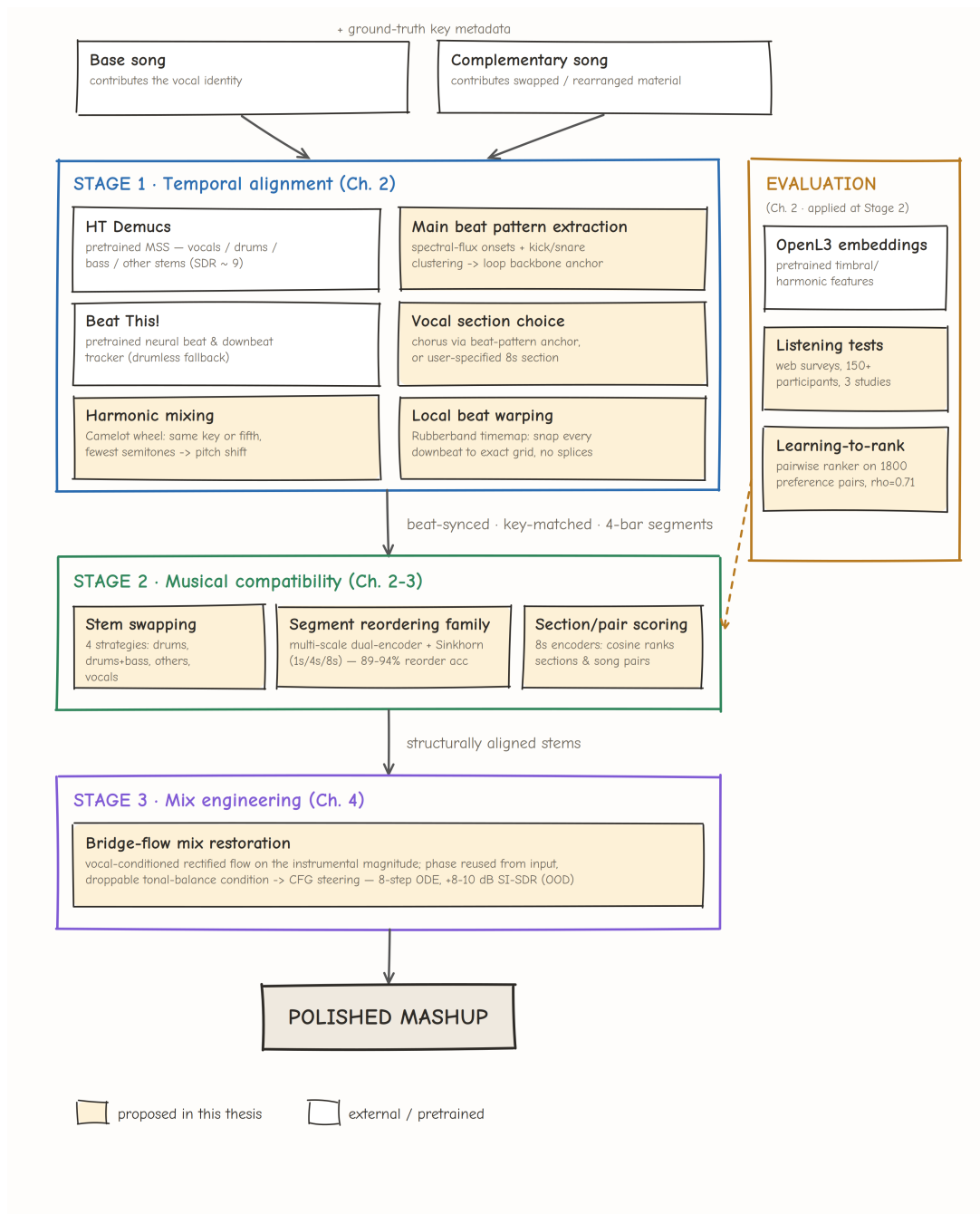


Figure 1.1. Overview of the automated mashup pipeline. Two source songs are separated into stems, aligned on a downbeat-accurate grid, matched and rearranged by the multi-scale reordering family, and integrated by the flow-based mixing system. Shaded components are proposed in this thesis; unshaded components are adopted from prior work. The perceptual ranking model evaluates the combined outputs of the musical compatibility stage.

bined. The simplest instantiation is direct stem swapping, which we evaluate across four strategies through large-scale listening tests; a learning-to-rank model built on OpenL3 embeddings [25] predicts which combinations listeners will prefer. Going beyond selection by heuristics, a multi-scale family of vocal-conditioned reordering models—asymmetric dual-encoder transformers with Sinkhorn-based differentiable permutation learning, trained at 1-, 4-, and 8-second granularities—provides both operations of the generation pipeline: at the 8-second scale its encoders act as compatibility feature extractors that scan the complementary song and select the accompaniment section best matching the chosen vocal, and at the 1-second scale the Sinkhorn assignment rearranges the selected section’s segments to match the vocal’s fine structure.

In the **mix engineering** stage (Chapter 4), the combined stems—each carrying the equalization, compression, and loudness decisions of its original production—are integrated by a vocal-conditioned rectified-flow restoration model operating on the instrumental’s STFT magnitude. A single model corrects frequency and volume imbalances toward a steerable tonal-balance target while preserving the identity of the input signal exactly, by construction, through phase reuse.

The stages are strictly modular: each consumes well-defined inputs (stems, downbeat grids, section selections, segment orderings) and produces interpretable intermediate outputs (warp maps, compatibility curves, assignment matrices, restored spectra). This modularity is what enables the agent-based orchestration envisioned in Chapter 5, in which a language-model planner operates the pipeline components as callable tools to compose full-length mashups.

The subsequent chapters build upon these objectives. Chapter 2 presents our temporal alignment framework—including main beat pattern extraction, beat warping, and stem swapping—and evaluates it through systematic listening tests with a perceptual ranking model. Chapter 3 investigates adaptive instrumental rearrangement to improve musical compatibility beyond what simple swapping can achieve. Chapter 4 presents our mix engineering model for sonic integration of mashup stems. Finally, Chapter 5 summarizes the findings and outlines directions for future research.

Table 1.1. Components of the automated mashup pipeline. “Proposed (DSP)” denotes signal-processing methods designed in this thesis; “proposed (learned)” denotes neural models trained in this thesis.

Component	Kind	Role	Ch.
HT Demucs	pretrained	Four-stem music source separation	2
Main beat pattern extraction	proposed (DSP)	Loop-backbone and downbeat anchors; intact-phrase segment boundaries	2
Beat This!	pretrained	Neural beat/downbeat tracking (alternative warp anchors)	2
Harmonic mixing	proposed (DSP)	Camelot key matching with minimal pitch shift	2
Local beat warping	proposed (DSP)	Timemap-based downbeat grid correction	2
Reordering model family	proposed (learned)	Vocal-conditioned Sinkhorn rearrangement at 1s/4s/8s granularities	3
Vocal-anchored section scan	proposed (learned)	8s-scale encoders as compatibility features; selects mashup material	3
Flow mix restoration	proposed (learned)	Rectified-flow restoration of the instrumental magnitude with phase reuse	4
OpenL3	pretrained	Audio embeddings for the ranking model	2
Learning-to-rank model	evaluation	Pairwise preference ranking of mashup quality	2
Listening tests	evaluation	Web-survey protocol, 150+ participants across three studies	2–3

CHAPTER 2

TEMPORAL ALIGNMENT FOR MASHUP GENERATION

Precise temporal alignment is the foundation of any successful mashup. If the beats of two songs do not align, even the most harmonically compatible pair will sound disjointed. In this chapter, we present our mashup framework with a focus on the temporal alignment stage. We first describe the overall pipeline, including music source separation and our proposed main beat pattern extraction method [26, 27]. We then discuss the limitations of relying solely on global tempo estimation and introduce a local beat warping method for downbeat-accurate alignment. Finally, we present evaluations through stem swapping experiments.

2.1 Framework Overview

Our mashup generation framework processes music input through the following stages:

1. **Music Source Separation.** Each input song is decomposed into four stems—vocals, drums, bass, and other instrumentals—using HT Demucs [7, 28], one of the leading open-source MSS models. Compared to earlier tools such as Spleeter [6] and U-Net-based vocal separation [29], which achieved an average Signal-to-Distortion Ratio (SDR) of approximately 5, HT Demucs achieves an SDR of approximately 9, significantly improving the quality of isolated stems.
2. **Main Beat Pattern Extraction.** The separated drum stem is analyzed to identify the main kick–snare pattern, which determines the rhythmically representative segment for mashup layering. This ensures that mashup components are aligned at structurally significant positions rather than arbitrary time points.
3. **Temporal Alignment.** The complementary song’s components are time-stretched to match the base song’s tempo. We perform both global tempo

alignment and local beat warping to correct for cumulative drift, ensuring downbeat-accurate synchronization throughout the mashup.

4. **Mashup Assembly.** The aligned stems from the base and complementary songs are combined according to the chosen swapping strategy (e.g., drum swapping) and mixed into the final mashup output.

A key design principle of our framework is that mashup components should be layered starting precisely at the downbeat of each song’s main beat pattern. This ensures that the core rhythmic elements—particularly the kick drum on the downbeat—are synchronized, preventing the disruptive effects of misaligned layering that can significantly degrade the listening experience.

2.2 Main Beat Pattern Extraction

To determine the optimal alignment points for mashup layering, we extract the main kick–snare pattern from the separated drum stem [27]. The extraction process consists of five steps.

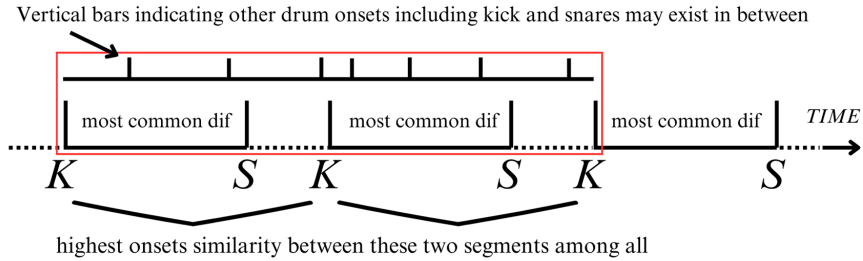


Figure 2.1. Main beat pattern extraction. Detected onsets are classified as kick (K) or snare (S). The most common kick-to-snare delay identifies structurally paired beats, and the segment pair with the highest cross-correlation of onset sequences is selected as the main beat pattern (red box).

2.2.1 Pre-processing

The drum-stem waveform is first low-pass filtered with a cutoff frequency of approximately 2 kHz to attenuate high-frequency percussive content (hi-hats, cymbals), then downsampled to reduce computational cost while preserving the spectral characteristics of kick and snare drums.

2.2.2 Spectral Flux Onset Detection

We compute the magnitude spectrogram of the downsampled signal and calculate the half-wave rectified spectral flux [30, 31]:

$$F[p] = \sum_{k=0}^{K-1} \max(0, |X[k, p]| - |X[k, p-1]|) \quad (2.1)$$

where:

- $F[p]$ is the spectral flux at frame p , measuring the total increase in spectral energy from the previous frame.
- $|X[k, p]|$ is the magnitude of the STFT at frequency bin k and time frame p .
- K is the total number of frequency bins in the spectrogram.
- The $\max(0, \cdot)$ operator (half-wave rectification) ensures only energy increases contribute, ignoring decreases that correspond to note offsets rather than onsets.

Onsets are detected via peak picking [32], selecting frames where the spectral flux exceeds a threshold based on the mean plus a multiple of the standard deviation, with a minimum inter-onset interval (e.g., 50 ms) to prevent duplicate detections.

2.2.3 Kick/Snare Clustering

For each detected onset, we extract a fixed 0.1 s audio excerpt centered on the onset and compute its mel-spectrogram, which is flattened into a feature vector. We apply K -means clustering with $K = 2$ to partition all onsets into two groups. To determine which cluster corresponds to kicks and which to snares, we compute the average spectral centroid of each cluster:

$$c_j = \frac{\sum_{i \in C_j} \sum_k f_k |X_i[k]|}{\sum_{i \in C_j} \sum_k |X_i[k]|} \quad (2.2)$$

where:

- c_j is the average spectral centroid of cluster C_j , representing the energy-weighted mean frequency of the onsets in that cluster.
- C_j is the set of onset indices assigned to cluster j (with $j \in \{1, 2\}$).
- $|X_i[k]|$ is the magnitude spectrum of the i -th onset excerpt at frequency bin k .
- f_k is the center frequency (Hz) of bin k .

The cluster with the lower centroid is labeled “kick” and the other “snare,” since kick drums concentrate energy in lower frequencies [33, 34].

2.2.4 Main Pattern Selection

Given the timestamps of all kicks $\{K_i\}$ and snares $\{S_i\}$, we find the most common kick-to-snare delay Δ by counting co-occurrences within a tolerance of 0.1 s. We then collect all kicks that have a corresponding snare at the expected offset. For each pair of consecutive beat intervals in this filtered set, we compute the cross-correlation of their onset sequences and select the pair with the highest correlation as the main beat pattern.

This method identifies the most rhythmically stable and representative segment of the song—typically corresponding to the chorus or a structurally prominent section—which serves as the alignment anchor for mashup layering.

2.2.5 Intact Musical Phrase Extraction

A critical property of the main beat pattern extraction is that the resulting segments are theoretically intact musical phrases. Because the pattern boundaries are defined by recurring kick–snare structures, the extracted segment aligns with the song’s loop backbone [35, 27]—the foundational repeating unit that structures the music. This prevents disrupting the musical flow by avoiding cuts in the middle of a phrase.

Furthermore, the main beat pattern determines the rate at which complementary songs must be time-stretched to achieve synchronization, even in the presence of potential octave-related tempo ambiguities (e.g., 120 BPM vs. 60 BPM) [36].

2.2.6 Harmonic Mixing

Beyond temporal alignment, the framework addresses harmonic compatibility [37] by shifting the complementary song’s key to match the base song’s actual key or its perfect fifth, whichever requires fewer semitone steps. This minimizes pitch-shifting artifacts while ensuring harmonic coherence. Given the challenges of automated key estimation—with recent research achieving at most 80% accuracy on certain datasets [38, 39]—the system incorporates ground truth key metadata as an additional input to ensure reliable harmonic matching.

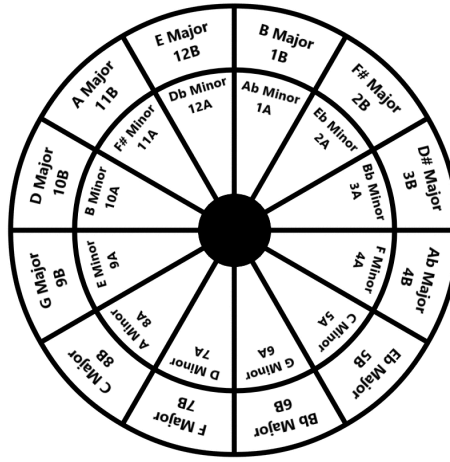


Figure 2.2. The Camelot wheel, a circular representation of key compatibility based on the circle of fifths. Adjacent keys on the wheel are harmonically compatible. The system selects the target key (same key or perfect fifth) that minimizes the number of semitone shifts required.

2.2.7 Limitations

A limitation of this extraction method is that it is not applicable to songs without drum elements. However, for the pop music domain that is the focus of this work, drum presence is nearly universal.

2.3 Global Tempo Alignment and Its Limitations

2.3.1 Tempo Estimation

Once the main beat pattern is identified, the tempo of each song must be determined so that the complementary song can be time-stretched to match the base song.

Tempo estimation has been extensively studied in the MIR literature [30], with approaches ranging from autocorrelation-based methods [40] to recurrent neural networks [41] and convolutional neural networks that treat tempo as a classification problem [38, 36].

Most tempo estimation algorithms are evaluated using tolerance-based accuracy metrics—typically Accuracy1 (correct within $\pm 4\%$) and Accuracy2 (allowing octave errors) [42]. Even state-of-the-art systems do not achieve perfect accuracy, meaning small estimation errors on the order of 0.5–1 BPM are common. Furthermore, treating tempo as a classification problem introduces quantization: the predicted tempo is assigned to a discrete class, and songs whose true tempo falls between class boundaries will incur a small but systematic error.

An additional source of inaccuracy arises from the audio distribution pipeline. When music is distributed digitally, resampling, format conversion, or lossy compression can introduce subtle stretching of the waveform, causing the effective tempo to deviate slightly from the intended BPM at which the track was originally produced.

2.3.2 The Cumulative Drift Problem

These small tempo errors become problematic because they accumulate over time. In a typical mashup pipeline, we need to locate a musically meaningful segment—such as the chorus—within a full-length track. A common approach is to estimate the chorus position roughly (e.g., via a chorus detection algorithm [43]), then snap to the nearest bar boundary by counting bars from the song’s beginning using the estimated bar length. If the tempo estimate is off by even 1 BPM, the drift grows with every bar counted.

For example, at 120 BPM one bar lasts exactly 2.0 seconds. At 121 BPM, one bar lasts approximately 1.983 seconds—a difference of about 17 ms per bar. Over 30 bars (roughly 60 seconds into the song, where a chorus might begin), this accumulates to over 500 ms of drift. The bar-counting approach would land the segment extraction point nearly half a second off the true downbeat, producing a mashup that is misaligned from its very first beat.

This makes global tempo estimation unreliable for precise segment localization

in longer tracks, motivating the need for a method that directly detects and corrects beat positions rather than inferring them from a single tempo value.

2.4 Local Beat Warping

To address cumulative drift, we propose a local beat warping method that corrects timing at the downbeat level, analogous to the warp marker functionality in professional digital audio workstations. The warping operates on downbeat positions obtained from any reliable detection method—including our kick/snare pattern extraction (Section 2.2) or a neural network-based tracker—and applies continuous time-correction to snap each downbeat to its ideal grid position.

2.4.1 Downbeat Detection Schemes

The warping stage requires accurate downbeat positions within the extracted segment. We consider two complementary approaches for obtaining these positions:

Approach 1: Kick/snare pattern–derived downbeats. The main beat pattern extraction method described in Section 2.2 identifies downbeat positions through musically grounded analysis—spectral flux onset detection, kick/snare clustering, and cross-correlation-based pattern matching. Since this method explicitly models the rhythmic semantics of the drum track (distinguishing kick from snare), its detected positions are anchored to structurally meaningful events. The downbeat positions identified during pattern extraction can be directly reused as warp anchors, providing consistency between the segment localization and warping stages.

Approach 2: Neural network–based beat tracking. With the growing application of neural networks to Music Information Retrieval tasks, data-driven beat trackers have achieved strong performance on diverse musical material. We adopt Beat This! [23], a state-of-the-art beat and downbeat tracker that achieves high accuracy without requiring dynamic Bayesian network (DBN) post-processing. Beat This! can be applied efficiently to individual segments, providing an alternative downbeat detection scheme that does not depend on the separated drum stem and can generalize to material where kick/snare patterns are less clearly defined.

Both approaches yield a set of downbeat timestamps within the segment. For an

8-second segment at 120 BPM, we expect approximately 4 downbeats at 0, 2, 4, and 6 seconds. In practice, due to residual drift from global tempo alignment, the detected positions may be slightly offset—for example, at 0, 2.02, 4.01, and 6.03 seconds. The choice of detection scheme depends on the musical material and the availability of a clean drum stem; both can serve as input to the subsequent timemap construction.

2.4.2 Downbeat Filtering and Normalization

Before constructing the warp map, we apply two filtering steps to ensure robustness:

1. **Outlier removal.** Downbeats that deviate more than 10% from the median inter-downbeat interval are discarded. This handles cases where the beat tracker detects false downbeats at section boundaries or structural transitions.
2. **Metrical normalization.** If the detector identifies beats at the beat level (approximately 0.5 s intervals at 120 BPM) rather than at the downbeat level (approximately 2.0 s intervals), we subsample every second detection. Conversely, if the detector identifies half-time bars (approximately 4.0 s intervals), we insert midpoints. This ensures consistent downbeat-level granularity regardless of the tracker’s metrical interpretation.

2.4.3 Timemap Construction and Continuous Warping

We construct a timemap—a set of source-to-target frame mappings—that specifies where each detected downbeat should land in the output:

Table 2.1. Example timemap for an 8-second segment at 44.1 kHz. Source frames reflect detected downbeat positions; target frames enforce an exact 2.0 s grid.

Downbeat	Source Frame	Target Frame
1	0	0 (0.00 s)
2	89081	88200 (2.00 s)
3	177281	176400 (4.00 s)
4	265482	264600 (6.00 s)
End	352800	352800 (8.00 s)

The timemap is passed to Rubberband [24], which performs continuous phase-vocoder-based time-stretching [44] in a single pass. Unlike a piecewise approach—where the audio would be cut at each warp point, stretched independently, and concatenated—Rubberband’s timemap mode interpolates smoothly between anchor points, avoiding splice artifacts at segment boundaries. The R3 (fine) engine is used with smoothing enabled and transient detection disabled to prevent phase-reset artifacts, particularly on vocal content.

This approach mirrors the warp marker workflow used by professional producers in digital audio workstations, but applied programmatically as part of an automated pipeline.

2.5 Evaluation: Stem Swapping Experiments

With precise beat synchronization and warping in place, all generated mashups exhibit reasonable rhythmic alignment. The evaluation therefore focuses not on whether the synchronization succeeds, but on whether the resulting mashup combinations are musically appealing—specifically, whether certain stem-swapping pairings are preferred over the original tracks. We conducted two rounds of experiments: an initial drum swapping study [26] and an extended multi-strategy study [27].

2.5.1 Experiment 1: Drum Swapping

We selected 10 diverse pop songs from various styles (pop, hip-hop, dance pop) and tempo ranges (98–174 BPM). For each ordered pair of songs, we designated one as the base song (retaining its vocals, bass, and other instrumentals) and extracted the main beat pattern from the other song’s drum stem. The extracted beat pattern was time-stretched to match the base song’s tempo and layered from the onset of the base song’s main beat pattern, producing a total of 90 unique mashup combinations. Each mashup was presented as an 8-bar sample.

Twenty participants evaluated the mashups through a web survey. For each base song, participants listened to the original track followed by 9 variations with different drum tracks, and were asked to select any variations they preferred over the original. This design tests whether the surprising combinations enabled by automated

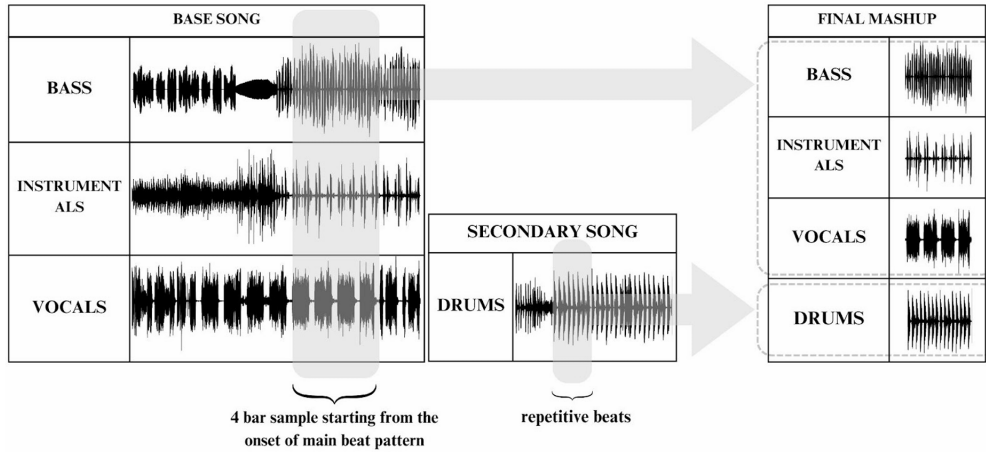


Figure 2.3. Drum swapping mashup creation process. The base song’s vocals, bass, and other instrumentals are retained, while the drum stem is replaced with the secondary song’s drum pattern, time-stretched to match the base song’s tempo.

mashup generation can outperform the original productions.

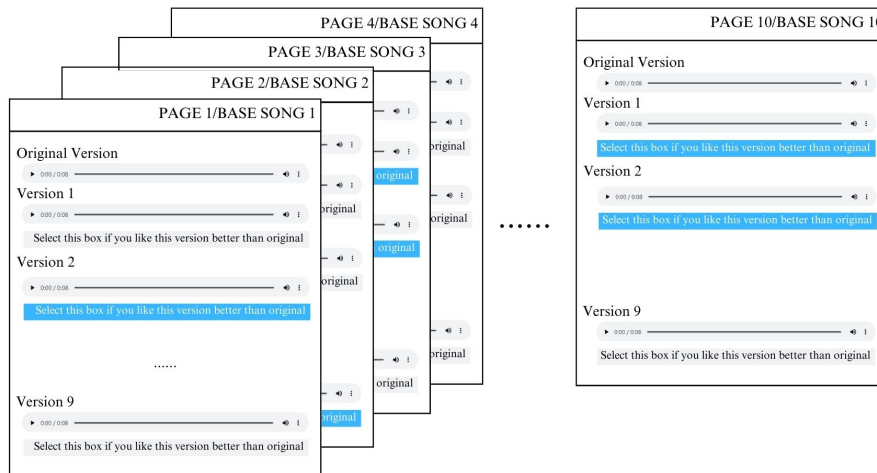


Figure 2.4. Web survey layout for the listening test. Each of the 10 pages corresponds to one base song. Participants first listen to the original version, then hear 9 mashup variations and select any they prefer over the original by clicking the selection box (highlighted in blue).

Of the 90 mashups generated, 27% of the variations were preferred over the original tracks. Remarkably, only one variation out of 90 was not selected by any participant, indicating that when beat synchronization is executed correctly, nearly every mashup has the potential to be favored by certain listeners. The distribution of listener preferences followed an approximately normal distribution centered around 5 selections out of 20 participants.

The experiment revealed several trends:

Tempo. Drum tracks with very fast tempos (exceeding 168 BPM) tended to be less favored, while base tracks with fast tempos were preferred for mashups. This suggests that high-tempo songs provide a compelling rhythmic foundation when paired with varied drum patterns.

Rhythm style. Non-uniform beat patterns—characterized by syncopation and variable rhythms, common in hip-hop—were preferred over uniform (four-on-the-floor) patterns for drum tracks.

Musical style. Hip-hop drum tracks were the most favored, followed by dance pop, with general pop drum tracks receiving the least enthusiasm. Conversely, pop songs were preferred as base tracks, suggesting that listeners favor combinations of hip-hop rhythmic elements with pop melodic content.

Table 2.2. Mean preference frequency for different base song and drum track style combinations (out of 20 participants). Pop base with hip-hop drums received the highest preference.

Base Style	Drum Style	Mean Preference
Pop	Hip-hop	6.83
Pop	Dance pop	6.11
Hip-hop	Hip-hop	5.75
Dance pop	Dance pop	5.50
Dance pop	Hip-hop	5.25
Hip-hop	Dance pop	5.25
Pop	Pop	5.33
Dance pop	Pop	4.44
Hip-hop	Pop	4.08

2.5.2 Experiment 2: Extended Multi-Strategy Stem Swapping

The success of drum swapping—where the temporal alignment framework alone produced mashups that listeners frequently preferred over originals—motivated an extension to additional stem-swapping strategies [27]. Using the same set of 10 songs, we generated 360 mashups across four distinct strategies:

1. **Strategy 1 (Drum swapping):** base song’s vocals, bass, and other instrumentals combined with the complementary song’s drum stem.

2. **Strategy 2 (Drum and bass swapping):** base song’s vocals and other instrumentals combined with the complementary song’s drum and bass stems.
3. **Strategy 3 (Other instrumentals swapping):** complementary song’s other instrumentals combined with the base song’s remaining components.
4. **Strategy 4 (Vocal swapping):** base song’s vocal retained while layering all other components from the complementary song.

Each mashup was configured to last eight bars, corresponding to approximately 10 to 20 seconds depending on tempo. This duration was chosen to be substantial enough to engage listeners meaningfully while remaining concise enough to prevent fatigue throughout the listening test [45]. For each mashup, the base song was chosen to always include the vocal component, as vocals are easily recognizable and anchor the listener’s perception of the mashup’s identity.

The experiments were structured as four separate listening tests, each corresponding to one strategy and conducted at different times over the course of a year. In each test, participants were provided with a web survey containing ten pages—one for each base song. On each page, participants first listened to the original version of the base song to establish baseline familiarity, then heard nine mashup variations (one for each complementary song). After listening, participants selected their preferred variation(s); they were allowed to choose one, multiple, or none, depending on their personal preference and perceived quality.

A total of 108 unique participants were involved across all sessions, all college students aged 18–30 enrolled in music-related courses with varying years of musical experience. The ranking correlation between different strategies for the same song pairs yielded a mean correlation of 0.18, indicating that each strategy offers a distinct listening experience and that mashup quality is highly strategy-dependent.

2.5.3 Learning to Rank Mashup Quality

The listening test results provide not only aggregate preferences but also pairwise ranking data: for a given base song, participants’ selections implicitly rank the nine complementary song pairings. To leverage this structure, we develop a ranking

model that predicts mashup quality as a continuous preference ordering. The pairwise formulation is dictated by the size of the data. The listening tests yield 360 rated mashups in total: treated as a pointwise regression problem, this provides only 360 scalar targets; treated listwise, only $4 \times 10 = 40$ ranked lists. Forming preference comparisons within each context instead expands the same responses into $\binom{9}{2} = 36$ pairs per context and 1,335 differing pairs in total, at no additional annotation cost. The pairwise targets are also the more faithful reading of the data: within a fixed context only the order between two candidates enters the training signal, so differences in how selective individual participants are do not bias the targets. Figure 2.5 summarizes the pipeline.

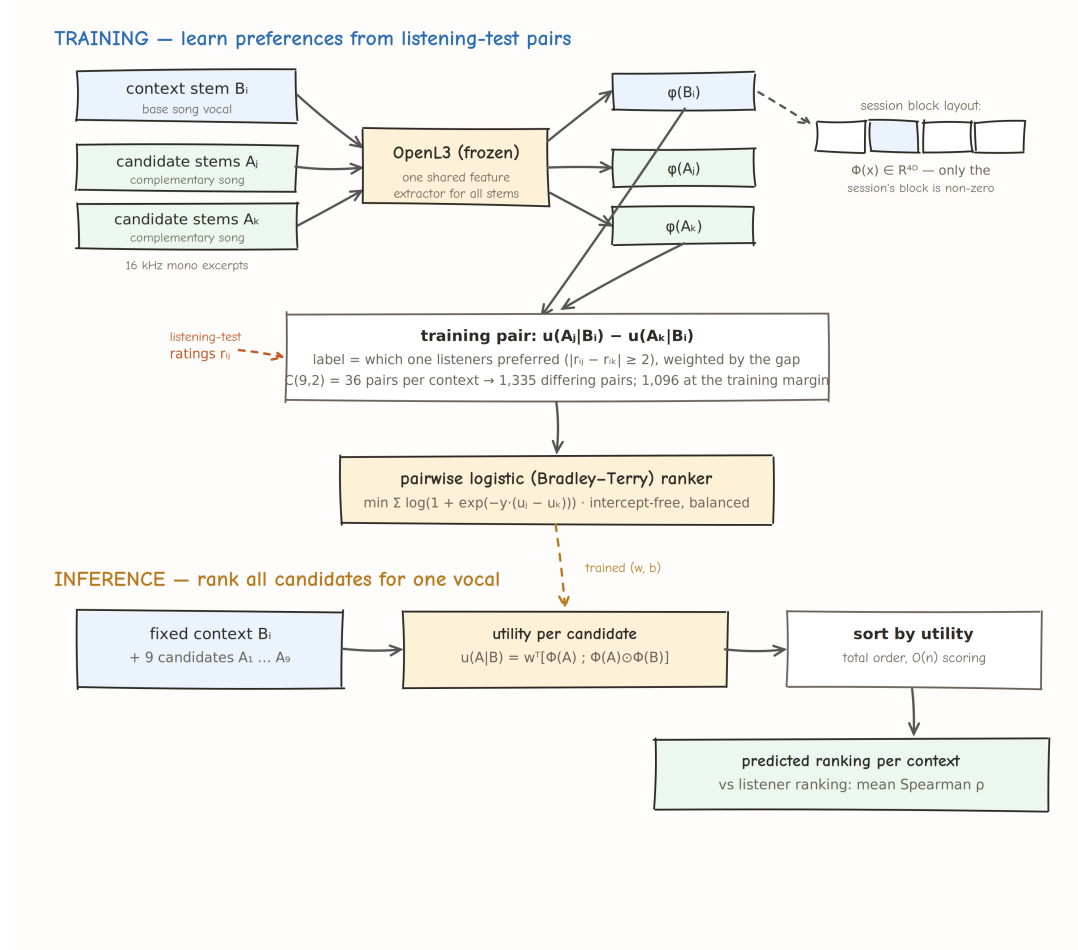


Figure 2.5. Learning to rank mashup quality. *Training* (top): the context stem B_i and two candidate stems A_j, A_k are embedded separately by one shared frozen OpenL3 extractor, placed in session-aware blocks, and formed into utility-difference pairs labeled by the listening-test ratings; a pairwise logistic (Bradley-Terry) ranker is trained on the resulting comparisons. *Inference* (bottom): each candidate receives a utility conditioned on the vocal, and sorting utilities yields the predicted ranking, evaluated against listeners by Spearman’s ρ .

Feature representation. Each stem is resampled to 16 kHz mono and processed through OpenL3 [25, 46] (mel-128 input representation, music content type), yielding one 512-dimensional embedding per second of audio; frame embeddings are averaged over time into a single vector $\phi(x) \in \mathbb{R}^D$, $D = 512$. The context stem and the candidate stems are embedded separately by the same frozen extractor.

Session-aware encoding. To control for strategy-specific effects across the four listening test sessions, each feature vector is embedded into a block-sparse representation in which only the block of the mashup’s session is active. Writing $\text{cat}(x) \in \{0, 1, 2, 3\}$ for the session index,

$$\Phi(x) = [0, \dots, 0, \phi(x), 0, \dots, 0]^\top \in \mathbb{R}^{C \cdot D}, \quad C = 4, \quad (2.3)$$

so that the ranker learns one preference direction per strategy while sharing a single training problem.

Ranking model. The ranker assigns each candidate an explicit *utility* conditioned on its context,

$$u(A \mid B) = w^\top \begin{bmatrix} \Phi(A) \\ \Phi(A) \odot \Phi(B) \end{bmatrix}, \quad (2.4)$$

where $\odot$ is the elementwise product; the product term couples each candidate to the specific vocal it must support, whereas a purely additive context term would contribute identically to every candidate under the same context and could not alter their order. The preference probability for a pair follows the Bradley–Terry form

$$P(A_j \succ A_k \mid B_i) = \sigma(u(A_j \mid B_i) - u(A_k \mid B_i)), \quad (2.5)$$

which is antisymmetric by construction, and the model is trained as an intercept-free logistic regression over utility differences with balanced class weights on standardized features.

Pairwise construction. For a fixed context B_i , training examples are formed from all unordered candidate pairs whose ratings differ by at least two rank steps, $|r_{i,j} - r_{i,k}| \geq 2$; adjacent ranks reflect distinctions too fine for listeners to make reliably, and excluding them removes ambiguous supervision. Each example is weighted by the rating gap $|r_{i,j} - r_{i,k}|$, so confidently separated pairs contribute more. This yields 1,096 training comparisons; swapping the pair negates the utility difference and flips the label, so each unordered pair contributes one example without orientation bias.

Evaluation protocol. A *context* is one base song’s group of nine candidate mashups. Evaluation is leave-one-context-out: each of the 40 contexts is held out in turn, a model is trained on the 39 remaining contexts, and the held-out context’s nine candidates are ranked by their utilities; all reported numbers are averages over the 40 folds, so every prediction that enters a metric is made for a base song the model has never seen. Two metrics are reported: pairwise accuracy over the held-out context’s differing pairs, and the Spearman correlation ρ between the predicted and listener-derived rankings, averaged per context.

Results. Table 2.3 reports the full configuration alongside two alternative formulations and one-factor ablations, all trained and evaluated identically. Both pairwise formulations outperform the pointwise baseline by a clear margin, confirming that the preference comparisons carry supervision that pointwise scores do not; the utility formulation performs on par with win-count ranking and is adopted because its rankings are transitive by construction, its scoring cost is linear rather than quadratic in the number of candidates, and its antisymmetry removes any dependence on presentation order. Among the components, the session-aware blocks carry the largest contribution (-0.033ρ when removed); the rating-gap weights contribute a small consistent gain; the margin filter and the context-interaction term are approximately neutral on these labels and are retained for robustness. The differences of a few thousandths of a point in ρ among the pairwise variants are well within the fold-to-fold noise—the per-context standard error of the mean is ≈ 0.05 —so the ablated components are effectively tied with the full configuration rather than improving on it.

Table 2.3. Leave-one-context-out ranking performance (40 folds). Pairwise accuracy over differing pairs; ρ is the mean per-context Spearman correlation. A random ranking attains accuracy 0.5 and $\rho \approx 0$.

Model	acc	ρ
Pointwise regression	0.784	0.676
Pairwise, win-count ranking	0.810	0.735
Pairwise, utility ranking (full configuration)	0.807	0.738
– context interaction	0.807	0.740
– margin filter (all differing pairs)	0.805	0.741
– rating-gap weighting	0.803	0.730
– session-aware blocks (shared weights)	0.792	0.705

To situate these numbers against prior work, Table 2.4 compares the utility ranker with the harmonic “mashability” heuristic that underlies rule-based mashup sys-

tems [10], which scores a candidate by the beat-synchronous chromagram compatibility and spectral balance between the two stems. The comparison is reported per swapping strategy, since the strategies differ in which stem the heuristic can act on.

Table 2.4. Spearman’s ρ between true and predicted rankings, per swapping strategy. The learned ranker outperforms the mashability heuristic on every strategy.

Model	Vocal	Other	Drum	Bass	Mean
Mashability heuristic	−0.230	0.031	−0.173	0.183	−0.047
Utility ranker (ours)	0.798	0.799	0.639	0.717	0.738

The mashability heuristic is near chance on average ($\rho = -0.05$): its beat-tracking and chromagram machinery, designed for full phrase sections, is unreliable on the short stems evaluated here, and it carries no notion of the graded listener preference the task measures. The learned ranker is robust across all four strategies, confirming that mashup preference reflects more than the harmonic similarity such heuristics capture.

The analysis in Figure 2.6 shows that the ranker’s probabilities behave as confidences should: the pairwise classifier is discriminative (AUC 0.89), the probabilities are calibrated, they scale with the strength of listener preference—including on the adjacent-rank pairs excluded from training, where the classifier still reaches AUC 0.64—and the per-context correlations are positive for every one of the 40 base songs. Per-context variation remains substantial, which is why the 40-fold average, rather than any single context, is the appropriate summary of ranking quality.

2.6 Chapter Summary

In this chapter, we presented the temporal alignment and stem swapping stages of our automated mashup framework. The main beat pattern extraction method identifies rhythmically representative segments through spectral flux onset detection, kick/snare clustering, and cross-correlation-based pattern matching. We showed that relying solely on global tempo estimation is insufficient for locating segments precisely within longer tracks, as even small tempo errors accumulate to hundreds of milliseconds when counting bars toward a chorus. To address this, we introduced a local beat warping method that applies continuous, artifact-free time correction

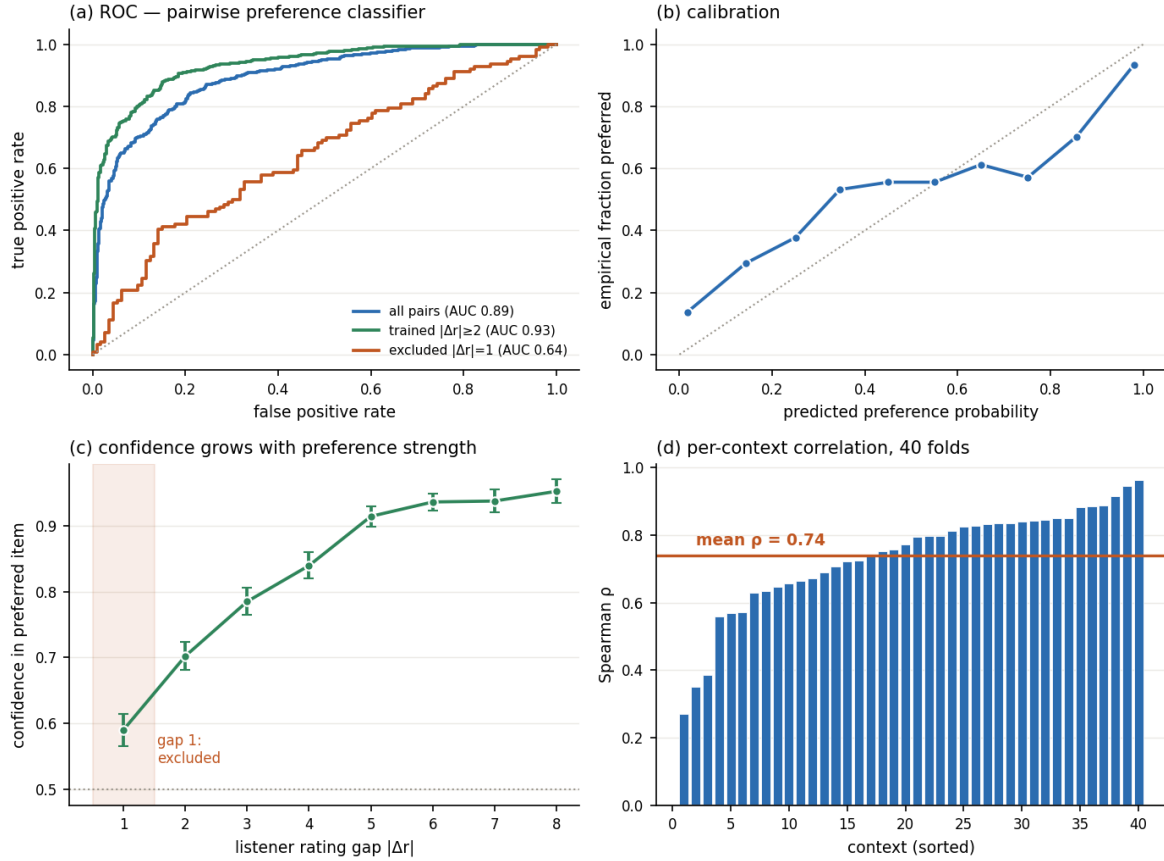


Figure 2.6. Behavior of the utility ranker on out-of-sample predictions (all 40 leave-one-context-out folds). (a) The pairwise preference classifier separates preferred from non-preferred candidates well (AUC 0.89 overall, 0.93 on the trained pairs, and 0.64 on the excluded adjacent-rank pairs). (b) Predicted probabilities are well calibrated against empirical outcomes. (c) The model’s confidence in the listener-preferred item grows monotonically with the listeners’ rating gap, from 0.59 at gap 1 to 0.95 at gap 8; gap-1 pairs are excluded from training, so their above-chance confidence reflects genuine generalization to distinctions finer than the supervision. (d) Per-context Spearman correlations across the 40 folds: all contexts are positive (minimum 0.27), with mean $\rho = 0.74$.

using downbeat positions from either our kick/snare extraction or a neural beat tracker.

With reliable temporal alignment established, we evaluated stem swapping across four strategies with 108 participants and 360 mashups. The results showed that 27% of drum-swapped mashups were preferred over originals, and that each swapping strategy produces a distinct listening experience. A logistic regression ranking model trained on these results achieved a Spearman correlation of 0.71, substantially outperforming traditional mashability heuristics.

However, simple stem swapping—replacing one song’s stems with another’s without modification—preserves the original musical arrangement of each source. When the harmonic progressions, melodic contours, or structural phrasing of two songs are incompatible, no amount of temporal alignment can resolve the clash. This motivates the investigation of adaptive instrumental rearrangement in the following chapter, where we explore methods that modify the musical content itself to achieve deeper compatibility between mashup components.

CHAPTER 3

ADAPTIVE INSTRUMENTAL REARRANGEMENT

The temporal alignment and stem swapping framework presented in Chapter 2 produces mashups where every beat is precisely synchronized. However, simple stem swapping preserves the original musical arrangement of each source song. When the vocal track of one song is combined with the accompaniment of another, their musical structures typically differ—the two songs may have different verse/chorus orderings, varying energy contours, and distinct phrase patterns. Without structural alignment, a chorus vocal may be paired with a verse-like accompaniment, or a quiet vocal passage may coincide with an energetic instrumental section, producing an incoherent result.

In this chapter, we present a learned reordering model that addresses this structural mismatch. We first formulate structural alignment as a permutation prediction problem, then describe our model—an asymmetric dual-encoder transformer with Sinkhorn-based differentiable permutation learning—that predicts the optimal rearrangement of accompaniment segments to match a vocal template, trained as a family at three temporal granularities whose encoders double as vocal-accompaniment compatibility feature extractors. Finally, we describe how the model is applied to mashup generation and present subjective listening test results.

3.1 Problem Formulation

We frame structural alignment as a permutation prediction problem. The core insight is that while existing mashup approaches select globally compatible segments, they neglect the finer synchronization of individual sub-segments—melody trajectories, chord progressions, energy contours, and onset patterns. Achieving alignment at this granular level through simple layering is a low-probability event.

Our approach preserves the vocal track of the base song while breaking the complementary song’s instrumentals into smaller segments. These segments are then rearranged to align with the vocal’s musical structure at the segment level.

Specifically, an 8-second audio segment at 120 BPM (4 bars) is divided into 8 segments of 1 second each. Given:

- Vocal segments $\mathbf{V} = \{v_0, v_1, \dots, v_7\}$ in their original order, providing the structural template. Each v_i is the feature representation of the i -th 1-second vocal segment.
- Accompaniment segments $\mathbf{A} = \{a_{\pi(0)}, a_{\pi(1)}, \dots, a_{\pi(7)}\}$ in unknown (shuffled) order. Each $a_{\pi(i)}$ is the feature representation of the accompaniment segment at position i after shuffling by an unknown permutation π .

the model predicts a doubly-stochastic assignment matrix $\mathbf{P} \in \mathbb{R}^{8 \times 8}$, where:

- $\mathbf{P}$ is a soft assignment matrix with rows summing to 1 and columns summing to 1.
- $P[j, i]$ is the probability that output position j should contain input segment i .

The hard permutation σ is obtained via:

$$\sigma(j) = \underset{i}{\operatorname{argmax}} P[j, i] \quad (3.1)$$

where:

- $\sigma(j)$ returns the index of the input segment assigned to output position j .
- argmax is applied along the input-segment dimension for each output position.

The objective is to find σ such that the reordered accompaniment $\{a_{\sigma(0)}, a_{\sigma(1)}, \dots, a_{\sigma(7)}\}$ best matches the musical structure implied by the vocal sequence $\mathbf{V}$.

3.2 Feature Extraction

Each 1-second audio segment is converted to a mel-spectrogram feature vector, computed with librosa [47]. We employ a deliberately non-standard mel configuration that prioritizes semantic content matching over temporal fingerprinting.

3.2.1 Coarse Mel Design

The feature extraction uses the following parameters:

$$\text{sr} = 32000, \quad n_{\text{fft}} = 4000, \quad \text{hop} = 4000, \quad n_{\text{mels}} = 128 \quad (3.2)$$

where:

- sr is the audio sampling rate in Hz.
- n_{fft} is the FFT window size in samples.
- hop is the hop length (frame step) in samples, determining the stride between consecutive analysis windows.
- n_{mels} is the number of mel frequency bins produced by the mel-scale filterbank.

This yields exactly 9 time frames per 1-second segment ($32000/4000 = 8$, plus boundary frame), producing a feature vector of dimension $128 \times 9 = 1152$ after flattening. The normalization applied is:

$$\mathbf{f} = \frac{\text{power_to_dB}(\text{Mel}(x), \text{ref} = \max)}{80} \quad (3.3)$$

where:

- $\mathbf{f}$ is the final normalized feature vector.
- $\text{Mel}(x)$ denotes the mel-spectrogram computation applied to the input waveform x .
- $\text{power_to_dB}(\cdot, \text{ref} = \max)$ converts power-scale magnitudes to decibels, with the maximum value used as the 0 dB reference.

- Division by 80 scales the decibel range (typically 0 to -80 dB) to approximately $[-1, 0]$, providing a bounded input for the neural network.

3.2.2 Zero-Overlap Design Rationale

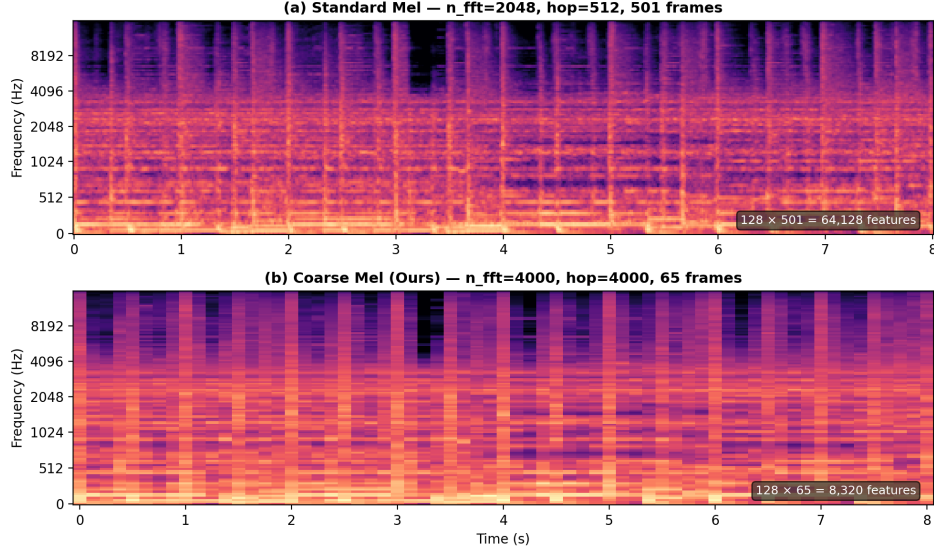


Figure 3.1. Visual comparison of (a) standard mel ($n_{\text{fft}} = 2048$, $\text{hop} = 512$, 501 frames) vs. (b) our coarse mel ($n_{\text{fft}} = 4000$, $\text{hop} = 4000$, 65 frames) for the same 8-second accompaniment chunk. The standard mel captures fine temporal transients and spectral detail; the coarse mel retains only broad spectral character, with each column corresponding to one analysis window.

The critical design decision is setting $n_{\text{fft}} = \text{hop}$, resulting in zero overlap between analysis frames. Standard mel spectrograms use $n_{\text{fft}} \gg \text{hop}$ (typically 75% overlap) to achieve smooth, detailed temporal resolution. We avoid this for three reasons:

1. **Fixed feature dimension.** Every 1-second segment produces exactly 1152 features regardless of boundary conditions. No padding or truncation is required.
2. **Coarse temporal resolution encourages semantic matching.** With only 9 frames per second, the representation naturally emphasizes overall spectral character—energy distribution, harmonic content, timbral quality—rather than fine transient patterns or onset timing. These properties are more likely to generalize across different recordings.

3. **No inter-segment information leakage.** With zero overlap, the analysis window for each frame is entirely contained within its time slice. When audio is cut at segment boundaries, no spectral information from adjacent segments bleeds into the features, eliminating a potential source of boundary-matching shortcuts.

For comparison, a standard mel spectrogram ($n_{\text{fft}} = 2048$, $\text{hop} = 512$) produces $128 \times 63 = 8064$ features per second—7 times more information. An initial concern was that this higher resolution could capture exact transient positions and spectral leakage patterns that constitute “temporal fingerprints,” enabling the model to match segments by waveform boundary artifacts rather than musical content. However, as our ablation study in Section 3.5.3 demonstrates, this concern does not materialize in practice. The coarse mel design nonetheless provides a substantial efficiency advantage, reducing the model size from 7.3M to 3.75M parameters while achieving comparable accuracy.

3.3 Model Architecture

The model consists of two independent transformer encoders [48] connected through a learned similarity-based matching mechanism—a dual-encoder design analogous to cross-modal embedding-matching architectures [49]—with approximately 3.75M parameters.

3.3.1 Asymmetric Dual-Encoder Design

Table 3.1. Architectural comparison of the two encoders. The critical asymmetry is that only the vocal encoder receives positional embeddings.

Component	Vocal Encoder	Instrumental Encoder
Input projection	Linear(1152, 256)	Linear(1152, 256)
Positional embedding [50]	Learned (1, 8, 256)	None
Transformer layers	2 layers, 4 heads	2 layers, 4 heads
FFN dimension	1024	1024
Dropout	0.1	0.1

The asymmetry is intentional and critical:

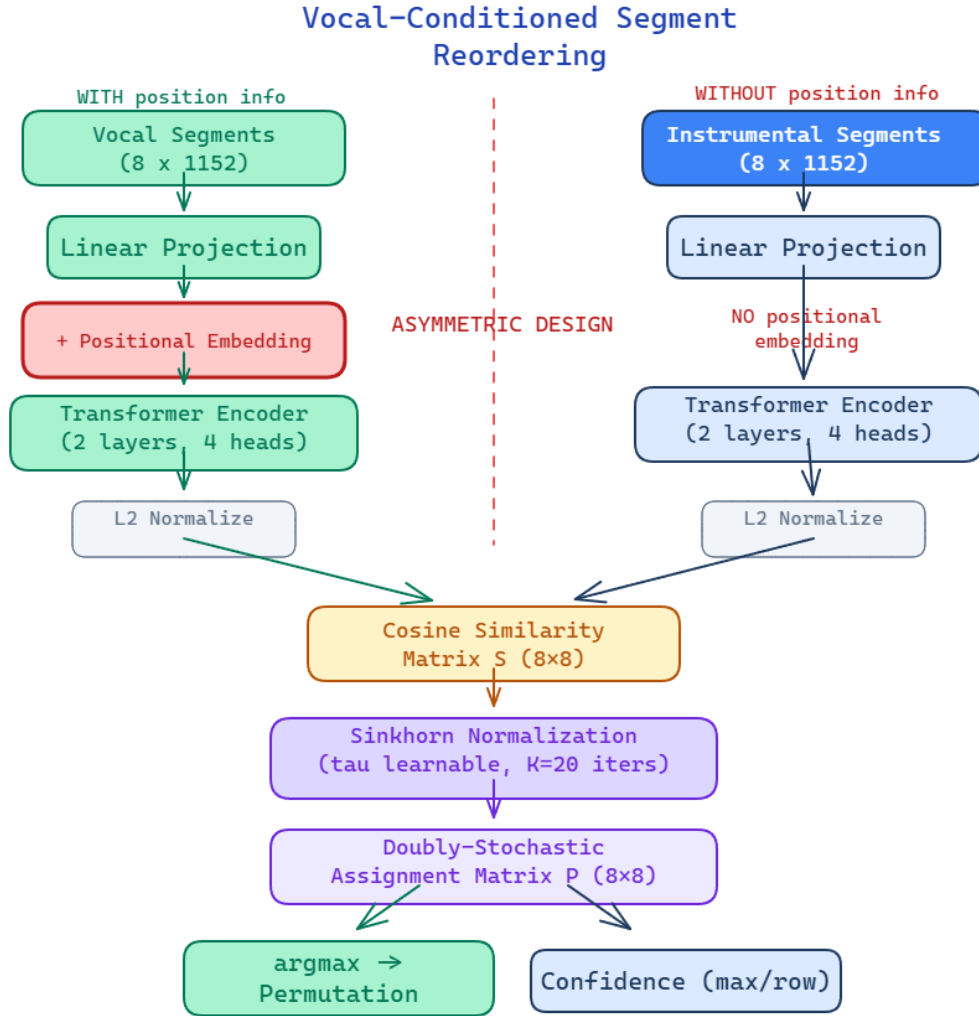


Figure 3.2. Architecture of the vocal-conditioned segment reordering model. The vocal encoder (left, green) receives learned positional embeddings; the instrumental encoder (right, blue) does not. This asymmetry forces the model to use the vocal as a positional template while matching instrumentals by content alone.

- **Vocal encoder with positional embeddings:** The model learns that position 0 typically corresponds to an introduction or verse start, position 3 to a pre-chorus or energy build-up, etc. The positional embedding produces position-aware representations—“what kind of content belongs at this position?”
- **Instrumental encoder without positional embeddings:** Since the instrumentals are shuffled, their absolute positions carry no information. The encoder produces position-invariant representations that capture *what* a segment sounds like, independent of *where* it appears.

3.3.2 Matching and Assignment

After encoding, the matching proceeds in three steps:

1. **L2 normalization:** Both sets of embeddings are normalized to unit length, establishing a cosine similarity space.
2. **Similarity computation:**

$$\mathbf{S} = \mathbf{E}_{\text{inst}} \mathbf{E}_{\text{vocal}}^\top \in \mathbb{R}^{8 \times 8} \quad (3.4)$$

where:

- $\mathbf{E}_{\text{inst}} \in \mathbb{R}^{8 \times 256}$ is the matrix of instrumental segment embeddings (one per row).
 - $\mathbf{E}_{\text{vocal}} \in \mathbb{R}^{8 \times 256}$ is the matrix of vocal segment embeddings (one per row).
 - $\mathbf{S}[i, j]$ represents how well instrumental segment i matches vocal position j (cosine similarity, since both are L2-normalized).
3. **Sinkhorn assignment:** To produce a valid one-to-one permutation, we apply Sinkhorn normalization [51, 52, 53] to the assignment logits $\mathbf{L} = \mathbf{S}^\top$:

$$\mathbf{P} = \text{Sinkhorn}(\mathbf{L}/\tau, K) \quad (3.5)$$

where:

- $\mathbf{P} \in \mathbb{R}^{8 \times 8}$ is the resulting soft assignment matrix, with $P[j, i]$ representing the probability that output position j should contain input segment i .
- $\mathbf{L} = \mathbf{S}^\top$ is the transposed similarity matrix, so that rows correspond to output positions and columns to input segments.
- τ is a learnable temperature parameter (effective value $|\tau| + 0.01$) that controls assignment sharpness: low τ produces near-hard (confident) assignments, high τ produces soft (uncertain) distributions.
- $K = 20$ is the number of Sinkhorn iterations (alternating row and column normalization steps).

Each Sinkhorn iteration alternates between row and column normalization in log space:

$$\mathbf{L}^{(k+1)} = \mathbf{L}^{(k)} - \text{logsumexp}(\mathbf{L}^{(k)}, \text{dim} = -1) \quad (3.6)$$

$$\mathbf{L}^{(k+1)} = \mathbf{L}^{(k+1)} - \text{logsumexp}(\mathbf{L}^{(k+1)}, \text{dim} = -2) \quad (3.7)$$

where:

- $\mathbf{L}^{(k)}$ is the log-domain assignment matrix at iteration k .
- $\text{logsumexp}(\cdot, \text{dim} = -1)$ computes the log of the sum of exponentials along the last (column) dimension, effectively normalizing each row to sum to 1 in probability space.
- $\text{logsumexp}(\cdot, \text{dim} = -2)$ performs the analogous normalization along the second-to-last (row) dimension, normalizing each column to sum to 1.

The resulting matrix $\mathbf{P} = \exp(\mathbf{L}^{(K)})$ is doubly-stochastic: every row and column sums to 1. This is the continuous relaxation of a permutation matrix, enabling gradient-based optimization while enforcing the one-to-one assignment constraint.

Sinkhorn normalization is fully differentiable, enabling end-to-end gradient flow through the assignment stage during training.

3.4 Training

3.4.1 Self-Supervised Pair Generation

Training data is generated from naturally aligned vocal–instrumental pairs without any human annotation, following the self-supervised pretext-task paradigm of solving artificially constructed ordering problems [54, 55]. For each training sample:

1. Load a matched vocal/instrumental pair from the dataset (same song, same time segment)
2. Generate a random permutation: $\pi = \text{randperm}(8)$

3. Shuffle the instrumental segments: $\mathbf{A}_{\text{shuffled}} = \mathbf{A}[\pi]$
4. The model predicts the assignment to unshuffle them

The ground truth is the inverse permutation π^{-1} . During training, the soft assignment matrix produces a differentiable “soft reorder”:

$$\tilde{a}_j = \sum_{i=0}^7 P[j, i] \cdot a_i \quad (3.8)$$

where:

- $\tilde{a}_j$ is the reconstructed (soft-reordered) feature at output position j —a weighted combination of all 8 input segments.
- $P[j, i]$ is the Sinkhorn-normalized probability that output position j should contain input segment i .
- a_i is the feature vector of the i -th input (shuffled) segment.

The reconstruction loss is:

$$\mathcal{L} = \|\tilde{\mathbf{A}} - \mathbf{A}_{\text{original}}\|_2^2 \quad (3.9)$$

where:

- $\mathcal{L}$ is the training loss.
- $\tilde{\mathbf{A}} = \{\tilde{a}_0, \dots, \tilde{a}_7\}$ is the stack of soft-reordered segments produced by the model.
- $\mathbf{A}_{\text{original}}$ is the ground-truth unshuffled sequence of segments.
- $\|\cdot\|_2^2$ is the squared L_2 norm, summing squared element-wise differences.

This MSE formulation provides smooth gradients even for partially correct assignments, which is preferable to direct permutation supervision (e.g., cross-entropy over hard labels) that would only reward exact matches.

Table 3.2. Training configuration for the reordering model.

Parameter	Value
Training samples	$\sim 260,000$
Validation samples	$\sim 29,000$
Batch size	128
Optimizer	AdamW
Learning rate	3×10^{-4}
LR schedule	Cosine annealing, $T_{\max} = 100$
Precision	Mixed (AMP)
Convergence	~ 40 epochs
Random baseline	12.5% (1/8)

3.4.2 Dataset and Training Configuration

The training data is derived from a BPM-normalized pop music dataset containing approximately 572,000 song chunks at 120 BPM. Each chunk provides aligned vocal and instrumental stems from which mel features are extracted per 1-second segment. All audio entering the model has already been BPM-normalized to 120 BPM via Rubberband time-stretching, grid-aligned at the downbeat level using the local beat warping method described in Chapter 2, and pitch-shifted to a compatible key.

3.4.3 Multi-Scale Training as Compatibility-Encoder Learning

The architecture described above is agnostic to the temporal granularity of its segments: the encoders, the Sinkhorn assignment stage, and the training objective operate on any sequence of eight segment feature vectors, and only the input projection layer changes with segment length. For a segment of k seconds, the zero-overlap mel configuration of Section 3.2 yields $8k + 1$ frames and hence a feature dimension of $128 \times (8k + 1)$. We exploit this property to train a family of models at three granularities—1, 4, and 8 seconds per segment, corresponding to half a bar, two bars, and four bars at the normalized 120 BPM tempo. Because the model always operates on eight segments, the effective musical context grows with scale: the 1-second model observes a single 4-bar phrase, while the 8-second model observes eight 4-bar sections spanning 64 seconds—roughly a third of a typical pop song per window.

The motivation for the coarser scales is not only reordering. The self-supervised unshuffling objective can only be solved by embedding vocal and instrumental seg-

ments into a *shared matching space*: the model succeeds exactly when the cosine similarity between an instrumental segment’s embedding and a vocal position’s embedding reflects how well that content belongs with that vocal. Training the reordering task therefore doubles as training a pair of *vocal–instrumental compatibility encoders*, without any human annotation of compatibility. At the finest scale the encoders are used together with the Sinkhorn assignment as a reordering model; at coarser scales the same trained encoders can be used bare, as feature extractors whose similarity space directly scores the compatibility of whole sections (Section 3.6). This dual reading—assignment machinery for rearrangement, raw embedding similarity for selection—is the basis of the hierarchical application in Section 3.6.

All scales are trained on the same corpus with identical optimization settings (AdamW, learning rate 3×10^{-4} , cosine annealing, early stopping with patience 8), differing only in input dimension and consequently parameter count. Table 3.3 summarizes the family.

Table 3.3. Multi-scale training summary. 95% confidence intervals are Wilson intervals over per-position placement decisions ($8 \times n_{\text{val}}$); the random baseline is 12.5% at every scale.

Scale	Unit @120	Context	Feat. dim	Params	Train/val	Train acc	Val acc (95% CI)
1 s	half bar	8 s	1,152	3.75M	260k / 29k	98.12%	89.24% [89.1, 89.4]
4 s	2 bars	32 s	4,224	5.32M	158.9k / 8.5k	99.37%	93.94% [93.8, 94.1]
8 s	4 bars	64 s	8,320	7.42M	71.3k / 3.8k	98.82%	91.91% [91.6, 92.2]

3.5 Results

3.5.1 Segment Placement Accuracy

Table 3.4. Reordering accuracy on same-song pairs (standard evaluation). The model correctly places segments at nearly 9 out of 10 positions.

Condition	Accuracy	vs. Baseline
Random baseline	12.50%	—
Full model (val)	89.24%	+76.74 pp
Full model (train)	98.12%	+85.62 pp

The model achieves 89.24% segment placement accuracy on the validation set—compared to a 12.5% random baseline (1 in 8 chance)—demonstrating that

it learns meaningful structural correspondences between vocal and instrumental content. This is measured on held-out *same-song* pairs, the only setting in which a ground-truth permutation is defined; Section 3.8.1 addresses what it implies for cross-song use.

3.5.2 Accuracy Across Scales

Validation reordering accuracy remains far above the 12.5% random baseline at every granularity—89.2%, 93.9%, and 91.9% for the 1s, 4s, and 8s models respectively—with tight binomial confidence intervals owing to the large number of placement decisions evaluated (Table 3.3). The moderate train–validation gaps (4–6 points) indicate that the models generalize rather than memorize. The non-monotonic accuracy profile across scales reflects a trade-off between per-segment feature richness, which favors coarser segments, and training-set size, which favors finer segments that yield more chunks per song. The consistency of accuracy across granularities is the key result: vocal–accompaniment structural correspondence is learnable at the half-bar, phrase, and section levels alike, which Section 3.6 exploits by assigning different pipeline roles to different scales.

3.5.3 Ablation Studies

To understand the contributions of each design decision, we conduct ablation experiments across both vocal conditioning and feature design. We compare our coarse mel model (3.75M parameters) against an identically structured model trained with standard mel features ($n_{\text{fft}} = 2048$, $\text{hop} = 512$, $\text{dim} = 8064$), which results in a larger model (7.3M parameters) due to the wider input projection.

Table 3.5. Ablation study comparing coarse mel and standard mel features across vocal conditioning conditions. Both models show similar patterns, with the vocal signal providing the dominant structural cue.

Condition	Coarse Mel (3.75M)	Standard Mel (7.3M)
Normal vocal	89.24%	92.95%
Shuffled vocal	87.88%	87.96%
Zeroed vocal	28.88%	23.69%
Random noise vocal	16.09%	15.37%
Random baseline	12.50%	12.50%

Vocal conditioning. Both models show a sharp drop when the vocal signal is removed: accuracy falls from $\sim 89\text{--}93\%$ (normal vocal) to $\sim 24\text{--}29\%$ (zeroed vocal) and $\sim 15\text{--}16\%$ (random noise). This confirms that the vocal template provides the dominant structural signal for segment reordering, regardless of feature resolution. The zeroed-vocal condition achieves above-random performance ($24\text{--}29\%$), indicating that the instrumental encoder learns some ordering biases from the accompaniment alone (e.g., quiet segments tend to occur at boundaries).

Shuffled vocal. When the vocal segments are randomly shuffled—so that the positional embeddings no longer correspond to the actual content at each position—both models maintain high accuracy ($\sim 88\%$). This indicates that the models primarily match by content similarity between vocal and instrumental segments rather than relying on positional shortcuts.

Feature design. An initial concern motivating the coarse mel design was that standard mel features, with their $7\times$ higher temporal resolution, could enable the model to exploit “temporal fingerprints”—fine transient patterns, spectral leakage artifacts, and waveform boundary signatures—as matching shortcuts that would not generalize to cross-song scenarios. However, the results show that the standard mel model does not exhibit this behavior: its accuracy under shuffled vocals (87.96%) is virtually identical to the coarse mel model (87.88%), and its degradation patterns under zeroed and noise conditions are consistent.

We hypothesize that the temporal fingerprinting concern is mitigated by the aggressive dimensionality reduction in the model’s input projection layer. The standard mel model projects 8064 features down to 256 dimensions—a $31.5\times$ compression—in the first linear layer. This bottleneck forces the network to discard fine temporal details and retain only the most informative spectral patterns, effectively learning a compressed representation similar to what the coarse mel provides by design. The standard mel model achieves slightly higher accuracy on normal vocals (92.95% vs. 89.24%), likely due to the richer input signal available before compression. However, the coarse mel design achieves comparable performance with roughly half the parameters (3.75M vs. 7.3M), representing a more efficient architectural choice for this task.

3.6 Application to Mashup Generation

The reordering model serves as a structural alignment component in the mashup pipeline, operating after temporal alignment (Chapter 2) and before mix engineering (Chapter 4). The multi-scale family is applied coarse-to-fine: the 8s-scale encoders select the mashup material by learned compatibility, and the 1s-scale model rearranges within the selection.

3.6.1 Mashup Generation Process

The mashup generation process proceeds as follows:

1. **Stem preparation.** The base song's vocal and the complementary song's harmonics (bass + other instrumentals combined) and drums are separated using Demucs [7]. Both songs are BPM-normalized to 120 BPM and key-matched.

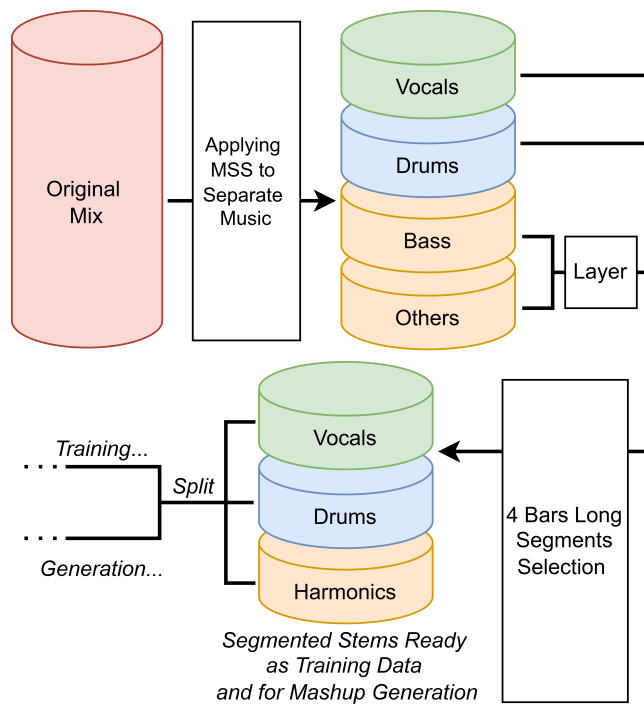


Figure 3.3. Data preparation pipeline. Original songs are separated into four stems via MSS, then reorganized into three stems (vocals, drums, harmonics). Segments are selected starting from the downbeat nearest the chorus and used for both training and mashup generation.

2. **Vocal-anchored section scan.** Given the base song's chosen 8-second vocal section—the chorus located by the beat pattern extraction of Chapter 2, or a

user-chosen section—the scan uses the trained 8s-scale model’s two encoders directly as a compatibility embedding space (Section 3.4.3). The vocal section is encoded as a single token, using the mean of the learned positional embeddings, into a vector e_{vocal} . The complementary song’s accompaniment is then scanned with an 8-second sliding window at 1-second (half-bar) hops on the downbeat grid; each candidate window is encoded as a single token by the instrumental encoder—which carries no positional embeddings—yielding one embedding $e_{\text{win}(t)}$ per window start t . Each window is scored by

$$c(t) = \cos(e_{\text{win}(t)}, e_{\text{vocal}}), \quad (3.10)$$

and the selected window is $t^* = \operatorname{argmax}_t c(t)$; all window encodings batch into a single forward pass, so the entire scan costs two passes (Figure 3.4). No assignment stage is involved: the Sinkhorn machinery exists to resolve competition among ordered template positions, and with a single query there is nothing to compete—the raw similarity in the learned matching space *is* the compatibility. In contrast to chorus detection, which localizes a salient section in each song independently, the scan is intrinsically pairwise: it finds the section of the complementary song that best fits *this* vocal. On a representative song pair (Figure 3.5), the sweep cleanly separates compatible groove regions ($c \approx 0.2$ – 0.3) from sparse breakdowns ($c \approx 0$). The selected window provides both the accompaniment material and the drum source, preserving the beat alignment of Chapter 2. Section 3.6.5 analyzes the design choices of the scan—the encoding regime, the positional-embedding treatment, and the choice of similarity readout.

3. **Segment rearrangement.** The reordering model takes the base song’s ordered vocal segments as the structural template and predicts the optimal permutation for the complementary song’s harmonic segments. Concretely, the selected 8-second window is divided into eight 1-second (half-bar) segments and rearranged against the vocal template in a single Sinkhorn window; here, unlike in the section scan, the one-to-one constraint is active and essential, as this step performs the actual permutation. The rearranged harmonics are reconstructed as a looping 2-bar phrase [35] that repeats, ensuring:

- **Catchiness:** Clear phrases serve as effective hooks, enhancing memorabil-

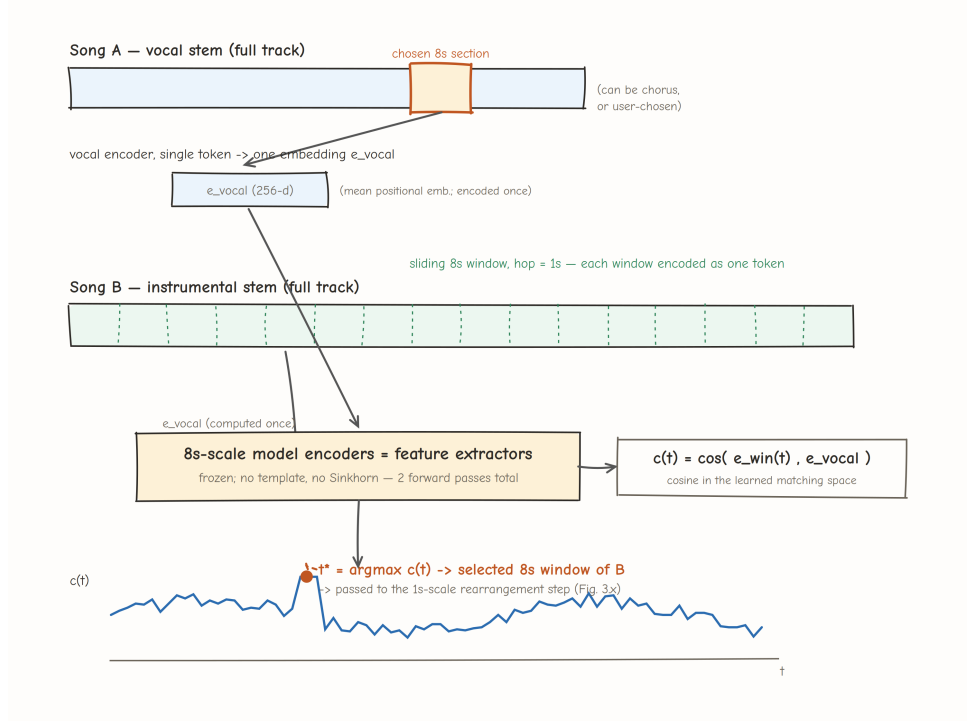
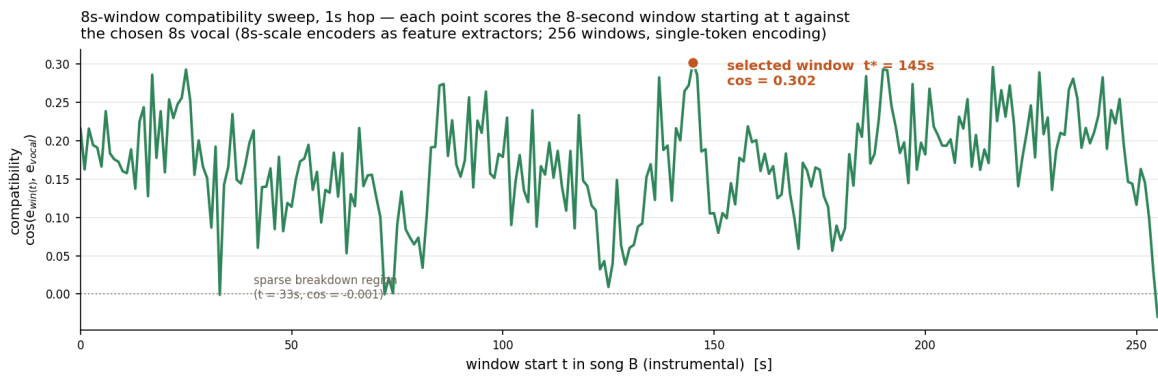


Figure 3.4. Vocal-anchored section scan. The frozen encoders of the 8s-scale model act as a compatibility embedding space: the chosen vocal section and every 8-second window of the complementary song’s accompaniment (1-second hop) are embedded, and windows are ranked by cosine similarity.



ity.

- **Rhythmic balance:** A looping structure maintains stability and engages listeners.
- **Harmonic resolution:** The format ensures that the reconstructed harmonics deliver a satisfying progression without a sense of incompleteness.

4. **Final assembly.** The base song’s vocals are layered with the rearranged harmonics and the complementary song’s drum segments. The drum segments are synchronized using the beat alignment from Chapter 2.

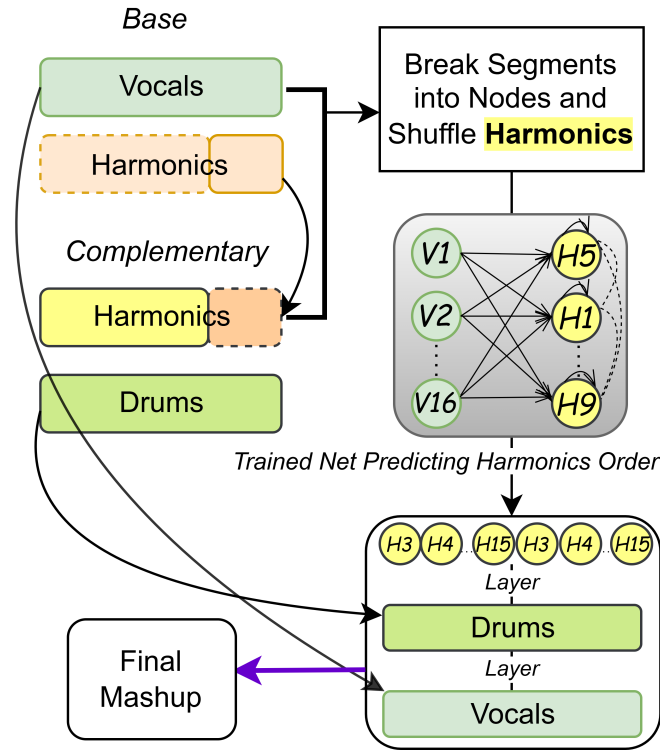


Figure 3.6. Mashup generation process. The base song’s vocal and a portion of its harmonics are combined with the complementary song’s shuffled harmonics, which are rearranged by the trained model. The reconstructed harmonics are layered with the complementary song’s drums and the base song’s vocals to produce the final mashup.

3.6.2 Cross-Song Accompaniment Alignment

Beyond same-song rearrangement, the model enables cross-song structural alignment. Given Song A’s vocal (8 segments) and Song B’s accompaniment (8 segments

from a different song), the model reorders Song B’s segments to match Song A’s structure. The argmax of the assignment matrix provides the permutation, while the maximum probability per position indicates confidence.

3.6.3 Multi-Window Selection

When the complementary song provides an extended accompaniment (e.g., 24 seconds = 24 segments), the model is applied on sliding windows of 8 segments. For each vocal position, the best-scoring accompaniment segment across all windows is selected, enabling richer material selection from a larger pool.

3.6.4 Compatibility Scoring

The average assignment confidence across all 8 positions serves as an automatic compatibility score between any two songs. Song pairs with consistently high confidence across positions are likely to produce better mashups, enabling automated song-pair recommendation without requiring listening tests.

The compatibility readout is scale-dependent by design. At the 8-second scale, raw embedding similarity is the section-selection signal of the generation pipeline, and across candidate songs the maximum window score ranks song pairings for automated recommendation. At the 1-second scale, where an ordered template competes for candidate segments and an actual permutation is executed, the Sinkhorn-constrained assignment confidence is the appropriate signal: the one-to-one structure carries information precisely where a permutation will be performed, and is vacuous where the query is a single section (Section 3.6.5).

3.6.5 Analysis of the Section Scan

Robustness to the encoding regime. The instrumental encoder is position-free, so a section can be embedded in three regimes: independently as a single token, within a sliding context of eight consecutive sections, or with the entire song as one sequence. To verify that the scan does not depend on this choice, we score all sections of the representative song pair under each regime. All three select the same section, and their full rankings agree with pairwise rank correlations of $\rho = 0.91\text{--}0.99$; the full-song

regime, which additionally embeds every section in a single forward pass, agrees with the windowed regime at $\rho = 0.994$ and is adopted as the default. Since the encoder is trained on eight-token inputs, the single-token and full-sequence regimes operate outside the training sequence length; the cross-regime agreement indicates that the learned matching space transfers across these input lengths.

Positional-embedding treatment. The vocal encoder adds a learned positional embedding to its input, so encoding a single vocal section requires choosing which embedding to apply. We compare encoding the section with each of the eight learned slots, with their mean, and with the embedding omitted entirely: all ten variants select the same section, and their rankings agree at $\rho \geq 0.996$. The learned embeddings remain small relative to the content features, so the choice is immaterial; the mean is adopted as the default because it keeps the input closest to the training distribution. Encoding the section once with the mean embedding is also exactly equivalent to filling all eight template slots with the replicated section, since self-attention over identical tokens collapses to the single-token case.

Choice of similarity readout. The model offers two confidence readouts: the raw cosine similarity between embeddings, and the Sinkhorn-constrained assignment confidence. The two are appropriate at different levels of the pipeline, and the distinction is measurable. Over the 256 windows of the representative pair, scoring each window by constrained assignment confidence (with the vocal section’s eight 1-second segments as an ordered template) and by unconstrained row-softmax confidence over the same similarities yields only weakly correlated sweeps (Pearson $r = 0.33$) that select different windows: the one-to-one constraint carries information exactly where positions compete for candidates. Conversely, the cosine sweep of the section scan correlates only weakly ($\rho = 0.18$) with the 1s-scale Sinkhorn-confidence sweep over the same windows—the two quantities measure different properties, section-level character versus fine-grained rearrangeability—so the section scan reads raw similarity, the rearrangement step reads constrained confidence, and the latter can additionally serve as a tie-breaker among the scan’s top-ranked windows. Figure 3.7 makes the constrained signal concrete: at a structurally compatible window the assignment matrix approaches a hard permutation, with each vocal position claiming a distinct segment at high confidence, while at an incompatible window the matrix remains diffuse; the gap between these two regimes is precisely

the confidence signal used throughout this section.

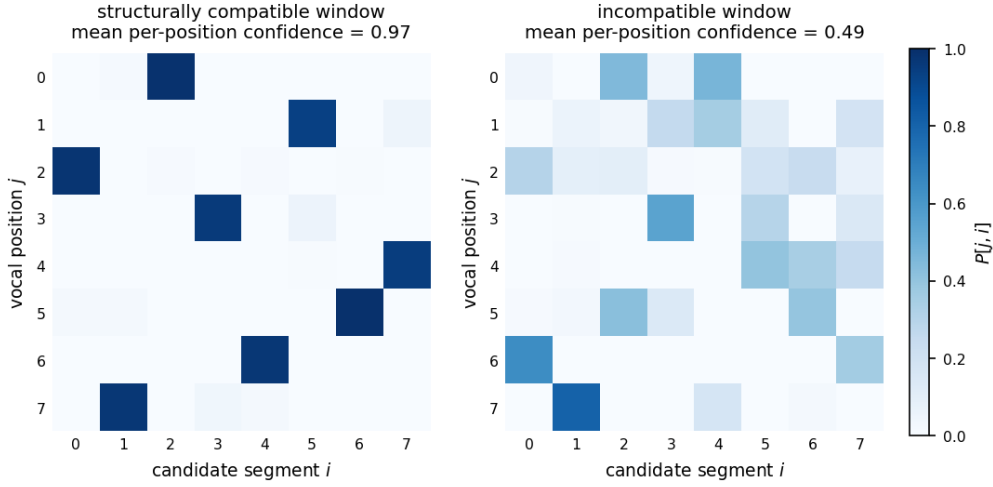


Figure 3.7. Assignment matrices of the 1s-scale model at two windows of the representative song pair. At a structurally compatible window (left) the Sinkhorn assignment approaches a hard permutation—each vocal position claims a distinct candidate segment with high confidence—while at an incompatible window (right) the assignment remains diffuse. The gap between the two regimes is the confidence signal used for compatibility scoring.

Computational cost. The scan is inexpensive: the vocal is encoded once, all candidate windows batch into a single forward pass through the 7.42M-parameter 8s-scale encoders, and a full song (256 windows) is scanned in seconds on a CPU. The subsequent rearrangement adds one forward pass of the 3.75M-parameter 1s-scale model, so the complete selection-and-rearrangement stage runs comfortably in real time and requires no GPU at inference.

3.7 Subjective Listening Test

To evaluate the perceptual quality of mashups generated using segment rearrangement, we conducted a web-based listening test. The test compared mashups generated by our rearrangement model against a mashability baseline [10] and the original base songs.

3.7.1 Experiment Setup

Five distinct categories of song pairs were selected from ten songs unseen during training, grouped by variations in tempo and key:

Table 3.6. Song pair categories for subjective evaluation. Each pair was tested with both songs serving as base and complementary, yielding 10 mashup evaluations.

Category	Tempo (BPM)	Key Shift (semitones)
Similar Key, Distinct Tempo	105 / 150	0
Similar Key, Similar Tempo	104 / 118	0
Moderately Distinct Key and Tempo	117 / 150	3
Distinct Key, Similar Tempo	124 / 128	5
Distinct Key, Distinct Tempo	110 / 174	5

For the baseline, we used the mashability algorithm to identify the most compatible segment from the complementary song for the same vocal segment used in our model’s generation.

3.7.2 Evaluation Criteria

Thirty college student participants with musical backgrounds (aged 19–25) evaluated the mashups. Each participant listened to three randomized versions per song pair—the original, the mashability baseline, and our model’s output—and rated each on a 1–5 scale across four criteria:

- **Rhythm:** How well do the vocals and instrumentals align rhythmically?
- **Harmony:** How compatible are the harmonic elements?
- **Creativity:** Does the combination sound musically fresh and engaging?
- **Overall Compatibility:** Does it resemble a professionally produced piece?

3.7.3 Results

Table 3.7. Mean subjective ratings (1–5 scale) across 30 participants and 10 mashup evaluations.

Criterion	Mashability	Ours	Original
Rhythm	2.67	3.51	4.06
Harmony	2.74	3.31	3.89
Creativity	2.73	3.24	3.62
Overall	2.63	3.38	3.93

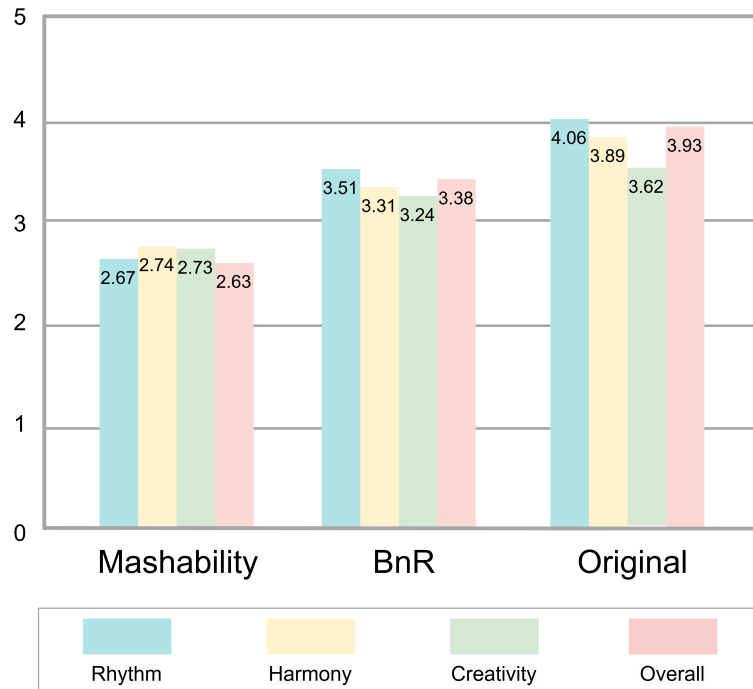


Figure 3.8. Subjective ratings comparing the mashability baseline, our rearrangement model, and original songs across four evaluation criteria (1–5 scale). The rearrangement model significantly outperforms the mashability baseline across all criteria.

Our rearrangement model significantly outperformed the mashability baseline across all four evaluation criteria, with the original songs serving as an upper-bound benchmark. Notably, both the original and our model’s ratings show rhythm as the highest-rated aspect, while the baseline model does not exhibit this trend—suggesting that segment-level rearrangement produces mashups that are perceived as rhythmically coherent in a way that simple compatibility-based segment selection cannot achieve.

The harmony compatibility scores for our model (3.31), while slightly lower than rhythm (3.51), still substantially surpass the baseline (2.74). Creativity received the lowest scores across all conditions, which may reflect the listening test design—participants were not presented with the complementary song’s original instrumentals, limiting their ability to assess how creatively the material was rearranged.

Overall, our model achieves ratings approximately 0.6–0.8 points above the baseline across all criteria, closing roughly half the gap between the baseline and the orig-

inal songs. This demonstrates that granular instrumental rearrangement produces a markedly more engaging listening experience compared to rule-based segment selection.

3.8 Discussion

3.8.1 Same-Song Training and Cross-Song Application

The model is trained on same-song pairs and applied to cross-song ones. The asymmetry is structural rather than incidental: the correct permutation is defined by the recording that produced it, so supervision exists only where vocal and accompaniment share a song. For a cross-song mashup there is no ground truth and none can be constructed, since a corpus of correct mashups would already encode the judgment the system is built to make. Cross-song quality is accordingly established by listeners (Section 3.7) rather than by placement accuracy, and the accuracies of Section 3.5 should be read as evidence that vocal–accompaniment structural correspondence is learnable, not as an estimate of mashup quality.

What can be controlled is whether the model solves the pretext task by musical correspondence or by cues peculiar to a shared production. Four design decisions, each introduced earlier for its own reasons, together remove the obvious ones.

Tempo is normalized on both sides. Training and inference alike operate at a normalized 120 BPM (Chapter 2). Tempo is therefore constant across every pair the model ever sees, and cannot act as a same-song cue or as a feature the model could overfit to.

Segments are complete metrical units. At 120 BPM a one-second segment is exactly half a bar, and segments are cut on detected downbeats. Each segment is thus a whole musical unit with a clean boundary: no partial bars, no material spanning a cut, and no splice residue that could be matched as an artifact of how the audio was divided.

Features are deliberately coarse. The zero-overlap coarse mel of Section 3.2.2 retains nine frames per second rather than sixty-three, discarding the transient detail and spectral-leakage signature by which a particular recording could be fingerprinted. The representation preserves harmonic and timbral character—the proper-

ties that transfer between songs—while removing those that identify a take. The ablation of Section 3.5.3 confirms that even the higher-resolution variant does not exploit such detail.

One-to-one content matching is verified. Shuffling the vocal segments, which decouples the positional embeddings from the content at each position, costs 1.4 points (89.24% $\rightarrow$ 87.88%); removing the vocal entirely costs sixty (28.88%). The model is therefore matching instrumental content against vocal content, position by position, rather than relying on where sections conventionally fall.

Taken together, these establish that the learned correspondence rests on harmonic and structural content rather than on production provenance or segmentation artifacts—the properties that carry across songs. The cross-song listening results are consistent with this reading: mashups assembled by the model outperform the mashability baseline on all four criteria (Table 3.7).

3.8.2 Limitations

The main practical constraint of the current model is its fixed temporal grid: segments are one second long at the normalized tempo of 120 BPM, so both songs must be time-stretched to 120 BPM (Chapter 2) before feature extraction, and rearrangement operates on half-bar units of that grid. Since tempo normalization is already part of the alignment pipeline, this costs no additional processing in practice; however, material whose feel degrades under time-stretching—extreme tempi or rubato performances—inherits that degradation.

3.9 Chapter Summary

In this chapter, we presented a vocal-conditioned segment reordering model that addresses the structural mismatch problem in music mashups. The model uses an asymmetric dual-encoder transformer with Sinkhorn-based differentiable permutation learning, operating on deliberately coarsened mel-spectrogram features that force semantic matching over temporal fingerprinting. Trained self-supervisedly on approximately 260,000 vocal–instrumental pairs, the model achieves 89.24% segment placement accuracy on held-out same-song pairs at the finest scale, and the

same architecture trained at three granularities (1, 4, and 8 seconds) maintains 89–94% accuracy from the half-bar to the section level. Ablation studies confirm that vocal conditioning provides the dominant structural signal and that the coarse mel design achieves comparable accuracy with roughly half the parameters. Beyond re-ordering, the trained encoders constitute a vocal–accompaniment compatibility embedding space: at the 8-second scale they drive a vocal-anchored section sweep that selects mashup material by learned pairwise compatibility, replacing heuristic chorus detection.

Subjective listening tests demonstrate that mashups generated with segment rearrangement significantly outperform mashability-based baselines across rhythm, harmony, creativity, and overall compatibility. Being cross-song, these tests carry the chapter’s evidence that the learned correspondence transfers beyond the same-song condition under which it was trained. These results validate the approach of modifying the musical content itself—rather than merely selecting compatible segments—to achieve deeper structural alignment in mashups.

However, even with correct structural alignment, a nontrivial mixing problem remains. The rearranged mashup combines stems from two different productions, each with distinct volume levels, frequency profiles, stereo imaging, and dynamic ranges. In the listening test evaluation above, we applied a fixed preset of audio compression and reverb to the final mix to achieve a listenable result—a pragmatic but limited solution that does not adapt to the specific sonic characteristics of each mashup pair. The reliance on static mixing presets represents a clear limitation: a compressor tuned for one song pair may over-compress or under-compress another, and a single reverb setting cannot account for the varying spatial characteristics of different productions. The following chapter addresses this mixing challenge directly, developing a learned model for adaptive frequency equalization, volume balancing, and spectral blending to produce polished mashup outputs.

CHAPTER 4

MIX ENGINEERING WITH RECTIFIED FLOW MODELS

The preceding chapters established methods for temporal alignment (Chapter 2) and structural rearrangement of instrumental segments (Chapter 3). However, as noted in the Chapter 3 evaluation, combining stems from different productions introduces nontrivial sonic imbalances—differences in volume levels, frequency profiles, and dynamic ranges—that static presets of compression and reverb cannot adequately address. Professional music mixing involves fine-tuning the levels, dynamics, and frequency content of musical elements to ensure clarity and harmony in the final production [4, 56, 5]. Spatial effects such as reverb and delay introduce depth and width to the mix [57, 58].

In this chapter, we present an automatic mixing system based on a rectified-flow bridge that restores the tonal balance and level of an instrumental so that it sits correctly under a given vocal. The model operates on the instrumental’s STFT magnitude only—exploiting the fact that the corruptions of interest preserve phase exactly—and is conditioned on the vocal and on a droppable, low-dimensional tonal-balance descriptor that makes the correction target both learnable and user-steerable via classifier-free guidance. Trained on a large internal corpus with synthetically introduced tone and level imbalances, the system produces near-transparent restorations and substantially outperforms existing automatic mixing approaches on a fully out-of-domain benchmark.

4.1 Related Work

4.1.1 Rule-Based Automatic Mixing

Rule-based models have long been used in audio processing, offering structured approaches to automated mixing. De Man and Reiss [59] developed a system that uses musical tags and audio samples as input, analyzing factors such as EQ, level, and

compression to suggest optimized effect settings. Everardo [60] proposed an Answer Set Programming logic-based system to encode professional audio engineering knowledge, focusing on volume, panning, and frequency balance in stereo mixing. These systems apply industry guidelines to adjust the depth (volume), width (panning), and height (frequency) of audio tracks, producing both a mixed audio file and a detailed mixing plan. While valuable as structured starting points, rule-based approaches do not easily generalize beyond their encoded heuristics.

4.1.2 Neural Network–Based Automatic Mixing

Recent advancements have introduced neural network-based systems for automatic music mixing. Martínez-Ramírez et al. [61] present a time-domain model that processes raw waveforms through three main components: an adaptive front-end, a latent-space mixer using temporal dilated convolutions [62] and BiLSTM layers, and a synthesis back-end. The adaptive front-end uses time-domain convolutions to decompose audio into frequency bands, and the latent-space mixer learns a mixing mask to adjust frequency and volume dynamics. Koszewski et al. [63] employed a Wave-U-Net [64] architecture with one-dimensional convolutions, encoding and decoding unmixed stems individually before combining them through a final mixing stage [65].

4.1.3 Diffusion and Flow Models for Audio

Denoising Diffusion Probabilistic Models (DDPM) [66] progressively add Gaussian noise to a data sample over multiple timesteps, then train a network to reverse this process. Welker et al. [67] applied conditional diffusion in the complex STFT domain for speech enhancement, and Lemercier et al. [68] provide a comprehensive survey of diffusion models for audio restoration across denoising, dereverberation, and bandwidth extension. More recently, flow matching [69] and rectified flow [70] reformulate generative transport as learning the velocity field of a deterministic interpolation between two distributions, unified with diffusion under the stochastic-interpolants framework [71]; image-to-image Schrödinger bridges [72] extend this to paired restoration, where the path connects a corrupted observation directly to its clean counterpart rather than to Gaussian noise. Our method instantiates this bridge

view for mix restoration: because the source of the bridge is the observed degraded signal—not noise—the sampler starts from the input itself, and every intermediate state of the few-step solver lies on the training path by construction.

4.2 Problem Definition

When stems from different songs are combined in a mashup, each stem carries the mixing characteristics of its original production—its own equalization curve, compression settings, loudness level, and spectral balance. Simply summing these stems produces a mix where some elements may be excessively loud, others buried, and frequency ranges may clash or leave gaps.

We ground the task in the mashup setting of the preceding chapters: the *vocal* is the anchor whose identity must be preserved verbatim, while the *instrumental*—separated from a different production—carries a foreign tonal balance and level. The system must therefore solve a *vocal-conditioned instrumental restoration* problem: given a broken mix (a mis-equalized, mis-leveled instrumental summed with the clean vocal), recover the mix in which the instrumental sits correctly under that vocal.

We frame this as signal restoration: given an imbalanced mix x , the model produces an output $\hat{y}$ close to a well-mixed reference y , measured by the Signal-to-Distortion Ratio (SDR) [73]:

$$\text{SDR}(y, \hat{y}) = 10 \log_{10} \frac{\|y\|^2}{\|y - \hat{y}\|^2} \quad (4.1)$$

where:

- y is the original well-mixed signal (reference).
- $\hat{y}$ is the model’s output (processed signal).
- $\|\cdot\|^2$ denotes the squared L_2 -norm (energy of the signal).

Because a single waveform-distance number conflates several perceptually distinct failure modes, we complement the scale-invariant form of this metric

(SI-SDR) [74] with a suite targeting the quantities a mixing engineer actually adjusts: mel-spectrogram error, multi-resolution STFT distance (MR-STFT), a 31-band *tonal-balance error* (mean $|\Delta\text{dB}|$ of the long-term band spectrum, the quantity displayed by commercial tonal-balance tools), and deltas of integrated loudness (LUFS), spectral centroid, and crest factor.

4.3 Per-Stem Diffusion Restoration

Before arriving at the vocal-conditioned formulation of this chapter, we first investigated mix restoration as an *unguided, per-stem* problem: each stem of a mix is corrupted independently, and a model must restore it with no external reference for what the corrected balance should be. This study established that a diffusion process can be adapted to imbalance correction—its formulation is well-suited to fixing *intra-stem* imbalances—and its findings directly shaped the design presented in the remainder of the chapter.

4.3.1 Setup

The study used the four well-mixed stems of MUSDB18 (drums, bass, vocals, others; 140 training and 10 test songs), with 15-second segments represented as complex STFTs (FFT size 4095, hop 646, shape [2, 2048, 1024] for real and imaginary channels). Imbalances were simulated by repeated random gain manipulations over random frequency bands and time spans (30–50 per stem), applied to *each stem independently*. This is a substantially harsher setting than corrupting the mix once: when the four independently corrupted stems are summed, their errors compound rather than average out, so the assembled imbalanced mix is severely degraded before any restoration is attempted.

4.3.2 Architecture

An Attention-UNet backbone predicts the corruption directly in the complex domain, treating the real and imaginary components as channels—valid because a complex Gaussian with independent components is equivalent to a two-dimensional real Gaussian [67]. The architecture consists of three primary components:

ConvBlock. Two sequential 3×3 convolutions, each followed by Group Normalization and GELU activation, with an optional residual connection adding the input to the output before the final activation.

Encoder. Each encoder level reduces spatial resolution by a factor of 4 via Max-Pool, applies two ConvBlocks (the first residual), and incorporates a timestep embedding t that is transformed and added element-wise to the feature map, conditioning the model on the diffusion timestep.

Decoder. Each decoder level mirrors its encoder counterpart with bilinear up-sampling (scale factor 4), two ConvBlocks, skip connections from the corresponding encoder level, and timestep conditioning. The bottleneck between encoder and decoder consists of three ConvBlocks, reducing the spatial shape to $(2, 1)$; self-attention is applied at the fourth and fifth encoder/decoder levels and in the bottleneck.

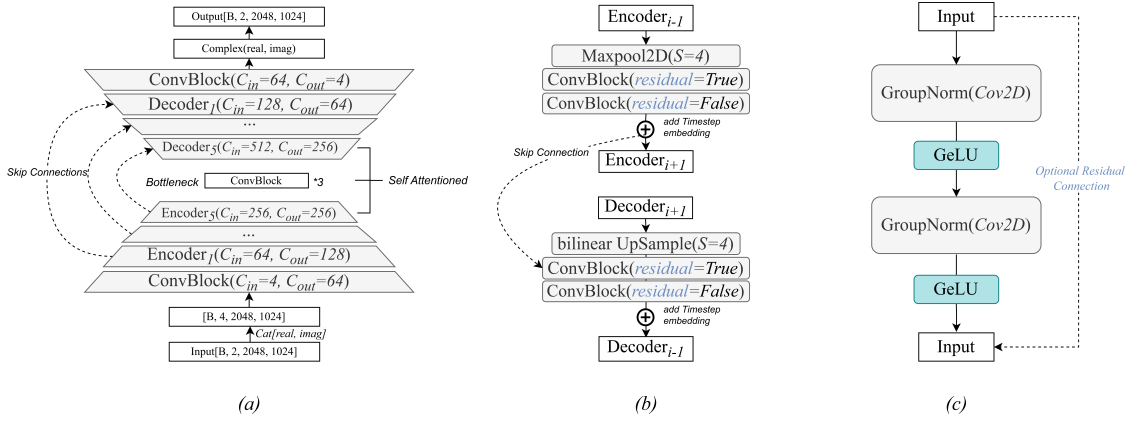


Figure 4.1. Architecture of the per-stem Attention-UNet backbone. (a) Overall encoder-decoder structure with skip connections. (b) Downsampling encoder and up-sampling decoder units. (c) The fundamental ConvBlock unit with optional residual connections.

4.3.3 Formulation

The diffusion process departs from standard DDPM [66] in both directions. The forward process models a noise term that couples the mixing imbalance with scheduled Gaussian noise:

$$N_t = n(x, y) + \mathcal{N}(0, \sigma_t^2), \quad X_t = N_t + y \quad (4.2)$$

where:

- $n(x, y) = \text{STFT}(x) - \text{STFT}(y)$ is the direct difference in the STFT domain between the imbalanced mix x and the clean mix y , representing the mixing imbalance the model must learn to remove.
- $\mathcal{N}(0, \sigma_t^2)$ is Gaussian noise with zero mean and variance controlled by the schedule parameter σ_t^2 .
- X_t is the noisy STFT input at timestep t , and y the clean (well-mixed) STFT reference.

The noise variance evolves over the continuous timestep $t \in [0, 1]$ according to:

$$\sigma(t)^2 = \sigma_{\min}^2 \left(\left(\frac{\sigma_{\max}}{\sigma_{\min}} \right)^{2t} - e^{-2\theta t} \right) \frac{\log(\sigma_{\max}/\sigma_{\min})}{\theta + \log(\sigma_{\max}/\sigma_{\min})} \quad (4.3)$$

where:

- $\sigma_{\min}$ and $\sigma_{\max}$ are the minimum and maximum noise levels, bounding the range of the schedule.
- θ controls the rate of noise decay across the $[0, 1]$ timestep interval.

The network predicts N_t under an L_1 loss, which is more robust to outliers than the L_2 loss of standard DDPM. The reverse process then starts from the imbalanced input itself—not from random noise—and at each of 100 steps removes the predicted noise while *re-injecting* the original input to preserve signal identity:

$$x_{t-1} = x_t - p \cdot \beta_t + \left(\mathcal{N}(0, \sigma_t^2) + (x_1 - (x_t - p)) \cdot \alpha_t \right) \quad (4.4)$$

where:

- x_t is the current input at timestep t , and x_{t-1} the updated input after noise reduction; t decreases continuously from 1 to 0.
- p is the noise predicted by the model at timestep t .
- β_t controls the step size of noise removal at timestep t .

- α_t controls the rate of re-injection of the original input, conditioning the reverse process on the source.
- $\mathcal{N}(0, \sigma_t^2)$ maintains stochasticity in the denoising trajectory.
- x_1 is the original imbalanced input (the starting point of the reverse process), and $(x_1 - (x_t - p))$ the residual noise that remains to be removed, progressively re-injected via the α_t weighting.

The re-injection term is the key departure from standard DDPM: because restoration must transform a *specific* input rather than generate freely, the trajectory is continually re-anchored to the source. This mechanism is what makes the formulation effective for intra-stem correction, where the signal’s own structure is the only available anchor.

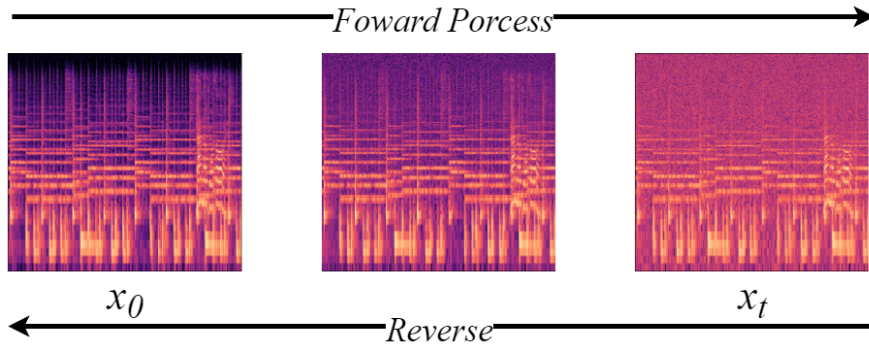


Figure 4.2. The per-stem diffusion process in the STFT domain: the forward process couples the imbalance with scheduled Gaussian noise; the reverse process denoises from the imbalanced input under re-injection conditioning.

4.3.4 Per-Stem System and Findings

Following Koszewski et al. [63], separate models processed each stem, with a final “mastering” model combining their outputs; the drums model was trained from scratch and the remaining models transfer-learned from it.

The compounding effect of per-stem corruption is visible in the *Imbalanced* column of Table 4.1: the assembled mix of four independently corrupted stems averages only 1.14 dB SDR, and drops below zero in the worst case—each stem’s errors add in the sum, leaving very little of the reference mix intact. Against this severely

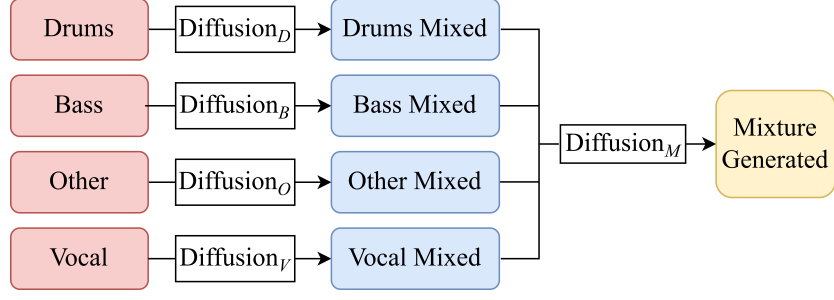


Figure 4.3. The per-stem mixing system: four stem-specific diffusion models followed by a mastering model.

Table 4.1. Per-stem study: SDR (dB) on the 10-song MUSDB test split (per-stem protocol).

SDR (dB)	Imbalanced	FxNorm-Automix	Wave-U-Net	Per-stem diffusion
Best	4.39	4.03	6.08	12.89
Mean	1.14	0.36	3.53	9.32
Worst	-2.12	-2.47	-2.58	6.99

degraded starting point, the per-stem models outperformed both baselines, confirming that a diffusion formulation anchored to the corrupted input can correct synthetic imbalances where feed-forward baselines fail or introduce artifacts.

4.3.5 Toward Real-World Mixing Issues

Three observations from this study set the agenda for the remainder of the chapter.

Unguided correction. With no reference for the intended balance, the model must infer a generic tonal target from the corrupted signal alone, which caps the attainable quality and offers the user no control over the outcome.

Mixing is inter-stem. The quality of a mix is defined by how its elements sit against one another—the instrumental against the vocal, not each stem against some notion of its own ideal—so a model confined to intra-stem correction cannot, even in principle, recover the balance that matters: that balance is a relation, and the relation is invisible from inside a single stem. What the task calls for is a model that studies these inter-stem relations directly, with an anchored reference—the vocal, in the mashup setting—defining what a correct mix means for the signal being restored.

A validated path to scale. The study’s central positive finding is that a diffusion-family formulation which *starts from the imbalanced mix* and moves toward the bal-

anced reference corrects imbalances more reliably than feed-forward alternatives; with the formulation validated on this small, controlled setting, the natural next step is to carry it to training at scale, where the corpus is no longer the limiting factor.

The remainder of the chapter builds on exactly these three points: the vocal becomes an explicit condition and anchor, so the instrumental is corrected *relative to* the signal it must sit under; a droppable tonal-balance descriptor turns the previously implicit target into an explicit, steerable control; and a single model is trained at scale on a corpus of roughly 100,000 songs, evaluated on a fully out-of-domain benchmark. The anchoring insight of the per-stem formulation—keeping the trajectory tied to the corrupted input—survives in a cleaner form: the bridge flow of Section 4.6 makes the input itself the *starting point* of the transport path, so anchoring is built into the formulation rather than added as a sampler correction.

4.4 Data Preparation

4.4.1 Dataset

We train on an internal corpus of approximately 100,000 professionally produced songs, decomposed by a state-of-the-art source separator into vocal/instrumental pairs whose sum reconstructs the original master. Random 8-second windows at 44,100 Hz (mono) are drawn from sharded storage with on-the-fly corruption, giving an effectively unbounded stream of (clean vocal, clean instrumental, broken instrumental) triples. The MUSDB18-HQ dataset [75] is reserved *entirely* for evaluation: none of its 150 songs appears in training, making it a fully out-of-domain benchmark.

4.4.2 STFT Representation

Each signal is transformed with an STFT of FFT size 4096 and hop 1024, yielding a 2048×352 time–frequency grid for 8-second mono segments (Nyquist bin dropped; time padded to a multiple of 2^5 for the network pooling). Both dimensions remain pooling-friendly, and the finer hop preserves onset detail at 44.1 kHz full band.

4.4.3 Imbalanced Input Simulation

Real-world production mismatch is dominantly *time-invariant*: a foreign instrumental arrives with the wrong tonal curve and the wrong level, not with rapidly time-varying artifacts. The corruption model reflects this: a random parametric-EQ curve of 2–4 log-frequency Gaussian bells with gains $\mathcal{U}(-10, 10)$ dB, random centers (80 Hz–9 kHz) and widths, plus a broadband volume error $\mathcal{U}(-6, 6)$ dB, applied to the instrumental magnitude at full strength on every training sample:

$$g(f) = 10^{\left(\sum_k a_k \exp\left(-\frac{(\log f - \mu_k)^2}{2w_k^2}\right) + v\right)/20}, \quad m_{\text{brk}} = g \odot m_{\text{cln}} \quad (4.5)$$

where:

- $m_{\text{cln}} = |\text{STFT}(\text{instrumental})|$ is the clean instrumental magnitude, and m_{brk} its corrupted version; $\odot$ applies the per-frequency gain $g(f)$ identically across all time frames.
- $a_k \sim \mathcal{U}(-10, 10)$ dB is the gain of the k -th bell; μ_k and w_k are its center and width on the log-frequency axis; the number of bells is drawn from $\{2, 3, 4\}$.
- $v \sim \mathcal{U}(-6, 6)$ dB is the broadband volume error, modeling an instrumental delivered at the wrong level relative to the vocal.

Because g is real and positive, the corruption changes magnitude only: the phase of the broken instrumental is **identical** to the clean one. This observation drives the entire architecture (Section 4.5).

4.5 Model Formulation and Architecture

4.5.1 Phase Reuse: Predict Magnitude Only

The corruption model implies a strong structural property: since the EQ/volume gain is real and positive, the clean instrumental phase is *already present* in the broken signal. The model therefore predicts only the instrumental **magnitude**, and reconstruction reuses the exact broken-instrumental phase:

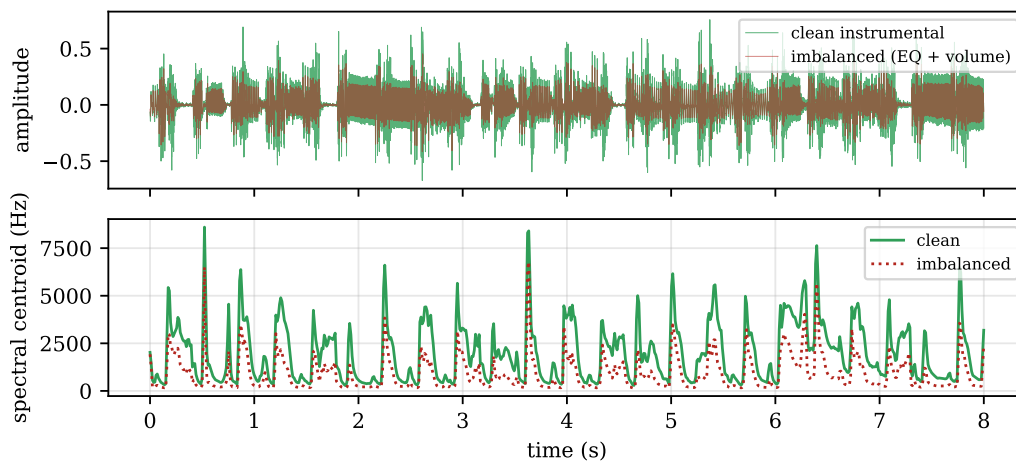


Figure 4.4. A clean instrumental and its imbalanced version under the corruption model of this chapter (EQ bells with a -5 dB volume error). The waveform overlay shows the broadband level change; the spectral centroid trace shows the tonal shift, constant over time—consistent with the time-invariant nature of production mismatch.

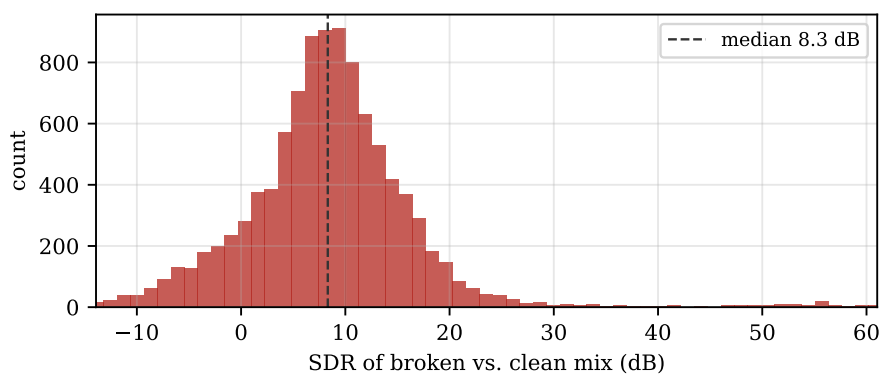


Figure 4.5. Distribution of SDR between the corrupted and clean mixes over 10,000 training windows drawn with the data-preparation pipeline above. Since corruption parameters are sampled afresh at every training step, this is a reference view of the difficulty distribution the model sees, not a fixed dataset: it spans near-transparent cases (mild bells, small volume error) through severely damaged mixes.

$$\hat{X}_{\text{inst}} = \hat{m} \cdot e^{j\angle(X_{\text{brk}} - X_{\text{voc}})}, \quad \hat{y} = \text{ISTFT}(\hat{X}_{\text{inst}} + X_{\text{voc}}) \quad (4.6)$$

where:

- $\hat{m}$ is the network’s estimate of the clean instrumental magnitude.
- X_{brk} and X_{voc} are the complex STFTs of the broken mix and the clean vocal; their difference is the broken instrumental, whose phase $\angle(\cdot)$ is reused unchanged.
- $\hat{X}_{\text{inst}}$ is the reconstructed complex instrumental, and $\hat{y}$ the output mix after adding the untouched vocal and inverting the STFT.

No Griffin–Lim, no phase estimation, and no phase error, structurally: predicting the complex STFT would require the network to reproduce phase that the input already carries exactly, so restricting prediction to the magnitude removes this redundancy and devotes the full model capacity to the one quantity the corruption actually alters.

4.5.2 Conditioning

Three conditions are concatenated as input planes alongside the state x_t (each 2048×352):

1. **Clean vocal magnitude.** The vocal spectrogram at its true level—the anchor against which the instrumental must be balanced.
2. **Tonal-balance map.** The target instrumental’s long-term band spectrum in 31 log-spaced bands (40 Hz–16 kHz) over two 4-second chunks, expressed in dB *relative to the vocal’s level*—so it encodes both the tonal shape and the instrumental-versus-vocal volume relation—painted across the frequency axis as an image plane.
3. **Indicator plane.** A binary plane marking whether the balance condition is supplied, which disambiguates a dropped condition from a genuinely zero-valued (flat, vocal-level) curve and keeps the conditioning dropout well-defined.

The balance condition is dropped (map and indicator zeroed) for 30% of training samples, classifier-free style [76]: when provided, it is a steerable target (computed from the ground-truth pairing during training, or from any reference at inference); when dropped, the model must infer a plausible balance for the vocal on its own.

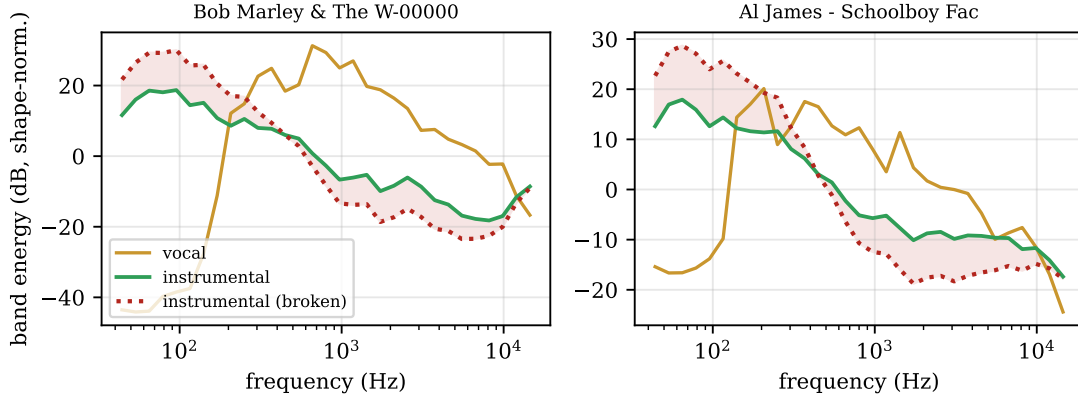


Figure 4.6. The tonal-balance representation (31 log-spaced bands, 40 Hz–16 kHz, band energy in dB) for two evaluation songs: clean vocal, clean instrumental, and the instrumental after a strong corruption (EQ ± 12 dB bells with a 5 dB volume offset; shaded region = the deviation to be corrected). The corruption is clearly legible in the descriptor while note-level musical content is not, making the representation an effective, content-blind conditioning target. In the conditioning plane the instrumental curve is expressed relative to the vocal’s level, so the descriptor carries the instrumental-versus-vocal volume relation as well as the tonal shape.

4.5.3 UNet Backbone

The backbone is a UNet [16] operating on the time–frequency grid. The network is purely convolutional (46 M parameters): the correction target—a smooth per-frequency gain—is well matched to the inductive bias of convolution and pooling, whose deepest level already spans the entire spectrum. It consists of three primary components:

ConvBlock. The fundamental building unit consists of two sequential 3×3 convolutions, each followed by Group Normalization and GELU activation; an optional residual connection [77] adds the input to the output before the final activation. The timestep embedding t (sinusoidal, passed through a two-layer MLP) is projected per block and added element-wise to the feature map, conditioning every stage on the bridge position.

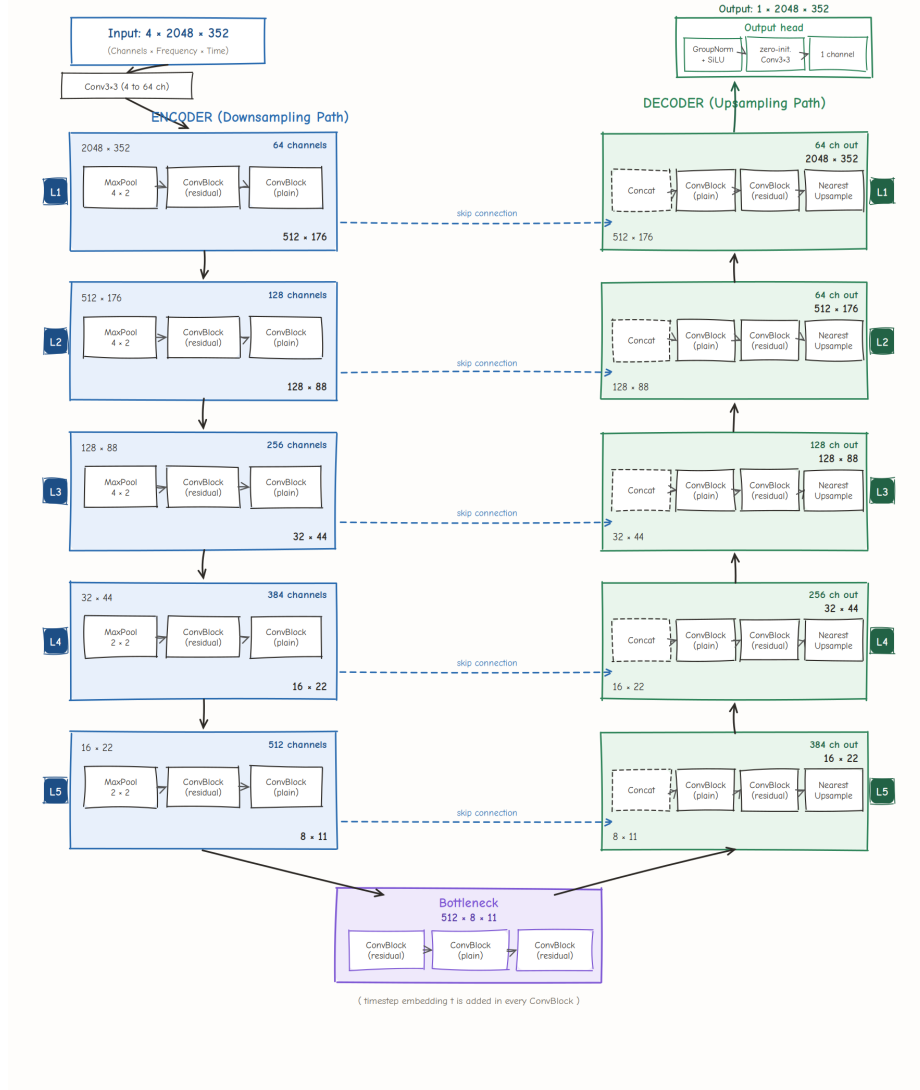


Figure 4.7. Architecture of the convolutional U-Net backbone. The four input planes pass through an input convolution and a five-level encoder with frequency-heavy pooling; the bottleneck operates at $512 \times 8 \times 11$; each decoder level concatenates its encoder skip connection at the current resolution, applies two ConvBlocks, and up-samples (nearest-neighbor) to the next skip’s resolution. The timestep embedding is added in every ConvBlock, and the zero-initialized output head produces the 1-channel velocity.

Encoder. Five levels with channel widths (64, 128, 256, 384, 512). Each level applies frequency-heavy MaxPool—(4×2) for the first three levels, (2×2) for the last two, so the 2048 frequency bins contract by 256× against 32× in time—followed by two ConvBlocks (the first residual). The aggressive frequency pooling reflects the task: a tonal-balance correction is smooth across frequency, and the deepest feature map (512 × 8 × 11) lets a single convolution relate energy across the entire spectrum.

Decoder. Each level first concatenates the skip connection from its encoder counterpart at the current resolution, applies two ConvBlocks with timestep conditioning, and then upsamples (nearest-neighbor) to the resolution of the next skip. The bottleneck between encoder and decoder consists of three ConvBlocks (residual, plain, residual).

The input is the 4-plane stack of Section 4.5.2; the output head (GroupNorm followed by a zero-initialized 3 × 3 convolution) produces the 1-channel velocity over the magnitude grid, so the untrained network starts as the identity flow.

4.6 Bridge-Flow Formulation

Training (rectified pair-flow). With the broken magnitude at $t=0$ and the clean magnitude at $t=1$, states are drawn on the straight interpolation path and the network predicts the (constant) velocity—precisely the mixing residual:

$$x_t = (1 - t) m_{\text{brk}} + t m_{\text{cln}}, \quad v = m_{\text{cln}} - m_{\text{brk}}, \quad \mathcal{L} = \|\hat{v}_\theta(x_t, t, c) - v\|_1 \quad (4.7)$$

where:

- $t \in [0, 1]$ is the bridge position: $t=0$ is the fully corrupted magnitude, $t=1$ the clean one; during training $t \sim \mathcal{U}(0, 1)$.
- x_t is the state on the straight interpolation path between the two endpoints.
- $v = m_{\text{cln}} - m_{\text{brk}}$ is the constant velocity along the path—exactly the mixing residual to be applied.

- $\hat{v}_\theta$ is the network; c collects the conditions of Section 4.5.2 (vocal magnitude, tonal-balance map, indicator plane), and the loss is the L_1 distance between predicted and true velocity.

There is no noise anywhere—inputs, targets, or sampler. Two properties follow. First, the target is fully deterministic, so the L_1 loss is signal-dominated at every t and its floor is genuine model error. Second, because the interpolation path itself is the training distribution, every state a solver can visit is in-distribution—the condition under which few-step integration is well-posed.

Inference (few-step guided ODE). Starting from the broken magnitude, an 8-step Euler solver integrates the velocity field under classifier-free guidance on the balance condition c :

$$\hat{v} = \hat{v}_\theta(x, t, \emptyset) + w (\hat{v}_\theta(x, t, c) - \hat{v}_\theta(x, t, \emptyset)), \quad x \leftarrow x + \hat{v} \Delta t, \quad \Delta t = \frac{1}{8} \quad (4.8)$$

where:

- $\emptyset$ denotes the dropped balance condition (map and indicator zeroed), i.e. the unconditional velocity estimate.
- w is the guidance strength: $w=0$ is fully unconditional, $w=1$ follows the reference balance, and $w>1$ extrapolates toward it.
- $\Delta t = 1/8$ is the Euler step of the 8-step solver; the state x is initialized at the broken magnitude ($t=0$).

Under guidance ($w=2$) the effective field varies along the path, so multi-step integration genuinely refines the estimate: 8 steps outperform a single step by 5–8 dB SI-SDR across all conditions, while 16 steps match 8 within measurement noise—the trajectory is fully resolved. Setting $w=0$ yields the unconditional model (the network chooses a plausible balance itself); intermediate w blends the model’s own judgment with the reference.

Training details. The network is optimized with AdamW (learning rate 10^{-4} , weight decay 0.01) under a cosine schedule with 1,000 warm-up steps, batch size 36, for approximately 9×10^4 steps. Corruption parameters, window positions, and

the conditioning dropout are re-sampled at every step, so no two training examples repeat.

4.7 Automatic Mixing System

A single vocal-conditioned model replaces a per-stem cascade: the mashup use case fixes the vocal and requires only the instrumental to be corrected, so the vocal enters as a condition rather than a processing target, and no mastering stage is needed—the corrected instrumental and the untouched vocal sum directly to the output mix. The model trains in a single run, and inference requires 16 network evaluations per window ($8 \text{ steps} \times 2$ for guidance).

Within the mashup pipeline, the model operates on the output of the preceding stages: the vocal and the foreign instrumental have already been tempo-aligned (Chapter 2) and structurally matched (Chapter 3), and arrive carrying the mixing decisions of two different productions. The balance condition is then computed from the *vocal’s original instrumental*—the accompaniment the vocal was produced and mixed against—so the foreign instrumental is steered toward the tonal balance and level relation under which the vocal is known to sit well. Two fallbacks complete the interface: any reference recording can substitute as the balance source when a different production style is desired, and dropping the condition entirely ($w=0$) lets the model choose a plausible balance on its own, at a modest quality cost (Table 4.2).

4.8 Evaluation

4.8.1 Baselines

We compare against two baselines: **FxNorm-Automix** [61, 62] (the pretrained `ours_S_Lb` variant, applied with vocals = clean vocal and other = broken instrumental) and **Wave-U-Net** [63, 64, 65], independently replicated and retrained on our exact task and corruption model, in two variants for fairness: without conditions (broken mix $\rightarrow$ clean mix), and with the same vocal input and tonal-balance conditioning (FiLM) as our model.

4.8.2 Results

Table 4.2. Restoration quality on 159 clips (10 internal-validation + 149 MUSDB18-HQ songs, all out-of-domain); mean $\pm$ std. Conditions: *clean* (transparency check); *EQ $\pm 4/8/12$ dB* (2–4 random parametric bells of the stated gain range, plus a broadband volume error $\mathcal{U}(-6, 6)$ dB); *volume-only* (a pure broadband level error of ± 3 –6 dB with no spectral shaping). w denotes guidance on the ground-truth balance condition; $w=0$ drops the condition. Bold marks the best method per metric within each condition (broken input excluded). FxNorm-Automix is the pretrained `ours_S_Lb` model applied to the same stems; the two Wave-U-Nets are retrained on the identical task and corruption model, without and with our conditioning.

condition	method	SI-SDR $\uparrow$	mel-MAE $\downarrow$	MR-STFT $\downarrow$	balance Δ dB $\downarrow$	LUFS Δ $\downarrow$
clean	broken input	—	0.00	0.00	0.00	0.00
	FxNorm-Automix	1.48 $\pm$ 2.39	6.67 $\pm$ 1.88	1.81 $\pm$ 0.41	6.01 $\pm$ 2.03	2.65 $\pm$ 2.23
	Wave-U-Net	17.29 $\pm$ 3.62	2.10 $\pm$ 1.02	0.74 $\pm$ 0.14	1.40 $\pm$ 0.74	1.37 $\pm$ 0.98
	Wave-U-Net (cond.)	16.93 $\pm$ 2.96	1.85 $\pm$ 0.65	0.59 $\pm$ 0.13	1.42 $\pm$ 0.88	1.43 $\pm$ 1.06
	ours ($w=0$, dropped)	24.28 $\pm$ 4.13	0.97 $\pm$ 0.47	0.23 $\pm$ 0.10	0.64 $\pm$ 0.37	0.49 $\pm$ 0.42
	ours ($w=2$, 8-step)	28.63 $\pm$ 2.70	0.50 $\pm$ 0.26	0.20 $\pm$ 0.12	0.19 $\pm$ 0.08	0.13 $\pm$ 0.13
	ours ($w=4$, 8-step)	22.56 $\pm$ 2.49	0.92 $\pm$ 0.43	0.34 $\pm$ 0.17	0.30 $\pm$ 0.12	0.19 $\pm$ 0.19
EQ ± 4 dB	broken input	15.77 $\pm$ 4.93	1.80 $\pm$ 0.93	0.40 $\pm$ 0.17	2.25 $\pm$ 1.30	2.12 $\pm$ 1.49
	FxNorm-Automix	1.42 $\pm$ 2.40	6.67 $\pm$ 1.88	1.81 $\pm$ 0.41	6.02 $\pm$ 2.04	2.66 $\pm$ 2.24
	Wave-U-Net	14.26 $\pm$ 3.02	2.50 $\pm$ 1.02	0.84 $\pm$ 0.15	1.85 $\pm$ 0.96	1.56 $\pm$ 1.21
	Wave-U-Net (cond.)	16.32 $\pm$ 2.84	2.01 $\pm$ 0.90	0.61 $\pm$ 0.16	1.56 $\pm$ 0.98	1.46 $\pm$ 1.24
	ours ($w=0$, dropped)	17.22 $\pm$ 4.01	1.62 $\pm$ 0.82	0.39 $\pm$ 0.14	1.59 $\pm$ 0.85	1.43 $\pm$ 0.98
	ours ($w=2$, 8-step)	23.71 $\pm$ 3.38	0.71 $\pm$ 0.33	0.27 $\pm$ 0.14	0.30 $\pm$ 0.18	0.23 $\pm$ 0.27
	ours ($w=4$, 8-step)	20.59 $\pm$ 2.54	1.04 $\pm$ 0.47	0.38 $\pm$ 0.17	0.38 $\pm$ 0.24	0.28 $\pm$ 0.36
EQ ± 8 dB	broken input	12.70 $\pm$ 5.19	2.99 $\pm$ 1.68	0.59 $\pm$ 0.27	2.67 $\pm$ 1.57	2.55 $\pm$ 2.19
	FxNorm-Automix	1.44 $\pm$ 2.45	6.66 $\pm$ 1.88	1.82 $\pm$ 0.41	6.03 $\pm$ 2.08	2.68 $\pm$ 2.27
	Wave-U-Net	12.62 $\pm$ 3.14	3.14 $\pm$ 1.31	0.93 $\pm$ 0.20	2.14 $\pm$ 1.07	1.68 $\pm$ 1.39
	Wave-U-Net (cond.)	15.10 $\pm$ 2.76	2.37 $\pm$ 0.94	0.67 $\pm$ 0.18	1.78 $\pm$ 1.05	1.53 $\pm$ 1.35
	ours ($w=0$, dropped)	14.75 $\pm$ 3.86	2.24 $\pm$ 1.02	0.49 $\pm$ 0.17	1.84 $\pm$ 0.94	1.55 $\pm$ 1.24
	ours ($w=2$, 8-step)	21.77 $\pm$ 4.09	0.87 $\pm$ 0.44	0.30 $\pm$ 0.15	0.38 $\pm$ 0.27	0.31 $\pm$ 0.42
	ours ($w=4$, 8-step)	19.28 $\pm$ 3.15	1.20 $\pm$ 0.57	0.40 $\pm$ 0.18	0.45 $\pm$ 0.28	0.35 $\pm$ 0.43
EQ ± 12 dB	broken input	10.64 $\pm$ 5.45	4.02 $\pm$ 2.55	0.74 $\pm$ 0.37	3.14 $\pm$ 1.81	2.94 $\pm$ 2.69
	FxNorm-Automix	1.40 $\pm$ 2.45	6.67 $\pm$ 1.90	1.82 $\pm$ 0.41	6.01 $\pm$ 2.04	2.66 $\pm$ 2.23
	Wave-U-Net	11.08 $\pm$ 3.52	3.78 $\pm$ 1.46	0.99 $\pm$ 0.19	2.40 $\pm$ 1.06	1.81 $\pm$ 1.41
	Wave-U-Net (cond.)	13.70 $\pm$ 3.13	2.92 $\pm$ 1.15	0.74 $\pm$ 0.18	2.05 $\pm$ 1.16	1.73 $\pm$ 1.41
	ours ($w=0$, dropped)	12.58 $\pm$ 4.81	3.03 $\pm$ 1.67	0.61 $\pm$ 0.25	2.25 $\pm$ 1.15	1.75 $\pm$ 1.36
	ours ($w=2$, 8-step)	20.28 $\pm$ 4.82	1.13 $\pm$ 1.00	0.35 $\pm$ 0.22	0.50 $\pm$ 0.54	0.49 $\pm$ 0.94
	ours ($w=4$, 8-step)	18.46 $\pm$ 3.87	1.37 $\pm$ 1.08	0.44 $\pm$ 0.24	0.58 $\pm$ 0.62	0.57 $\pm$ 1.05
volume-only	broken input	14.32 $\pm$ 3.56	1.39 $\pm$ 0.69	0.41 $\pm$ 0.13	2.95 $\pm$ 0.82	2.90 $\pm$ 1.08
	FxNorm-Automix	1.47 $\pm$ 2.38	6.67 $\pm$ 1.88	1.81 $\pm$ 0.41	6.01 $\pm$ 2.03	2.65 $\pm$ 2.24
	Wave-U-Net	13.73 $\pm$ 2.50	2.50 $\pm$ 0.92	0.86 $\pm$ 0.14	2.04 $\pm$ 1.07	1.65 $\pm$ 1.33
	Wave-U-Net (cond.)	16.65 $\pm$ 3.33	1.92 $\pm$ 0.83	0.61 $\pm$ 0.16	1.59 $\pm$ 1.08	1.45 $\pm$ 1.35
	ours ($w=0$, dropped)	17.09 $\pm$ 3.36	1.41 $\pm$ 0.66	0.38 $\pm$ 0.13	1.86 $\pm$ 0.85	1.83 $\pm$ 0.98
	ours ($w=2$, 8-step)	22.55 $\pm$ 2.57	0.75 $\pm$ 0.31	0.30 $\pm$ 0.14	0.28 $\pm$ 0.13	0.23 $\pm$ 0.22
	ours ($w=4$, 8-step)	20.05 $\pm$ 2.17	1.07 $\pm$ 0.48	0.39 $\pm$ 0.17	0.36 $\pm$ 0.16	0.27 $\pm$ 0.28

Three findings characterize the system beyond the headline numbers. **Transparency:** on clean input the model is near-invisible (0.50 mel-MAE; 0.19 dB balance error; SI-SDR 28.6 dB), so it can run unconditionally in a pipeline. **Volume restora-**

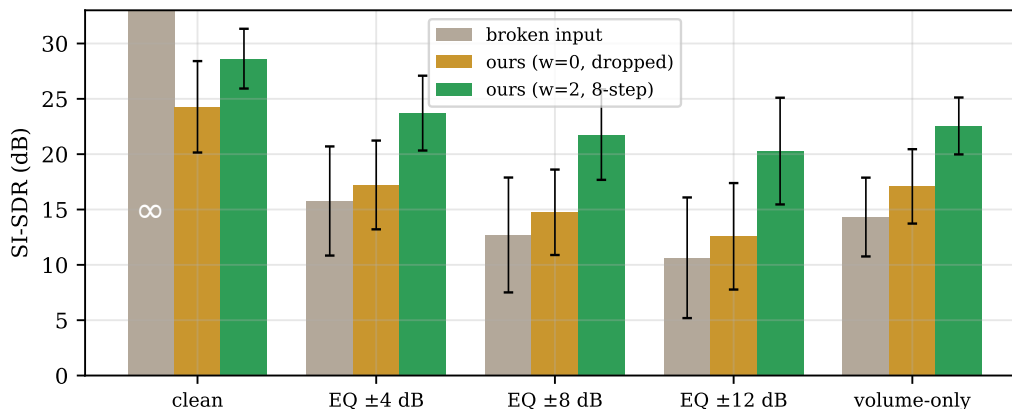


Figure 4.8. SI-SDR by corruption condition. Under the clean condition the broken input equals the reference, so its SI-SDR is unbounded (marked ∞). The steered 8-step model improves the broken input by 8–10 dB in every condition; the unconditioned model ($w=0$) improves it consistently but by less—the reference balance is worth ≈ 5 dB.

tion: on pure level errors of 3–6 dB the restored mix lands within 0.24 dB LUFS of the reference on average. **Steerability:** the model obeys the balance condition faithfully, including deliberately wrong ones: supplying a level-stripped or tone-stripped curve degrades output below the unconditioned mode, confirming the condition is load-bearing rather than vestigial (and implying that partial conditions must be trained in, not improvised at inference).

The baselines exhibit distinct failure modes. **FxNorm-Automix** scores ≈ 1.4 dB SI-SDR uniformly across all conditions—including clean input—with a ≈ 6 dB tonal-balance error: its effect-normalization chain re-processes every input toward its own trained target profile, replacing the reference mix character rather than restoring it. **Wave-U-Net** corrects part of the tonal imbalance but introduces waveform-level artifacts that cap it near a 17 dB transparency ceiling; on mild corruption it scores *below* the broken input. Supplying it with our conditioning improves it by 2–3 dB across all corrupted conditions—evidence that the conditioning contributes independently of architecture—yet it remains 6–7 dB behind the proposed model.

Figure 4.9 illustrates a restoration on an out-of-domain example with a strong corruption (EQ ± 12 dB bells and a -5 dB volume error). The mel spectrograms show the imbalanced input, the restored output, and the reference: the missing high-frequency energy and the depressed overall level are recovered without visible artifacts, and the restored tonal-balance curve tracks the reference within a fraction of a

decibel across the band while the broken curve deviates by up to 10 dB.

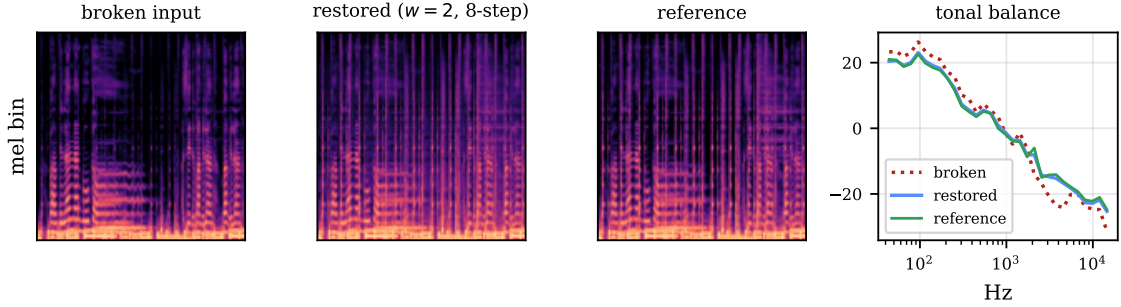


Figure 4.9. Qualitative restoration example (out-of-domain song, EQ ± 12 dB + -5 dB volume). Left to right: mel spectrogram of the broken input, the restored output ($w=2$, 8-step), and the reference mix; rightmost: the corresponding 31-band tonal-balance curves.

4.8.3 Conditioning Ablation

The balance condition of Section 4.5.2 carries two kinds of information in a single map: the tonal *shape* of the target instrumental, and its *level* relative to the vocal. To measure the contribution and the coupling of the two, we evaluate four conditioning variants on the identical corruption matrix: the full condition; the condition dropped entirely ($w=0$, the trained classifier-free fallback); a *tone-only* variant in which the map is re-centered to zero mean, stripping the level information; and a *level-only* variant in which the map is flattened to its mean, stripping the tonal shape. The partial variants are constructed at inference only—the model was trained with the condition either fully present or fully absent.

Table 4.3. Conditioning ablation, pooled over the four corrupted conditions (mean $\pm$ std over 636 clip-conditions); rightmost column: SI-SDR on clean input (transparency). Guidance $w=2$ for all provided-condition variants.

conditioning	SI-SDR $\uparrow$	mel-MAE $\downarrow$	balance Δ dB $\downarrow$	LUFS Δ $\downarrow$	clean SI-SDR $\uparrow$
full condition (tone + level)	22.08 ± 4.01	0.86 ± 0.62	0.37 ± 0.33	0.32 ± 0.56	28.63
condition dropped	15.41 ± 4.47	2.07 ± 1.28	1.88 ± 0.98	1.64 ± 1.16	24.28
tone only (level stripped)	8.41 ± 5.27	3.69 ± 2.93	8.35 ± 3.83	8.51 ± 3.89	8.79
level only (tone stripped)	2.90 ± 3.13	5.43 ± 1.93	5.31 ± 1.88	3.54 ± 1.94	2.72
broken input (reference)	13.36 ± 5.20	2.55 ± 1.93	2.75 ± 1.46	2.63 ± 1.99	—

Three findings follow from Table 4.3.

The condition is worth ≈ 6.7 dB. The full condition improves over the dropped fallback by 6.7 dB SI-SDR and reduces the balance error five-fold (0.37 vs. 1.88 dB)—the reference balance is the single largest contributor to restoration quality. The dropped fallback nonetheless improves the broken input by 2 dB and remains near-transparent on clean material, confirming that the classifier-free training produces a usable unconditional mode.

Partial conditions are worse than none. Both stripped variants fall far below the dropped fallback—and below the broken input itself. The model treats a provided map as a literal target: a tone-only map, re-centered to zero, specifies an instrumental at exactly the vocal’s level, and the model executes it, producing an 8.5 dB average loudness error. A level-only map specifies a spectrally flat instrumental, and the model duly flattens the spectrum (5.3 dB balance error). The clean-input column makes the mechanism explicit: even with nothing to correct, the stripped variants force the output away from the reference (8.8 and 2.7 dB SI-SDR against 28.6 for the full condition).

Condition semantics must be trained, not improvised. Because the model faithfully executes whatever curve it is given, inference-time edits of the conditioning map change the *task*, not the strength of guidance. Interfaces that expose partial control—tone without level, or level without tone—therefore require the corresponding ablations during training (structured conditioning dropout over the two components), a one-line extension of the current 30% all-or-nothing dropout. Until then, the supported modes are exactly two: a complete, plausible balance map, or none.

4.9 Discussion

4.9.1 Limitations

The current mix restoration model has several constraints:

1. **No spatial effects.** The model does not incorporate reverb, delay, or other spatial audio effects that are essential for comprehensive mixing when working with dry stems. The primary goal is to correct substantial imbalances in volume and frequency levels as a first step.

2. **Invertible-corruption scope.** The training corruption is an invertible gain; corruptions that destroy information (codec loss, clipping, heavy compression) require a genuinely generative target, for which the bridge formulation extends naturally with a stochastic interpolant [71].
3. **Reference dependence.** Peak steering quality requires a target balance curve; the unconditioned mode is robust but ≈ 5 dB behind. A small generative prior over balance curves given the vocal would close this gap and is left to future work.
4. **Fixed window.** The model processes 8-second windows; overlap-add or streaming inference would extend it to full songs, and the balance condition's chunked form ($31 \text{ bands} \times 4 \text{ s}$) already generalizes to arbitrary lengths.

4.10 Chapter Summary

In this chapter we presented a rectified-flow mix-restoration system for the mashup pipeline. Its contributions are: (i) a phase-reuse formulation—since production imbalance is a real positive gain, the instrumental phase is carried through unchanged and only the magnitude is learned, structurally eliminating phase artifacts; (ii) a noise-free pair-flow objective whose velocity target is exactly the mixing residual, keeping the loss signal-dominated at every timestep and every solver state on the training path; (iii) a droppable, vocal-referenced tonal-balance condition that carries both tone and level, turning classifier-free guidance into a mixing-desk control; and (iv) an out-of-domain evaluation protocol (all of MUSDB18-HQ) with a metric suite aligned to mixing practice. The steered 8-step model improves broken mixes by 8–10 dB SI-SDR across corruption types, restores loudness to within a quarter dB, and is near-transparent on clean input.

Together with the temporal alignment framework (Chapter 2) and the structural rearrangement model (Chapter 3), this mixing system completes the automated mashup pipeline—from beat synchronization to musical compatibility to sonic polish. The following chapter discusses the broader implications of this work and outlines directions for future research.

CHAPTER 5

CONCLUSIONS AND FUTURE WORK

The preceding chapters presented a modular pipeline for automated mashup generation: temporal alignment via beat extraction and local warping (Chapter 2), structural rearrangement of accompaniment segments via vocal-conditioned permutation learning (Chapter 3), and neural mix engineering via a rectified-flow restoration model (Chapter 4). Each component addresses a specific dimension of mashup quality and operates on short segments—typically 8 seconds at 120 BPM. The modular design naturally lends itself to progressive enhancement: each stage can be improved independently, and new capabilities can be integrated as additional tools within the same framework. In this chapter, we outline two directions for future development: enriching the individual editing tools with finer-grained musical control, and scaling the pipeline from short segments to full-length compositions through agent-based orchestration.

5.1 Enriching the Editing Toolkit

The current pipeline already achieves strong results within its scope—accurate vocal-conditioned rearrangement of instrumental segments, perceptually competitive mixes, and mashups that listeners frequently prefer over originals. The following extensions build on these foundations to address additional dimensions of musical quality.

5.1.1 Content-Level Accompaniment Adaptation

The segment reordering model (Chapter 3) successfully matches accompaniment segments to vocal positions based on overall spectral character. A natural next step is to go beyond rearrangement and enable content-level adaptation of the accompaniment itself—adjusting harmonic elements, modifying melodic contours, or reshaping energy envelopes to better complement the vocal at each position.

Recent advances in text-guided music editing make this increasingly practical. MelodyFlow [78] demonstrates high-fidelity localized editing of musical audio using flow matching with ODE trajectory inversion: it can modify instruments, harmonic content, or stylistic attributes in a specified region while faithfully preserving the remainder of the signal. Incorporating a similar editing capability into our pipeline would extend the system from structural alignment to harmonic alignment—for example, adjusting chord voicings in an accompaniment segment to better support the vocal melody at that position, or reshaping the dynamic envelope of an accompaniment passage to follow the vocal’s energy contour.

The reorder model’s per-position confidence scores (Section 3.6) provide a principled mechanism for targeting such edits. Positions with high confidence already exhibit strong content-level compatibility and need no modification; positions with lower confidence are natural candidates for content-level adaptation. This creates a smooth spectrum from pure rearrangement (high-confidence positions) to rearrangement plus editing (low-confidence positions), allowing the system to apply the minimum intervention needed at each point.

5.1.2 Spatial Audio and Stereo Upmixing

Professional mixes are characterized not only by frequency balance and level matching, but also by deliberate spatial placement—instruments panned across the stereo field, depth created through reverb and early reflections, and spatial separation that reduces perceptual masking between overlapping elements. Extending the mix engineering model to incorporate spatial dimensions would bring the system’s output closer to professional standards.

This direction is supported by emerging work in controllable spatial audio generation. Sun et al. [79] introduce SpatialSonic, a latent diffusion model with spatial-aware encoders that generates binaural audio with controllable source positions from text or image descriptions. Adapting similar spatial conditioning to our mix engineering stage would enable the system to learn professional spatial conventions—centering vocals, distributing instruments across the stereo panorama, and applying depth cues—as part of the mixing process rather than as a separate post-processing step.

A complementary approach is mono-to-stereo upmixing of individual separated stems prior to mixing. Source separation models typically produce mono or narrow-stereo outputs; upmixing each stem to a spatially rich stereo representation before the mixing stage would provide the mix engineering model with richer spatial material to work with, rather than requiring it to generate spatial structure from scratch.

5.2 From Segments to Full Compositions

5.2.1 Extending Beyond Fixed-Length Segments

The experiments presented in Chapters 3 and 4 operate on fixed 8-second segments, which is sufficient for developing and validating each component. A complete mashup, however, is a compact composition of approximately one minute—long enough to feature intentional structure such as energy builds, transitions, and narrative arc, while remaining concise and focused. The step from segment-level processing to a cohesive one-minute product is primarily an orchestration challenge: the individual tools already produce high-quality results at the segment level; the remaining question is how to sequence and coordinate their application across multiple segments to produce a coherent musical progression rather than a static loop.

This requires addressing two complementary problems: (1) selecting and sequencing material from both source songs to create a meaningful large-scale structure, and (2) varying the processing parameters—rearrangement strategy, mixing intensity, spatial placement, stem selection—across the composition to create dynamic progression.

5.2.2 Agent-Based Pipeline Orchestration

We propose to address these challenges through an agent-based system that treats the existing pipeline components as callable tools within a reasoning framework. Recent work on LLM-driven audio and music production demonstrates the viability and promise of this approach:

- Audio-Agent [80] uses GPT-4 to decompose complex audio tasks into se-

quences of atomic instructions, orchestrating a diffusion model for generation, editing, and composition tasks that would be difficult to specify as a single operation.

- **ComposerX** [81] employs multiple specialized LLM agents—dedicated to melody, harmony, arrangement, and review—collaborating to produce polyphonic compositions, demonstrating that multi-agent architectures can handle the inherent multi-dimensionality of music production.
- **FilmComposer** [82] simulates professional film scoring workflows through LLM-driven analysis, structural planning, and iterative music production, showing that language models can internalize domain-specific creative processes.

Applying this paradigm to mashup generation, we envision a concept we term *vibe mixing*: an intelligent agent that analyzes both source songs holistically, plans the structure of a compact one-minute mashup, and executes the plan by invoking the appropriate pipeline tools at each stage. The agent would operate through a structured workflow:

1. **Holistic analysis.** The agent analyzes both source songs using the full suite of MIR tools: structural segmentation identifies verses, choruses, and bridges; beat and downbeat tracking provides the temporal grid; chord recognition maps harmonic progressions; energy contour estimation characterizes dynamic profiles; and key/tempo metadata establishes the compatibility constraints.
2. **Structural planning.** Based on the analysis, the agent plans the mashup arrangement—aligning compatible sections from both songs (matching choruses with choruses, verses with verses), identifying natural transition points, planning energy trajectories, and deciding which stems to feature or withdraw at each point. This planning step mirrors how a human DJ mentally maps out a mashup before executing it.
3. **Section-by-section execution.** The agent executes the plan by invoking pipeline components for each section: reordering accompaniment segments,

applying content-level edits where the confidence scores indicate the need, running the mix engineering model with section-appropriate parameters, and rendering smooth transitions between adjacent blocks.

4. **Evaluation and refinement.** The agent evaluates the result using automated perceptual quality tools—such as AudioAesthetics [83] for holistic audio quality assessment and SongEval [84] for music-specific evaluation metrics—alongside the ranking model from Chapter 2. Sections that score poorly can be iteratively refined by adjusting source material selection, modifying the rearrangement, or tuning the mixing parameters.

5.2.3 Generative vs. Editing-Based Mashup Paradigms

The release of Suno’s mashup feature in January 2026 brought commercial attention to AI-driven mashup creation. The system is proprietary and no method description has been published, so what follows is an interpretation of its observable behaviour rather than an account of its design. That behaviour is consistent with a downstream application of a large pretrained music generation model: lyrics are extracted from the song contributing the vocal, the second song supplies a stylistic reference for the accompaniment, and a new track is generated conditioned on both. This is effectively a zero-to-one generation process—the output is newly synthesized audio rather than a recombination of the original recordings.

This generative paradigm has notable constraints. The feature is restricted to music created within the Suno platform, likely for two reasons: copyright considerations that preclude processing externally owned recordings, and the availability of internal metadata—exact lyrics, generation parameters, and structural annotations—that would not be accessible for arbitrary uploaded audio. Furthermore, because the output is generated rather than edited, the original performances, timbral characteristics, and production qualities of the input songs are not preserved; they serve only as stylistic references for a new generation.

The perceptual consequences follow directly. What listeners recognise as a mashup depends substantially on recognising the sources: the particular vocal performance, its timbre and phrasing, and the production signature of the accompaniment. A generated output reproduces the lyrics but not the performance, and

the style of the accompaniment but not the recording, so both sources remain recognisable in outline while neither is actually present.

Because generation is stochastic, quality varies between attempts, and an unsatisfactory result admits no diagnosis and no partial repair. The vocal cannot be kept while the accompaniment is reworked, nor a single section revised. The only available recourse is another attempt, and every attempt must satisfy two conditions at once, remaining recognisable as a combination of the two sources while also sounding good. Music production does not proceed this way. It is iterative and editorial: a producer makes a change, listens, and adjusts, and the value of a tool lies in how precisely that adjustment can be expressed. Editing is not a fallback for when generation disappoints. It is the interface through which creative work is done.

This is not a transitional limitation to be removed by training such a system directly on mashup quality. A model can only be optimised against a quantity that can be measured, and there is no corpus of correct mashups against which an output could be scored, since assembling one would require the judgment the system is meant to supply. An end-to-end system therefore optimises the likelihood of plausible music, and whether the result succeeds as a mashup is left to whatever correlates with that. A staged pipeline is under no such constraint. Each stage is optimised against a quantity that can be measured, which is what allows alignment, structural compatibility, and mix balance to be evaluated and improved on their own terms, and the perceptual ranking model of Chapter 2 supplies a quality signal over complete assemblies without any need to differentiate through them.

Our approach occupies the opposite end of this spectrum. Rather than generating new audio from scratch, we work directly with the original recordings—separating, analyzing, rearranging, and mixing the actual stems. Each processing step is transparent and individually controllable: the user can inspect which segments were rearranged and why (via the assignment matrix), evaluate the mixing quality, and intervene at any stage. The original vocal performance, instrumental timbres, and production characteristics are preserved throughout, as the pipeline edits and recombines existing material rather than regenerating it.

These two paradigms—generative synthesis and transparent editing—are complementary. The generative approach produces stylistically coherent results within its training domain, while the editing approach preserves the identity and produc-

tion quality of the original recordings with full transparency at each processing step. Together, they address different aspects of the mashup creation space, and advances in either paradigm can inform the other.

5.2.4 Toward Interactive Vibe Mixing

Such a system admits a spectrum of creative autonomy. At one end, a fully automatic mode analyzes a song pair and renders a complete mashup without intervention, enabling large-scale generation and systematic exploration of the pairing space. At the other, a mixed-initiative mode exposes the agent’s structural plan and its principal creative decisions—section pairing, transition placement, and stem selection—as editable intermediate representations. User feedback then takes the form of high-level aesthetic constraints rather than low-level parameter edits: constraints on transition character, on the sectional energy contour, or on the prominence of material from either source. The agent maps each constraint onto concrete adjustments of the underlying editing tools and re-plans the affected sections. This division of labor follows the mixed-initiative principle: the user specifies intent at the level of musical outcome, while the system is responsible for translating that intent into technically valid edit sequences.

This interactive paradigm aligns with the broader trend toward AI-assisted creative tools, where the most effective systems augment human creativity rather than replacing it. The modular architecture presented in this work is well-suited to this integration: each component accepts well-defined inputs and produces interpretable intermediate outputs (confidence scores, assignment matrices, latent representations, quality rankings) that can serve both as signals for autonomous agent reasoning and as transparent feedback for human users. The pipeline is, by design, a toolkit—and the natural next step is to provide it with an intelligent operator.

LIST OF PUBLICATIONS

The following publications resulted from the research presented in this thesis:

1. X. Wu and A. Horner, “An automated pop song mashup system using drum swapping,” *Proceedings of Meetings on Acoustics*, vol. 52, no. 1, p. 035009, 2023. doi: 10.1121/2.0001833.
2. X. Wu and A. Horner, “Learning to rank music mashups,” *Proceedings of Meetings on Acoustics*, vol. 56, no. 1, p. 035004, 2025. doi: 10.1121/2.0002123.
3. X. Wu and A. Horner, “Graph neural network guided music mashup generation,” in *Proc. 2024 IEEE International Conference on Big Data (BigData)*, Washington, DC, USA, 2024, pp. 3235–3241. doi: 10.1109/BigData62323.2024.10825542.
4. X. Wu and A. Horner, “Diffusion models for automatic music mixing,” in *Proc. 2024 IEEE International Conference on Big Data (BigData)*, Washington, DC, USA, 2024, pp. 3242–3247. doi: 10.1109/BigData62323.2024.10825221.
5. X. Wu and A. Horner, “Heard or read? Separating text shortcuts from audio perception in music multiple choice,” in *Proc. 2026 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2026, to appear.

Bibliography

- [1] C. Leider, *Digital Audio Workstation*. McGraw-Hill, 2004.
- [2] E. Jones, “The role of mashup music in creating web 2.0’s democratic promise,” *Convergence: The International Journal of Research into New Media Technologies*, vol. 27, no. 1, pp. 125–140, 2021. [Online]. Available: <https://doi.org/10.1177/1354856520983758>
- [3] J. Shiga, “Copy-and-persist: The logic of mash-up culture,” *Critical Studies in Media Communication*, vol. 24, no. 2, pp. 93–114, 2007. [Online]. Available: <https://doi.org/10.1080/07393180701262685>
- [4] J. Tot, “Multitrack mixing: An investigation into music mixing practices,” Master’s thesis, Staffordshire University, 2018. [Online]. Available: <https://doi.org/10.13140/RG.2.2.26537.49767>
- [5] R. Toulson, “Can we fix it? the consequences of ‘fixing it in the mix’ with common equalisation techniques are scientifically evaluated,” *Journal on the Art of Record Production*, no. 3, 2008.
- [6] R. Hennequin, A. Khlif, F. Voituret, and M. Moussallam, “Spleeter: a fast and efficient music source separation tool with pre-trained models,” *Journal of Open Source Software*, vol. 5, no. 50, p. 2154, 2020. [Online]. Available: <https://doi.org/10.21105/joss.02154>
- [7] S. Rouard, F. Massa, and A. Défossez, “Hybrid transformers for music source separation,” in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2023, pp. 1–5. [Online]. Available: <https://doi.org/10.1109/ICASSP49357.2023.10096956>
- [8] A. Agostinelli, T. I. Denk, Z. Borsos, J. Engel, M. Verzett, A. Caillon, Q. Huang, A. Jansen, A. Roberts, M. Tagliasacchi *et al.*, “MusicLM: Generating music from text,” *arXiv preprint arXiv:2301.11325*, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2301.11325>

- [9] J. Copet, F. Kreuk, I. Gat, T. Remez, D. Kant, G. Synnaeve, Y. Adi, and A. Défossez, "Simple and controllable music generation," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 47 704–47 720. [Online]. Available: <https://doi.org/10.52202/075280-2066>
- [10] M. E. P. Davies, P. Hamel, K. Yoshii, and M. Goto, "Automashupper: An automatic multi-song mashup system," in *International Society for Music Information Retrieval Conference*, 2013. [Online]. Available: <https://doi.org/10.5281/zenodo.1416049>
- [11] M. E. Davies, P. Hamel, K. Yoshii, and M. Goto, "Automashupper: Automatic creation of multi-song music mashups," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 22, no. 12, pp. 1726–1737, 2014. [Online]. Available: <https://doi.org/10.1109/taslp.2014.2347135>
- [12] K. Miyamura, R. Okada, and T. Nakanishi, "Automatic music mashup creation method by similarity of features," in *2021 IEEE/ACIS 19th International Conference on Computer and Information Science (ICIS)*, 2021, pp. 36–41. [Online]. Available: <https://doi.org/10.1109/ICIS51600.2021.9516600>
- [13] J.-C. Wang, J. Smith, X. Song, and Y. Wang, "Modeling the compatibility of stem tracks to generate music mashups," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, pp. 187–195, 2021. [Online]. Available: <https://doi.org/10.1609/aaai.v35i1.16092>
- [14] C.-L. Lee, Y.-T. Lin, Z.-R. Yao, F.-Y. Lee, and J.-L. Wu, "Automatic mashup creation by considering both vertical and horizontal mashabilities," in *Proceedings of the 16th ISMIR Conference*, 2015. [Online]. Available: <https://doi.org/10.5281/zenodo.1416712>
- [15] G. Bernardo and G. Bernardes, "Leveraging compatibility and diversity in computer-aided music mashup creation," *Personal and Ubiquitous Computing*, vol. 27, no. 5, pp. 1793–1809, 2023. [Online]. Available: <https://doi.org/10.1007/s00779-022-01702-z>
- [16] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, ser. Lecture Notes in Computer Science,

vol. 9351, 2015, pp. 234–241. [Online]. Available: https://doi.org/10.1007/978-3-319-24574-4_28

- [17] A. Défossez, “Hybrid spectrogram and waveform source separation,” in *Proc. Music Demixing Workshop (MDX) at ISMIR*, 2021, pp. 11–18. [Online]. Available: <https://doi.org/10.48550/arXiv.2111.03600>
- [18] K. Miyamura, R. Okada, and T. Nakanishi, “Mashuprium and timeline visualization: Implementation of 3d space and timeline visualization for automatic mashup creation,” in *2021 10th International Congress on Advanced Applied Informatics (IIAI-AAI)*, 2021, pp. 804–809. [Online]. Available: <https://doi.org/10.1109/IIAI-AAI53430.2021.00141>
- [19] B.-Y. Chen, J. B. L. Smith, and Y.-H. Yang, “Neural loop combiner: Neural network models for assessing the compatibility of loops,” in *Proc. 21st Int. Society for Music Information Retrieval Conf. (ISMIR)*, 2020, pp. 424–431. [Online]. Available: <https://doi.org/10.48550/arXiv.2008.02011>
- [20] B.-Y. Chen, W.-H. Hsu, W.-H. Liao, M. A. Martínez-Ramírez, Y. Mitsufuji, and Y.-H. Yang, “Automatic DJ transitions with differentiable audio effects and generative adversarial networks,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022, pp. 466–470. [Online]. Available: <https://doi.org/10.1109/icassp43922.2022.9746663>
- [21] A. Riou, S. Lattner, G. Hadjeres, M. Anslow, and G. Peeters, “Stem-JEPA: A joint-embedding predictive architecture for musical stem compatibility estimation,” in *International Society for Music Information Retrieval Conference (ISMIR)*, 2024. [Online]. Available: <https://doi.org/10.5281/zenodo.14877415>
- [22] M. Delabaere, L. Miqueu, M. Moreno, G. Bigois, H. Duong, E. Fernandez, F. Manent, M. Salgado-Herrera, B. Pasdeloup, N. Farrugia, and A. Marmoret, “AutoMashup: Automatic music mashups creation,” in *GRETSI Colloque Francophone de Traitement du Signal et des Images*, 2025, pp. 701–704. [Online]. Available: <https://doi.org/10.48550/arxiv.2508.06516>
- [23] F. Foscarin, J. Schlüter, and G. Widmer, “Beat this! accurate beat tracking without dbn postprocessing,” in *Proceedings of the 25th ISMIR Conference*, 2024. [Online]. Available: <https://doi.org/10.5281/zenodo.14877490>

- [24] B. McFee, “pyrubberband: Python wrapper for rubberband audio stretching library,” <https://github.com/bmcfee/pyrubberband>, 2020.
- [25] A. L. Cramer, H.-H. Wu, J. Salamon, and J. P. Bello, “Look, listen and learn more: Design choices for deep audio embeddings,” in *IEEE ICASSP*, 2019, pp. 3852–3856. [Online]. Available: <https://doi.org/10.1109/icassp.2019.8682475>
- [26] X. Wu and A. Horner, “An automated pop song mashup system using drum swapping,” *Proceedings of Meetings on Acoustics*, vol. 52, no. 1, p. 035009, 2023. [Online]. Available: <https://doi.org/10.1121/2.0001833>
- [27] —, “Learning to rank music mashups,” *Proceedings of Meetings on Acoustics*, vol. 56, no. 1, p. 035004, 2025. [Online]. Available: <https://doi.org/10.1121/2.0002123>
- [28] A. Défossez, N. Usunier, L. Bottou, and F. Bach, “Demucs: Deep extractor for music sources with extra unlabeled data remixed,” *arXiv preprint arXiv:1909.01174*, 2019. [Online]. Available: <https://doi.org/10.48550/arXiv.1909.01174>
- [29] A. Jansson, E. Humphrey, N. Montecchio, R. Bittner, A. Kumar, and T. Weyde, “Singing voice separation with deep u-net convolutional networks,” in *Proceedings of the 18th ISMIR Conference*, 2017, pp. 323–332. [Online]. Available: <https://doi.org/10.5281/zenodo.1414933>
- [30] S. Böck and M. E. P. Davies, “Deconstruct, analyse, reconstruct: How to improve tempo, beat, and downbeat estimation,” in *Proceedings of the 21st ISMIR Conference*, 2020. [Online]. Available: <https://doi.org/10.5281/zenodo.4245497>
- [31] R. Vogl, M. Dorfer, G. Widmer, and P. Knees, “Drum transcription via joint beat and drum modeling using convolutional recurrent neural networks,” in *Proceedings of the 18th ISMIR Conference*, 2017, pp. 150–157. [Online]. Available: <https://doi.org/10.5281/zenodo.1415135>
- [32] J. P. Bello, L. Daudet, S. Abdallah, C. Duxbury, M. Davies, and M. B. Sandler, “A tutorial on onset detection in music signals,” *IEEE Transactions on Speech and Audio Processing*, vol. 13, no. 5, pp. 1035–1047, 2005. [Online]. Available: <https://doi.org/10.1109/tsa.2005.851998>

- [33] E. Pampalk, P. Hlavac, and P. Herrera, "On the evaluation of perceptual similarity measures for music," in *Proc. 7th Int. Conf. Digital Audio Effects (DAFx-04)*, Naples, Italy, 2004.
- [34] A. Primavera, F. Piazza, and J. D. Reiss, "Audio morphing for percussive hybrid sound generation," in *Proc. 45th AES Int. Conf. Applications of Time-Frequency Processing in Audio*, Helsinki, Finland, 2012.
- [35] P. López-Serrano, C. Dittmar, J. Driedger, and M. Müller, "Towards modeling and decomposing loop-based electronic music," in *Proceedings of the 17th ISMIR Conference*, 2016, pp. 502–508. [Online]. Available: <https://doi.org/10.5281/zenodo.1417999>
- [36] F. Hörschläger, R. Vogl, S. Böck, and P. Knees, "Addressing tempo estimation octave errors in electronic music by incorporating style information extracted from wikipedia," in *Proceedings of the Sound and Music Computing Conference (SMC)*, 2015.
- [37] R. B. Gebhardt, M. E. P. Davies, and B. U. Seeber, "Psychoacoustic approaches for harmonic music mixing," *Applied Sciences*, vol. 6, no. 5, p. 123, 2016. [Online]. Available: <https://doi.org/10.3390/app6050123>
- [38] H. Schreiber and M. Müller, "Musical tempo and key estimation using convolutional neural networks with directional filters," in *Proceedings of the Sound and Music Computing Conference (SMC)*, 2019. [Online]. Available: <https://doi.org/10.48550/arxiv.1903.10839>
- [39] F. Korzeniowski and G. Widmer, "End-to-end musical key estimation using a convolutional neural network," in *Proc. 25th European Signal Processing Conference (EUSIPCO)*, 2017, pp. 966–970. [Online]. Available: <https://doi.org/10.23919/EUSIPCO.2017.8081351>
- [40] A. Gkiokas, V. Katsouros, G. Carayannis, and T. Stafylakis, "Music tempo estimation and beat tracking by applying source separation and metrical relations," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2012, pp. 421–424. [Online]. Available: <https://doi.org/10.1109/ICASSP.2012.6287906>

- [41] S. Böck, F. Krebs, and G. Widmer, “Accurate tempo estimation based on recurrent neural networks and resonating comb filters,” in *Proceedings of the 16th ISMIR Conference*, 2015. [Online]. Available: <https://doi.org/10.5281/zenodo.1416026>
- [42] F. Gouyon, A. Klapuri, S. Dixon, M. Alonso, G. Tzanetakis, C. Uhle, and P. Cano, “An experimental comparison of audio tempo induction algorithms,” *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 14, no. 5, pp. 1832–1844, 2006. [Online]. Available: <https://doi.org/10.1109/tsa.2005.858509>
- [43] M. Goto, “A chorus section detection method for musical audio signals and its application to a music listening station,” *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 14, no. 5, pp. 1783–1794, 2006. [Online]. Available: <https://doi.org/10.1109/tsa.2005.863204>
- [44] J. Laroche and M. Dolson, “Improved phase vocoder time-scale modification of audio,” *IEEE Transactions on Speech and Audio Processing*, vol. 7, no. 3, pp. 323–332, 1999. [Online]. Available: <https://doi.org/10.1109/89.759041>
- [45] D. Schwarz, G. Lemaitre, M. Aramaki, and R. Kronland-Martinet, “Effects of test duration in subjective listening tests,” in *Proc. Int. Computer Music Conference (ICMC)*, Utrecht, Netherlands, 2016, pp. 515–519.
- [46] R. Arandjelović and A. Zisserman, “Look, listen and learn,” in *IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 609–617. [Online]. Available: <https://doi.org/10.1109/iccv.2017.73>
- [47] B. McFee, C. Raffel, D. Liang, D. P. W. Ellis, M. McVicar, E. Battenberg, and O. Nieto, “librosa: Audio and music signal analysis in python,” in *Proceedings of the 14th Python in Science Conference (SciPy)*, 2015, pp. 18–25. [Online]. Available: <https://doi.org/10.25080/majora-7b98e3ed-003>
- [48] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, 2017. [Online]. Available: <https://doi.org/10.48550/arXiv.1706.03762>

- [49] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," in *International Conference on Machine Learning (ICML)*, 2021, pp. 8748–8763. [Online]. Available: <https://doi.org/10.48550/arxiv.2103.00020>
- [50] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186. [Online]. Available: <https://doi.org/10.48550/arxiv.1810.04805>
- [51] R. Sinkhorn, "A relationship between arbitrary positive matrices and doubly stochastic matrices," *The Annals of Mathematical Statistics*, vol. 35, no. 2, pp. 876–879, 1964. [Online]. Available: <https://doi.org/10.1214/aoms/1177703591>
- [52] M. Cuturi, "Sinkhorn distances: Lightspeed computation of optimal transport," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.
- [53] G. Mena, D. Belanger, S. Linderman, and J. Snoek, "Learning latent permutations with gumbel-sinkhorn networks," in *International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: <https://doi.org/10.48550/arxiv.1802.08665>
- [54] M. Noroozi and P. Favaro, "Unsupervised learning of visual representations by solving jigsaw puzzles," in *European Conference on Computer Vision (ECCV)*, 2016, pp. 69–84. [Online]. Available: https://doi.org/10.1007/978-3-319-46466-4_5
- [55] I. Misra, C. L. Zitnick, and M. Hebert, "Shuffle and learn: Unsupervised learning using temporal order verification," in *European Conference on Computer Vision (ECCV)*, 2016, pp. 527–544. [Online]. Available: https://doi.org/10.1007/978-3-319-46448-0_32
- [56] B. De Man, J. D. Reiss, and R. Stables, "Ten years of automatic mixing," in *Proc. 3rd Workshop on Intelligent Music Production (WIMP)*, 2017.
- [57] U. Zölzer, Ed., *DAFX: Digital Audio Effects*, 2nd ed. Chichester, UK: John Wiley & Sons, 2011. [Online]. Available: <https://doi.org/10.1002/9781119991298>

- [58] T. Wilmering, D. Moffat, A. Milo, and M. B. Sandler, "A history of audio effects," *Applied Sciences*, vol. 10, no. 3, p. 791, 2020. [Online]. Available: <https://doi.org/10.3390/app10030791>
- [59] B. De Man and J. D. Reiss, "A knowledge-engineered autonomous mixing system," in *Proc. 135th Convention of the Audio Engineering Society*, New York, NY, USA, 2013.
- [60] F. Everardo, "Towards an automated multitrack mixing tool using answer set programming," in *Proc. 14th Sound and Music Computing Conference (SMC)*, Espoo, Finland, 2017, pp. 422–428. [Online]. Available: <https://doi.org/10.5281/zenodo.1401980>
- [61] M. A. Martínez-Ramírez, W.-H. Liao, G. Fabbro, S. Uhlich, C. Nagashima, and Y. Mitsufuji, "Automatic music mixing with deep learning and out-of-domain data," in *Proc. 23rd Int. Society for Music Information Retrieval Conf. (ISMIR)*, 2022, pp. 411–418. [Online]. Available: <https://doi.org/10.48550/arXiv.2208.11428>
- [62] A. Pandey and D. Wang, "Tcnn: Temporal convolutional neural network for real-time speech enhancement in the time domain," in *IEEE ICASSP*, 2019, pp. 6875–6879. [Online]. Available: <https://doi.org/10.1109/ICASSP.2019.8683634>
- [63] D. Koszewski, T. Görne, G. Korvel, and B. Kostek, "Automatic music signal mixing system based on one-dimensional wave-u-net autoencoders," *EURASIP Journal on Audio, Speech, and Music Processing*, vol. 2023, no. 1, p. 1, 2023. [Online]. Available: <https://doi.org/10.1186/s13636-022-00266-3>
- [64] D. Stoller, S. Ewert, and S. Dixon, "Wave-u-net: A multi-scale neural network for end-to-end audio source separation," in *Proceedings of the 19th ISMIR Conference*, 2018. [Online]. Available: <https://doi.org/10.48550/arxiv.1806.03185>
- [65] M. A. Martínez Ramírez, D. Stoller, and D. Moffat, "A deep learning approach to intelligent drum mixing with the wave-u-net," *Journal of the Audio Engineering Society*, vol. 69, no. 3, pp. 142–151, 2021. [Online]. Available: <https://doi.org/10.17743/jaes.2020.0031>

- [66] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems*, 2020. [Online]. Available: <https://doi.org/10.48550/arXiv.2006.11239>
- [67] S. Welker, J. Richter, and T. Gerkmann, “Speech enhancement with score-based generative models in the complex stft domain,” in *Proceedings of Interspeech 2022*, 2022, pp. 2928–2932. [Online]. Available: <https://doi.org/10.21437/Interspeech.2022-10653>
- [68] J.-M. Lemerrier, J. Richter, S. Welker, E. Moliner, V. Välimäki, and T. Gerkmann, “Diffusion models for audio restoration: A review,” *IEEE Signal Processing Magazine*, vol. 41, no. 6, pp. 72–84, 2024. [Online]. Available: <https://doi.org/10.1109/MSP.2024.3445871>
- [69] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2210.02747>
- [70] X. Liu, C. Gong, and Q. Liu, “Flow straight and fast: Learning to generate and transfer data with rectified flow,” in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: <https://doi.org/10.48550/arxiv.2209.03003>
- [71] M. S. Albergo, N. M. Boffi, and E. Vanden-Eijnden, “Stochastic interpolants: A unifying framework for flows and diffusions,” *arXiv preprint arXiv:2303.08797*, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2303.08797>
- [72] G.-H. Liu, A. Vahdat, D.-A. Huang, E. A. Theodorou, W. Nie, and A. Anandkumar, “I2SB: Image-to-image schrödinger bridge,” in *International Conference on Machine Learning (ICML)*, 2023.
- [73] C. Raffel, B. McFee, E. J. Humphrey, J. Salamon, O. Nieto, D. Liang, and D. P. W. Ellis, “mir_eval: A transparent implementation of common mir metrics,” in *Proceedings of the 15th ISMIR Conference*, 2014.
- [74] J. Le Roux, S. Wisdom, H. Erdogan, and J. R. Hershey, “SDR – half-baked or well done?” in *IEEE International Conference on Acoustics, Speech*

- and *Signal Processing (ICASSP)*, 2019, pp. 626–630. [Online]. Available: <https://doi.org/10.1109/icassp.2019.8683855>
- [75] Z. Rafii, A. Liutkus, F.-R. Stöter, S. I. Mimilakis, and R. Bittner, “Musdb18-hq – an uncompressed version of musdb18,” 2019. [Online]. Available: <https://doi.org/10.5281/zenodo.3338373>
- [76] J. Ho and T. Salimans, “Classifier-free diffusion guidance,” *arXiv preprint arXiv:2207.12598*, 2022. [Online]. Available: <https://doi.org/10.48550/arXiv.2207.12598>
- [77] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *IEEE CVPR*, 2016, pp. 770–778. [Online]. Available: <https://doi.org/10.1109/CVPR.2016.90>
- [78] G. Le Lan, B. Shi, Z. Ni, S. Srinivasan, A. Kumar, B. Ellis, D. Kant, V. Nagaraja, E. Chang, W.-N. Hsu, Y. Shi, and V. Chandra, “High fidelity text-guided music editing via single-stage flow matching,” *arXiv preprint arXiv:2407.03648*, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2407.03648>
- [79] P. Sun, S. Cheng, X. Li, Z. Ye, H. Liu, H. Zhang, W. Xue, and Y. Guo, “Both ears wide open: Towards language-driven spatial audio generation,” *arXiv preprint arXiv:2410.10676*, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2410.10676>
- [80] Z. Wang, C.-K. Tang, and Y.-W. Tai, “Audio-agent: Leveraging llms for audio generation, editing and composition,” *arXiv preprint arXiv:2410.03335*, 2024. [Online]. Available: <https://doi.org/10.48550/arXiv.2410.03335>
- [81] Q. Deng, Q. Yang, R. Yuan *et al.*, “Composerx: Multi-agent symbolic music composition with llms,” in *Proc. 25th Int. Society for Music Information Retrieval Conf. (ISMIR)*, 2024, pp. 669–679. [Online]. Available: <https://doi.org/10.48550/arXiv.2404.18081>
- [82] Z. Xie, Q. He, Y. Zhu, Q. He, and M. Li, “Filmcomposer: Llm-driven music production for silent film clips,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2025, pp. 13 519–13 528. [Online]. Available: <https://doi.org/10.1109/CVPR52734.2025.01262>

- [83] A. Tjandra, Y.-C. Wu, B. Guo, J. Hoffman, B. Ellis, A. Vyas, B. Shi, S. Chen, M. Le, N. Zacharov, C. Wood, A. Lee, and W.-N. Hsu, "Meta audiobox aesthetics: Unified automatic assessment for speech, music and sound," in *Proc. IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, 2025, pp. 1–8. [Online]. Available: <https://doi.org/10.1109/ASRU65441.2025.11434623>
- [84] J. Yao *et al.*, "Songeval: A benchmark dataset for song aesthetics evaluation," *arXiv preprint arXiv:2505.10793*, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2505.10793>